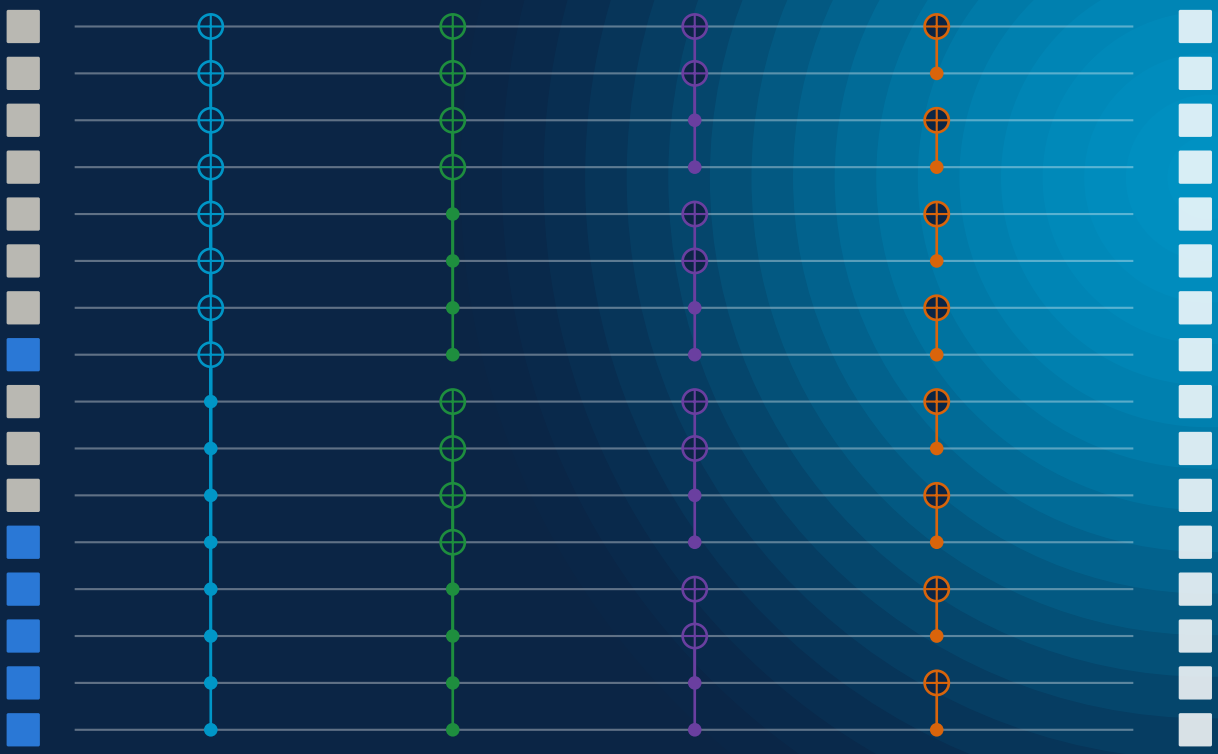


A COMPLETE VISUAL AND MATHEMATICAL GUIDE

POLAR CODES

From Zero to Expert

Channel polarization • Encoding • SC, SCL and CRC-aided decoding • 5G NR • Block diagrams • Tested Python and MATLAB code • 15 chapters, solved problems and a final project

Mahmoud Abdelaziz

Military Technical College, Cairo

Version 2 • Python + MATLAB • 2026

Polar Codes: From Zero to Expert — A Complete Visual and Mathematical Guide

Version 2, October 2026 (one volume with Python and MATLAB code and block diagrams).

All simulation results in this book were produced with the Python code printed in Chapter 11 and Chapter 15 (MATLAB version in Appendix D) (NumPy, fixed random seeds). They are Monte-Carlo estimates, not theoretical bounds; re-running the code reproduces them. Specification details of 3GPP NR are summarised for teaching; always consult the current release of TS 38.212 for implementation work.

Typeset with XeLaTeX. Figures drawn with Ti*k*Z and Matplotlib.

How to Use This Book

This book takes you from *zero* — you do not need to know what a channel code is — to the level of an engineer who can design, simulate and explain the polar codes used in 5G. Every topic is taught in the same seven steps:



The colour code of the boxes

Definition 0.1 • Definitions are cyan
A precise statement of a new term. Learn these by heart.

Intuition
Gold boxes explain the idea in everyday words, before any mathematics.

Worked Example 0.1 • Worked examples are green
Real numbers, every step shown. Re-do them with a pencil.

Advanced • Purple is for advanced material
You may skip purple boxes on a first reading.

Important note
Orange boxes hold facts that are easy to forget but important in practice.

! Common mistake
Red boxes show errors that students (and engineers!) really make.

Professional insight
Grey boxes connect the theory to real systems and current research.

Conventions used in the whole book

Polar-code books differ in small conventions, and mixing them is the most common source of confusion. This book fixes them once:

Kernel	$\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$
Generator matrix	$\mathbf{G}_N = \mathbf{F}^{\otimes n}$, $N = 2^n$, *without* bit-reversal (same as 3GPP NR)
Encoding	row vectors over GF(2): $\mathbf{x} = \mathbf{u}\,\mathbf{G}_N$
Indices	bits are numbered $0, 1, \ldots, N-1$ in natural order
Information / frozen sets	$\mathcal{A}$ (size K) and $\mathcal{F}$ (size $N-K$); frozen bits are 0
BPSK	bit $0 \mapsto +1$, bit $1 \mapsto -1$; symbol energy $E_s = 1$
LLR	$L = \ln \dfrac{P(\text{bit}=0 \mid y)}{P(\text{bit}=1 \mid y)}$; positive favours 0
Noise	real AWGN, variance $\sigma^2 = N_0/2$, $E_s/N_0 = R\,E_b/N_0$
Colours of bits	1 information 0 frozen 1 CRC 1 coded

The software

All Python code is in one small library, `polar.py`, printed in full in Chapter 11. It was tested against brute-force calculations (the SC decoder reproduces the exact bit-channel LLRs of Arıkan's definition, and the list decoder with a large list returns the maximum-likelihood codeword). All plots of error rates are Monte-Carlo results produced by that code with fixed random seeds, so you can reproduce every curve.

A complete MATLAB version of the same library is printed in Appendix D. Every Python function has a MATLAB file with the same name and arguments, and the two libraries make identical decisions on the same LLRs, so you can follow the book in either language.

What is new in version 2

- **One book, two languages.** The MATLAB code, which was a separate companion file in version 1, is now Appendix D of this book, with its outputs and the verification against Python.
- **Block diagrams for every system.** Sixteen new diagrams, marked "**Block diagram.**" in their captions, show each system as engineers draw it: coded versus uncoded links, the polar transmitter, combining and splitting, the recursive encoder, the SC, SCL and CA-SCL decoders, the CRC shift register, systematic encoding, Fast-SSC, adaptive SCL, GA construction, NR rate matching, the software structure, the Monte-Carlo loop and the SNR normalisation chain.

The theory, the worked examples and all simulation results are unchanged.

Contents

How to Use This Book iii

I Beginner Level: Digital Communication Foundations

1 Introduction to Digital Communication 2

1.1 What is digital communication? 2
1.2 Transmitter, channel and receiver 3
1.3 What causes transmission errors? 3
1.4 Bit error rate (BER) 4
1.5 Block error rate (BLER) 6
1.6 Why error-correcting codes are necessary. 7
1.7 Simple real-life examples 7
1.8 Chapter summary 7

2 Fundamentals of Channel Coding 10

2.1 What is channel coding?. 10
2.2 Codewords, block codes and code rate 11
2.3 Hamming distance 12
2.4 Linear block codes and generator matrices 12
2.5 Three channel models 13
2.6 Shannon channel capacity 14
2.7 Why polar codes are important 15
2.8 Chapter summary 15

II Intermediate Level: Understanding Polar Codes

3 Introduction to Polar Codes 19

3.1 What are polar codes? 19
3.2 History: Arıkan's 2009 breakthrough 20

3.3 Why were polar codes invented? . . . 20
3.4 Channel polarization explained intuitively . . . 21
3.5 Reliable and unreliable bit-channels . . . 21
3.6 Frozen bits and information bits . . . 22
3.7 Block length, information length and code rate . . . 23
3.8 Capacity-achieving behaviour . . . 23
3.9 Polar codes versus LDPC and turbo codes . . . 24
3.10 Chapter summary . . . 24

4 Channel Polarization — The Core Idea . . . 26

4.1 Channel combining . . . 26
4.2 Channel splitting . . . 27
4.3 The binary erasure channel: an exact example . . . 28
4.4 Visualizing polarization for N = 2, 4, 8, 16 . . . 29
4.5 The Bhattacharyya parameter . . . 30
4.6 Why do channels polarize? . . . 31
4.7 Chapter summary . . . 32

5 Polar Code Encoding . . . 35

5.1 The polar encoding kernel . . . 35
5.2 The Kronecker product . . . 36
5.3 Frozen-bit placement and information-bit selection . . . 36
5.4 Recursive encoding . . . 37
5.5 Encoding stages and butterfly diagrams . . . 37
5.6 Encoding complexity . . . 39
5.7 Chapter summary . . . 40

III Advanced Level: Decoding Algorithms

6 Successive Cancellation (SC) Decoding . . . 43

6.1 Why decoding is necessary . . . 43
6.2 The successive cancellation concept . . . 43
6.3 Likelihood ratios and log-likelihood ratios . . . 44
6.4 The f and g functions . . . 45
6.5 The SC decoding tree . . . 46
6.6 Hard decisions and frozen-bit handling . . . 47
6.7 Step-by-step numerical decoding example . . . 48
6.8 A simple SC decoder in Python . . . 49

6.9 Computational complexity . . . 49
6.10 Advantages and limitations . . . 50
6.11 Chapter summary . . . 50

7 Successive Cancellation List (SCL) Decoding . . . 53

7.1 Why SC decoding is sometimes insufficient . . . 53
7.2 Path splitting . . . 54
7.3 Path metrics . . . 54
7.4 Candidate pruning . . . 55
7.5 Worked SCL decoding example . . . 56
7.6 SCL versus SC performance . . . 57
7.7 Complexity and memory requirements . . . 58
7.8 Chapter summary . . . 58

8 CRC-Aided SCL Decoding . . . 60

8.1 What is a CRC? . . . 60
8.2 How a CRC helps polar decoding . . . 62
8.3 CRC attachment . . . 62
8.4 CRC-based candidate selection . . . 63
8.5 Complete transmitter and receiver workflow . . . 63
8.6 Practical examples . . . 63
8.7 The price of the CRC . . . 65
8.8 Chapter summary . . . 65

9 Additional Professional Topics . . . 67

9.1 Systematic polar encoding . . . 67
9.2 Belief propagation (BP) decoding . . . 68
9.3 SCAN decoding . . . 68
9.4 Fast-SSC decoding . . . 69
9.5 Adaptive SCL decoding . . . 70
9.6 Polar-code construction methods . . . 70
9.7 Puncturing, shortening and repetition . . . 72
9.8 Rate matching . . . 73
9.9 Finite-length performance . . . 73
9.10 Hardware implementation considerations . . . 74
9.11 Chapter summary . . . 74

IV Professional Level: 5G and Practical Implementation

10 Polar Codes in 5G NR . . . 78

10.1 Why polar codes are used in 5G . . . 78
10.2 Polar coding of control information . . . 79
10.3 CRC processing . . . 79
10.4 Interleaving . . . 79
10.5 Reliability sequences . . . 80
10.6 Rate matching . . . 80
10.7 Parity-check bits for small UCI . . . 82
10.8 The complete encoding and decoding chain . . . 82
10.9 Textbook polar coding versus 5G NR processing . . . 83
10.10 Relevant 3GPP specifications . . . 83
10.11 Chapter summary . . . 83

11 Python Implementation and Simulation . . . 86

11.1 Organisation of the software . . . 86
11.2 Module header . . . 87
11.3 Program 1 — Polar encoder . . . 87
11.4 Program 2 — Frozen-bit selection . . . 88
11.5 Programs 6 and 7 — BPSK modulation and the AWGN channel . . . 90
11.6 The f and g functions . . . 91
11.7 Programs 3, 4 and 5 — SC, SCL and CA-SCL decoders . . . 91
11.8 CRC . . . 93
11.9 Program 8 — BER and BLER calculation . . . 94
11.10 Demonstration with outputs . . . 95
11.11 Program 9 — BER vs E_b/N_0 graphs . . . 97
11.12 Program 10 — Comparing block lengths and code rates . . . 97
11.13 How the code was tested . . . 98
11.14 Chapter summary . . . 99

12 Performance Analysis and Engineering Design . . . 101

12.1 BER and BLER curves . . . 101
12.2 E_b/N_0 and SNR . . . 102
12.3 Coding gain . . . 102
12.4 Decoder latency . . . 103
12.5 Computational complexity and memory . . . 103
12.6 SC vs SCL vs CRC-SCL . . . 104
12.7 Comparison with LDPC codes . . . 104
12.8 Practical design trade-offs . . . 104
12.9 Current research directions . . . 105

12.10 Chapter summary . . . 105

V Expert Practice

13 Fully Solved Problems . . . 109

13.1 Basic level . . . 109
13.2 Intermediate level . . . 110
13.3 Advanced level . . . 111
13.4 Summary of methods . . . 113

14 Interview and Examination Questions . . . 114

14.1 Beginner level . . . 114
14.2 Intermediate level . . . 115
14.3 Advanced level . . . 116
14.4 Engineering interview level . . . 117
14.5 How to prepare . . . 118

15 Final Professional Project . . . 119

15.1 Project specification . . . 119
15.2 The complete program . . . 120
15.3 Step-by-step numerical example (one frame) . . . 121
15.4 Monte-Carlo results . . . 122
15.5 Performance interpretation . . . 123
15.6 Extensions for students and researchers . . . 123
15.7 Project checklist . . . 124
15.8 Final words . . . 124

Appendix A — Mathematical Prerequisites . . . 125

Appendix B — Glossary . . . 127

Appendix C — Bibliography . . . 129

Appendix D — MATLAB Implementation . . . 131

D.1 Overview and requirements . . . 131
D.2 Conventions and the 1-based index rule . . . 132
D.3 Program 1 — Polar encoder . . . 132
D.4 Program 2 — Frozen-bit selection . . . 133
D.5 Programs 3, 4 and 5 — SC, SCL and CA-SCL decoder . . . 135
D.6 CRC functions . . . 137
D.7 Programs 6, 7 and 8 — BPSK, AWGN and BER/BLER . . . 138
D.8 Demonstration with outputs . . . 139

D.9 Programs 9 and 10 — BER/BLER figures, block lengths and rates 140
D.10 Final project in MATLAB 142
D.11 Self-test script 143
D.12 Verification against the Python library 145
D.13 Common MATLAB mistakes 146

PART I

Beginner Level: Digital Communication Foundations

Bits, channels, noise and errors. Why we must add redundancy, and how much the channel allows. No prior knowledge is assumed.

CHAPTER

1



Introduction to Digital Communication

IN THIS CHAPTER YOU WILL LEARN

- what a digital communication system is;
- the roles of transmitter, channel and receiver;
- why bits arrive with errors;
- the AWGN channel and BPSK modulation;
- how to compute the bit error rate $Q(\sqrt{2E_b/N_0})$;
- BER and BLER, and how they are related;
- why error-correcting codes are unavoidable.

1.1 What is digital communication?

Intuition

Imagine you must tell a friend across a noisy football stadium whether to meet at Gate A or Gate B. You agree beforehand: one raised arm means A, two raised arms means B. You have turned your message into a small set of agreed *symbols*. Even if the view is blurred, your friend only has to decide between two possibilities — not guess a whole sentence. This is the core idea of *digital* communication: every message is first turned into a sequence of symbols from a small alphabet, usually the two symbols 0 and 1.

Definition 1.1 • Digital communication

Digital communication is the transfer of information as a sequence of symbols taken from a *finite* alphabet. In almost all modern systems the alphabet is binary, $\{0, 1\}$, and each symbol is called a **bit** (*binary digit*).

Voice, images and video are not naturally binary. A **source encoder** (for example a speech codec in a phone, or JPEG for images) converts them into bits and removes useless repetition. From then on, the communication system only sees bits and does not care what they mean. This separation is very powerful: the same 5G radio carries a voice call, a web page and a video without knowing the difference.

1.1.1 Why digital and not analogue?

- **Regeneration.** A noisy digital signal can be *decided* and re-created perfectly at each repeater; analogue noise accumulates.
- **Error control.** We can add clever redundancy and correct errors — the topic of this book.
- **Integration.** All kinds of data become one bit stream that can be encrypted, compressed, stored and processed by cheap digital chips.

1.2 Transmitter, channel and receiver

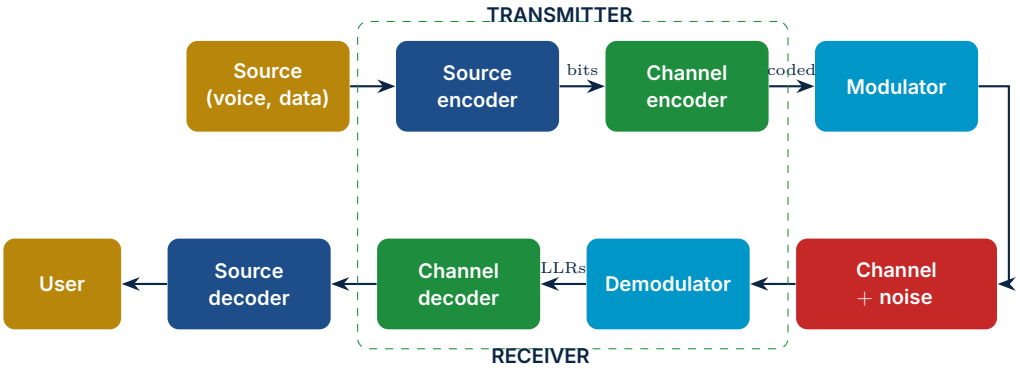


Figure 1.1. Block diagram of a digital communication system. Polar codes live in the green blocks (dashed box).

Fig. 1.1 shows the classic chain proposed by Shannon in 1948.

- The **transmitter** prepares the bits for the physical medium. The **channel encoder** adds controlled redundancy; the **modulator** turns bits into a physical waveform (a voltage, a light pulse, a radio wave).
- The **channel** is everything between the transmitter and the receiver: copper, optical fibre, air, the electronics of the radio, even a hard disk (storage is communication *in time*).
- The **receiver** undoes the transmitter operations. The **demodulator** measures the waveform and gives the decoder a *soft* number per bit (we will call it an LLR); the **channel decoder** uses the redundancy to find and correct errors.

1.3 What causes transmission errors?

A bit error happens when the receiver decides 1 while 0 was sent, or the opposite. The physical causes are many:

Thermal noise	random motion of electrons in every resistor and amplifier; always present.
Interference	signals of other users, neighbouring cells, other systems in the same band.
Fading	the radio wave arrives over many paths that sometimes cancel each other.
Distortion	filters and amplifiers that are not ideal; inter-symbol interference.
Synchronisation errors	small mistakes in timing, frequency or phase.

1.3.1 Noise and the AWGN model

For learning and for most code design we use one simple, very accurate model of thermal noise.

Definition 1.2 • AWGN channel

In the **additive white Gaussian noise** (AWGN) channel the received value is

$$y = s + w, \qquad w \sim \mathcal{N}(0, \sigma^2),$$

where s is the transmitted amplitude and w is a Gaussian random variable with mean 0 and variance $\sigma^2 = N_0/2$. *Additive*: the noise is added. *White*: noise samples at different times are independent. *Gaussian*: the noise has the bell-shaped probability density

$$p(w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{w^2}{2\sigma^2}\right).$$

The Gaussian shape is not a guess: thermal noise is the sum of a huge number of tiny independent effects, and the central limit theorem says such a sum is Gaussian.

1.3.2 BPSK: the simplest modulation

Definition 1.3 • BPSK

Binary phase-shift keying maps each bit to one amplitude: $0 \mapsto s = +1$, $1 \mapsto s = -1$. In formula form $s = 1 - 2c$ for code bit $c \in \{0, 1\}$. The energy per symbol is $E_s = s^2 = 1$.

The receiver sees $y = \pm 1 + w$. The natural decision is: say 0 if $y > 0$ and 1 if $y < 0$. Fig. 1.2 shows the two possible probability densities of y; the shaded tails are the error events.

1.4 Bit error rate (BER)

1.4.1 Derivation of the BPSK error probability

Assume bit 0 was sent, so $y = 1 + w$. An error happens when $y < 0$, that is when $w < -1$:

$$\begin{aligned} P_b &= P(w < -1) = P(w > 1) \qquad \text{(the Gaussian is symmetric)} \\ &= \int_1^\infty \frac{1}{\sqrt{2\pi\sigma^2}} e^{-w^2/2\sigma^2}\, dw = \int_{1/\sigma}^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt \qquad (t = w/\sigma) \\ &= Q\left(\frac{1}{\sigma}\right). \end{aligned} \tag{1.1}$$

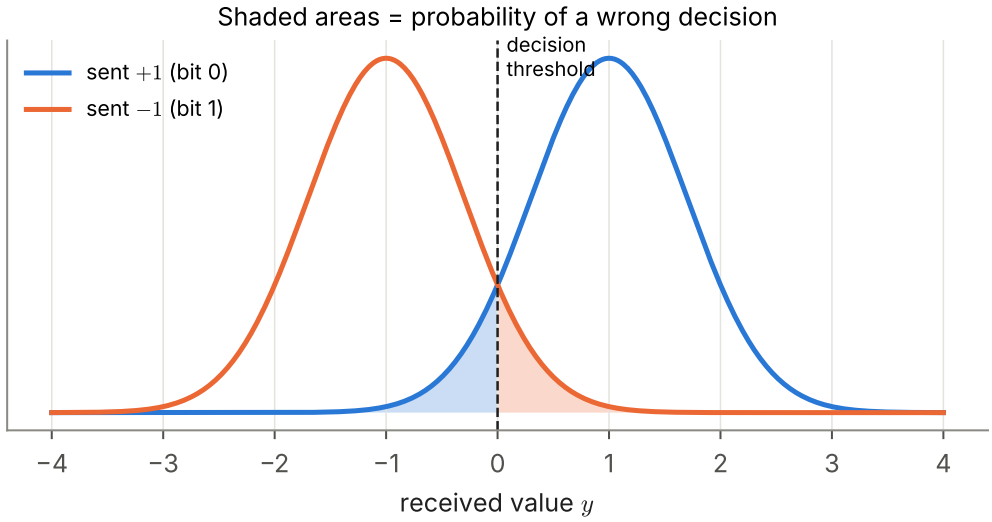


Figure 1.2. Densities of the received value for BPSK in AWGN ($\sigma = 0.7$).

Here $Q(x) = \int_x^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$ is the tail probability of a standard Gaussian. By symmetry the same result holds when 1 is sent.

Engineers prefer to write this with the **energy per information bit** E_b and the noise level N_0. Without coding each symbol carries one bit, so $E_b = E_s = 1$, and $\sigma^2 = N_0/2$ gives $1/\sigma = \sqrt{2E_b/N_0}$:

$$P_b^{\text{BPSK}} = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) \qquad \text{(uncoded BPSK over AWGN).}$$

The ratio E_b/N_0 is usually given in decibels: $(E_b/N_0)_{\text{dB}} = 10 \log_{10}(E_b/N_0)$.

Worked Example 1.1 • BER of uncoded BPSK

Question. Find the BER of uncoded BPSK at $E_b/N_0 = 4\,\text{dB}$.
Step 1 — leave decibels. $E_b/N_0 = 10^{4/10} = 2.512$.
Step 2 — argument of Q. $\sqrt{2 \times 2.512} = \sqrt{5.024} = 2.241$.
Step 3 — evaluate. $Q(2.241) = 0.0125$. So about one bit in 80 is wrong.
Repeating the steps: $6.8\,\text{dB} \Rightarrow P_b \approx 9.9 \times 10^{-4}$ and $9.6\,\text{dB} \Rightarrow P_b \approx 9.7 \times 10^{-6}$. To divide the error rate by 100 we need almost 3 dB more power — about twice the power!

Definition 1.4 • Bit error rate

The **bit error rate** (BER) is the fraction of information bits that are decoded wrongly:

$$\text{BER} = \frac{\text{number of wrong bits}}{\text{number of transmitted bits}}.$$

In simulation we estimate it by counting; in theory it is the probability P_b.

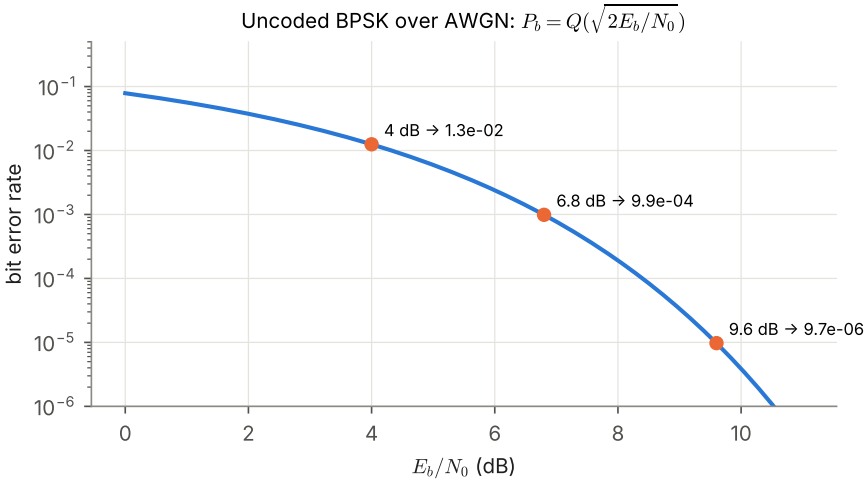


Figure 1.3. BER of uncoded BPSK. The curve falls slowly: power is an expensive way to buy reliability.

1.5 Block error rate (BLER)

Real systems send data in *blocks* (packets, frames, transport blocks). One wrong bit is usually enough to make the whole block useless, because the block's checksum (CRC) fails and the block is thrown away or re-transmitted.

Definition 1.5 • Block error rate

The **block error rate** (BLER), also called frame error rate (FER), is

$$\text{BLER} = \frac{\text{number of blocks with at least one wrong bit}}{\text{number of transmitted blocks}}.$$

If K bits are in error *independently* with probability p, the block is correct only if all K bits are correct:

$$\text{BLER} = 1 - (1-p)^K \;\approx\; Kp \quad (\text{when } Kp \ll 1).$$

Worked Example 1.2 • From BER to BLER

A packet has $K = 1000$ bits and the raw BER is $p = 10^{-3}$.

$$\text{BLER} = 1 - (1-10^{-3})^{1000} = 1 - e^{1000\ln(0.999)} = 1 - e^{-1.0005} = 0.632.$$

Almost two packets out of three are lost, although only one bit in a thousand is wrong! With $K = 100$ the BLER is 0.095. This simple calculation is the strongest argument for error-correcting codes.

BER or BLER?

5G control channels and most modern standards specify targets as BLER (for example 10^{-2} or 10^{-3} for control information). Coded errors are *not* independent — a decoding failure usually breaks several bits together — so the formula $1-(1-p)^K$ is only for uncoded or independent errors. For coded systems we measure BLER directly.

1.6 Why error-correcting codes are necessary

There are only two ways to make communication more reliable:

1. **More power.** Fig. 1.3 shows this is expensive: from BER 10^{-3} to 10^{-5} costs about 2.8 dB (almost 2× power). Batteries, interference to other users and regulation all limit power.
2. **Redundancy (coding).** Send a few extra, cleverly computed bits so that the receiver can detect and correct errors. A good code at rate 1/2 reaches a BLER of 10^{-3} at roughly 2–3 dB, where uncoded BPSK would still make several percent bit errors.

The improvement is called **coding gain** and is measured in dB (Chapter 12). In practice coding gains of 5–8 dB are common — this is equivalent to multiplying the transmitter power by 3 to 6.

1.7 Simple real-life examples

- **Mobile phone call.** Your voice is coded at a few kbit/s, protected by channel codes and sent in short blocks every few milliseconds. In 5G the *control* part of this traffic (who transmits, when, with which settings) is protected by polar codes.
- **Deep-space probes.** Signals from Voyager arrive with power far below a billionth of a watt. Without very strong codes no picture of Neptune would exist.
- **QR codes and CDs.** A scratched CD still plays because of Reed–Solomon codes; a dirty QR code still scans for the same reason.
- **Wi-Fi and 4G/5G data.** Convolutional, turbo and LDPC codes protect the data channels.

Professional insight

A modern receiver does not send hard bits (0/1) to the decoder. It sends *soft* values that say how sure it is (the LLRs of Chapter 6). A value of $y = 0.05$ is a weak "0"; a value of $y = 1.8$ is a strong "0". Using this soft information gives about 2 dB extra gain over hard decisions — free gain that every good decoder, including all polar decoders in this book, uses.

1.8 Chapter summary

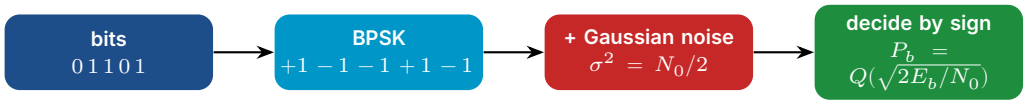
Key concepts

- A digital system sends bits; channel coding adds controlled redundancy against errors.
- Thermal noise is modelled as additive white Gaussian noise with variance $\sigma^2 = N_0/2$.
- BPSK maps $0 \to +1$, $1 \to -1$ and decides by the sign of the received value.
- BER counts wrong bits; BLER counts wrong blocks. Systems are specified in BLER.
- Buying reliability with power is expensive; coding is the efficient way.

Important formulas

AWGN	$y = s + w,\ w \sim \mathcal{N}(0, N_0/2)$
BPSK BER	$P_b = Q(\sqrt{2E_b/N_0})$
Decibels	$x_{\,\mathrm{dB}} = 10 \log_{10} x$
Independent errors	$\mathrm{BLER} = 1 - (1-p)^K \approx Kp$

Visual recap • Chapter 1



One wrong bit ⇒ wrong block: $\mathrm{BLER} = 1 - (1-p)^K$.
$K = 1000,\ p = 10^{-3} \Rightarrow \mathrm{BLER} = 63\% \Rightarrow$ **we need coding.**

Common student mistakes

! Common mistake

- Using $Q(\sqrt{E_b/N_0})$ instead of $Q(\sqrt{2E_b/N_0})$ for BPSK. The factor 2 comes from $\sigma^2 = N_0/2$.
- Forgetting to convert dB to a linear ratio before using a formula.
- Mixing E_s/N_0 (per transmitted symbol) and E_b/N_0 (per information bit). With coding they differ: $E_s/N_0 = R\,E_b/N_0$ (Chapter 2).
- Applying $1-(1-p)^K$ to decoded (coded) bits, whose errors are not independent.

Practice problems

Problem 1.1. Uncoded BPSK operates at $E_b/N_0 = 8\,\mathrm{dB}$. Find the BER.

Solution $10^{0.8} = 6.31$, $\sqrt{12.62} = 3.55$, $P_b = Q(3.55) \approx 1.9 \times 10^{-4}$.

Problem 1.2. A link needs $\mathrm{BLER} \le 10^{-2}$ for $K = 200$-bit blocks with independent errors. What raw BER is needed?

Solution $1-(1-p)^{200} \le 0.01 \Rightarrow p \le 1 - 0.99^{1/200} = 5.03 \times 10^{-5}$ (the approximation $p \approx 0.01/200 = 5 \times 10^{-5}$ is almost the same).

Problem 1.3. In BPSK with $\sigma = 0.5$, what is the probability that the received value has the wrong sign?

Solution $P_b = Q(1/\sigma) = Q(2) = 0.0228$.

Problem 1.4. A receiver measures $E_b/N_0 = 3\,\text{dB}$. Express it as a ratio, and give σ for uncoded BPSK with $E_s = 1$.

Solution $10^{0.3} = 1.995$. $N_0 = E_b/1.995 = 0.501$, $\sigma = \sqrt{N_0/2} = 0.5006$.

Self-assessment quiz

1. True or false: the AWGN noise sample at time t depends on the sample at time $t-1$.
2. Which block adds redundancy: the source encoder or the channel encoder?
3. By roughly how many dB must uncoded BPSK increase power to go from BER 10^{-3} to 10^{-5}?
4. Why does a system designer care more about BLER than BER?

Answers: 1) False (white = independent). 2) Channel encoder. 3) About 2.8 dB. 4) A block with one wrong bit is useless or must be re-sent.

CHAPTER

2



Fundamentals of Channel Coding

IN THIS CHAPTER YOU WILL LEARN

- what redundancy is and why it helps;
- codewords, block codes and the code rate $R = K/N$;
- Hamming distance and how many errors a code can fix;
- generator and parity-check matrices of linear codes;
- the three classic channels: BEC, BSC, AWGN;
- Shannon capacity and the channel coding theorem;
- why polar codes were a historic result.

2.1 What is channel coding?

Intuition

When you spell your name on a bad phone line you say "M as in Mary, A as in Alpha…". You add extra words that carry *no new information* but make the important letters easy to recover. This is redundancy. Channel coding is the mathematics of adding the *smallest* and *smartest* amount of redundancy that still lets the receiver fix the errors.

Definition 2.1 • Channel coding

Channel coding (also forward error correction, FEC) maps a block of K information bits $\mathbf{m} = (m_0, \dots, m_{K-1})$ to a longer block of N coded bits $\mathbf{c} = (c_0, \dots, c_{N-1})$, $N > K$. The $N - K$ extra bits are the **redundancy**. The receiver uses the known structure of the code to detect and correct transmission errors.

Fig. 2.1 compares the two systems. The only difference is one pair of blocks: the channel encoder, which turns K bits into $N > K$ bits, and the channel decoder, which uses the known structure of the code to remove the errors. Everything else (modulator, channel, demodulator) stays the same.

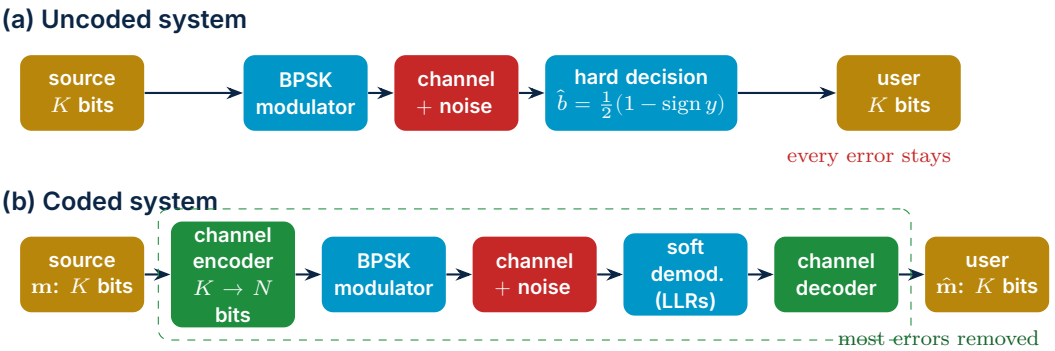


Figure 2.1. Block diagram. Uncoded and coded transmission. Channel coding adds the green blocks: $N - K$ redundant bits at the transmitter and a decoder that uses them at the receiver.

2.1.1 The simplest code: repetition

Send every bit three times: $0 \to 000$, $1 \to 111$. The decoder takes a *majority vote*.

Worked Example 2.1 • Repetition code over a binary symmetric channel

Each transmitted bit is flipped independently with probability $p = 0.1$. The majority vote fails only if two or three of the three copies are flipped:

$$P_e = \binom{3}{2} p^2(1-p) + p^3 = 3(0.01)(0.9) + 0.001 = 0.028.$$

The error probability dropped from 0.1 to 0.028 — but we now send three bits for every information bit. The *rate* is only 1/3. Good codes do much better than this: they reduce errors dramatically while keeping a high rate.

2.2 Codewords, block codes and code rate

Definition 2.2 • Block code

An (N, K) **block code** $\mathcal{C}$ is a set of 2^K binary vectors of length N, called **codewords**. The encoder is a one-to-one map from the 2^K possible messages to the codewords.

- K = number of **information bits** (message length);
- N = number of **coded bits** (block length, codeword length);
- $R = K/N$ = **code rate**, the fraction of useful information in each coded bit.

$$R = \frac{K}{N}, \qquad E_s = R\,E_b \implies \frac{E_s}{N_0} = R\,\frac{E_b}{N_0}.$$

The second relation is very important for fair comparisons. Each coded symbol has energy E_s; it carries only R information bits, so the energy spent per *information* bit is $E_b = E_s/R$. When we compare a coded system with an uncoded one at the same E_b/N_0, the coded system has *noisier* symbols but can correct errors. A code is good only if it wins this fair comparison.

Worked Example 2.2 • Converting E_b/N_0 to noise variance

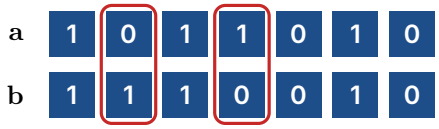
A rate-1/2 code with BPSK ($E_s = 1$) is simulated at $E_b/N_0 = 2\,\text{dB}$. Then $E_s/N_0 = 0.5 \times 10^{0.2} = 0.792$, so $N_0 = 1/0.792 = 1.262$ and $\sigma^2 = N_0/2 = 0.631$, $\sigma = 0.794$. Every simulation in this book uses exactly this conversion (function `awgn()` in Chapter 11).

2.3 Hamming distance

Definition 2.3 • Hamming weight and distance

The **Hamming weight** $w(\mathbf{a})$ is the number of ones in $\mathbf{a}$. The **Hamming distance** $d(\mathbf{a}, \mathbf{b})$ is the number of positions in which $\mathbf{a}$ and $\mathbf{b}$ differ, $d(\mathbf{a}, \mathbf{b}) = w(\mathbf{a} \oplus \mathbf{b})$. The **minimum distance** of a code is

$$d_{\min} = \min_{\mathbf{a} \neq \mathbf{b} \in \mathcal{C}} d(\mathbf{a}, \mathbf{b}).$$



$d(\mathbf{a}, \mathbf{b}) = 2$ (two positions differ)

Intuition

Think of codewords as cities on a map, and errors as steps away from a city. If any two cities are at least $d_{\min}$ steps apart, then after t steps you are still closer to your own city as long as $2t < d_{\min}$. Larger distance between codewords means more errors can be absorbed.

$$\text{detect up to } d_{\min} - 1 \text{ errors}, \qquad \text{correct up to } t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \text{ errors}.$$

For the repetition code $\{000, 111\}$, $d_{\min} = 3$, so it corrects $t = 1$ error — exactly what the majority vote does.

2.4 Linear block codes and generator matrices

Storing 2^K codewords in a table is impossible for $K = 100$ ($2^{100} \approx 10^{30}$). Linear codes need only a $K \times N$ matrix.

Definition 2.4 • Linear block code and generator matrix

A code is **linear** if the XOR (modulo-2 sum) of any two codewords is again a codeword. Every linear (N, K) code has a $K \times N$ **generator matrix** $\mathbf{G}$ whose rows are a basis of the code, and encoding is a vector–matrix product over GF(2) (arithmetic modulo 2: $1 \oplus 1 = 0$):

$$\mathbf{c} = \mathbf{m}\,\mathbf{G}.$$

A **parity-check matrix** $\mathbf{H}$, of size $(N - K) \times N$, satisfies $\mathbf{H}\mathbf{c}^T = \mathbf{0}$ for every codeword.

For a linear code, $d(\mathbf{a}, \mathbf{b}) = w(\mathbf{a} \oplus \mathbf{b})$ and $\mathbf{a} \oplus \mathbf{b}$ is itself a codeword, so $d_{\min}$ equals the smallest weight of a non-zero codeword. This makes analysis much easier.

Worked Example 2.3 • The (7,4) Hamming code

Take the systematic generator matrix $\mathbf{G} = [\mathbf{I}_4 \,|\, \mathbf{P}]$:

$$\mathbf{G} = \left[\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{array}\right], \qquad \mathbf{H} = [\mathbf{P}^T \,|\, \mathbf{I}_3] = \left[\begin{array}{cccc|ccc} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{array}\right].$$

Encoding $\mathbf{m} = (1, 0, 1, 1)$: add rows 1, 3 and 4 of $\mathbf{G}$ (the positions of the ones in $\mathbf{m}$):

$$\begin{array}{r} 1\,0\,0\,0\,1\,1\,0 \\ \oplus\, 0\,0\,1\,0\,1\,1\,1 \\ \oplus\, 0\,0\,0\,1\,1\,0\,1 \\ \hline \mathbf{c} = 1\,0\,1\,1\,1\,0\,0 \end{array}$$

Checking: $\mathbf{H}\mathbf{c}^T = (0, 0, 0)^T$. **An error** in the third bit gives $\mathbf{r} = 1001100$ and the *syndrome* $\mathbf{H}\mathbf{r}^T = (1, 1, 1)^T$, which is exactly the third column of $\mathbf{H}$: the decoder knows where the error is and flips it back. The weights of the 15 non-zero codewords are 3 (seven times), 4 (seven times) and 7 (once), so $d_{\min} = 3$ and one error is corrected.

Important note

Polar codes are linear block codes. Their generator matrix is built in a very special way (Chapter 5) that makes both encoding and decoding fast. Everything in this section — rate, distance, generator matrix — applies to them directly.

2.5 Three channel models

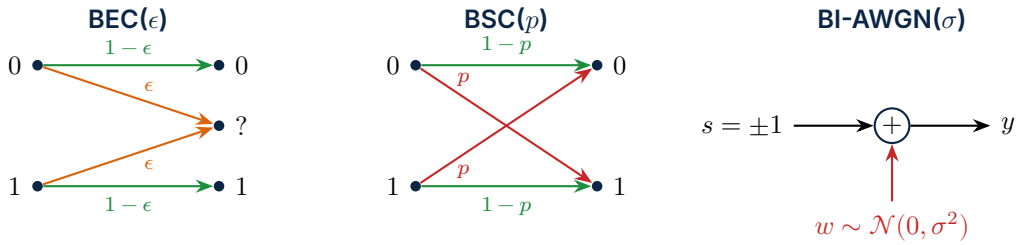


Figure 2.2. The binary erasure channel, the binary symmetric channel and the binary-input AWGN channel.

- **Binary erasure channel BEC(ϵ).** Each bit is either received correctly or *erased* (the receiver knows it is lost, output "?") with probability ϵ. Unrealistic but perfect for teaching: polar codes are easiest to analyse on the BEC (Chapter 4).
- **Binary symmetric channel BSC(p).** Each bit is flipped with probability p and the receiver does not know which bits were flipped. This is a hard-decision channel.
- **Binary-input AWGN channel.** BPSK plus Gaussian noise, the model of Chapter 1. The receiver has soft information. This is the main channel for practical design.

2.6 Shannon channel capacity

Intuition

Every channel has a "speed limit". Below that limit you can make errors as rare as you wish with long enough codes; above it errors are unavoidable whatever you do. Shannon proved this in 1948 and called the limit the *capacity* C, measured in bits per channel use.

Definition 2.5 • Capacity

For a channel with input X and output Y, the **capacity** is the largest mutual information over all input distributions, $C = \max_{p(x)} I(X;Y)$. For symmetric binary channels the maximum is reached with equally likely inputs, and the **symmetric capacity** is denoted $I(W)$.

$$C_{\mathrm{BEC}} = 1-\epsilon, \qquad C_{\mathrm{BSC}} = 1 - H_2(p), \qquad C_{\mathrm{AWGN}} = \tfrac{1}{2}\log_2\!\left(1+\tfrac{2E_s}{N_0}\right),$$

where $H_2(p) = -p\log_2 p - (1-p)\log_2(1-p)$ is the binary entropy function.

The AWGN formula is for a real channel with Gaussian inputs; with BPSK inputs the capacity is lower and must be computed numerically (Fig. 2.3).

Derivation of C_{BEC}. With equally likely inputs, the output is "?" with probability ϵ and then tells us nothing; otherwise it tells us the bit exactly (1 bit). So on average $I(X;Y) = (1-\epsilon)\cdot 1 + \epsilon\cdot 0 = 1-\epsilon$. Intuitively, a fraction ϵ of the bits is lost, so at most a fraction $1-\epsilon$ can carry information. Remarkably, this is achievable even though the transmitter does not know *which* bits will be erased.

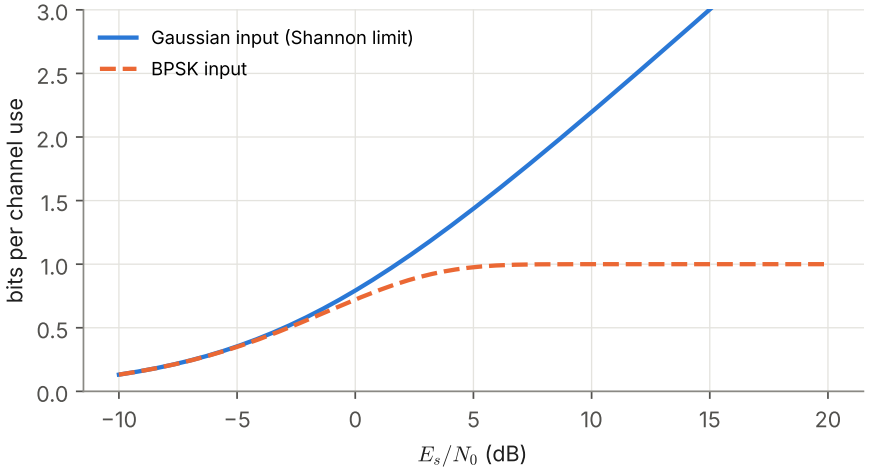


Figure 2.3. Capacity of the real AWGN channel: Gaussian inputs versus BPSK inputs.

Definition 2.6 • Channel coding theorem (Shannon, 1948)

For every rate $R < C$ there exist codes of rate R whose block error probability tends to zero as $N \to \infty$. For every rate $R > C$ the error probability is bounded away from zero for all codes.

2.6.1 The Shannon limit in E_b/N_0

Setting $R = C$ with $E_s = RE_b$ in the AWGN formula gives the smallest usable E_b/N_0:

$$R = \tfrac{1}{2}\log_2\!\left(1 + 2R\frac{E_b}{N_0}\right) \implies \left.\frac{E_b}{N_0}\right|_{\min} = \frac{2^{2R}-1}{2R}.$$

Worked Example 2.4 • Shannon limits

Rate R	1/4	1/2	3/4
Gaussian input: $(2^{2R}-1)/(2R)$	−0.82 dB	0.00 dB	0.86 dB
BPSK input (numerical)	−0.79 dB	0.19 dB	1.63 dB

For $R \to 0$ the Gaussian limit becomes $\ln 2 = -1.59$ dB, the famous *ultimate Shannon limit*. A practical rate-1/2 code that reaches BLER 10^{-3} at 2 dB is therefore about 1.8 dB from the BPSK limit. The size of this gap is the main figure of merit of a code family.

2.7 Why polar codes are important

Shannon's proof was not constructive: it used *random* codes, which nobody can decode. For 60 years researchers built codes that came close to capacity in practice (turbo codes in 1993, LDPC codes rediscovered in the 1990s), but none was *proved* to reach capacity with a practical decoder on a general channel.

Why the 2009 result was historic

Polar codes, introduced by Erdal Arıkan in 2009, were the first family of codes with *all* of these properties together:

1. an explicit, deterministic construction (no randomness);
2. proved to achieve the symmetric capacity of any binary-input memoryless symmetric channel;
3. encoding *and* decoding with complexity $O(N \log N)$.

In 2016 3GPP selected polar codes for the control channels of 5G New Radio. They are now in billions of devices.

2.8 Chapter summary

Key concepts

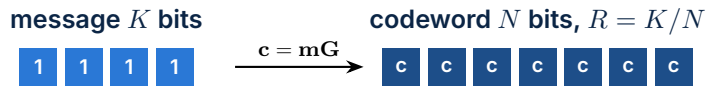
- An (N, K) code maps K information bits to N coded bits; the rate is $R = K/N$.
- Fair comparisons use E_b/N_0; the coded symbol SNR is $E_s/N_0 = R\,E_b/N_0$.
- Minimum distance $d_{\min}$ decides how many errors can be detected $(d_{\min} - 1)$ and corrected $(\lfloor (d_{\min}-1)/2 \rfloor)$.
- Linear codes are described by a generator matrix $\mathbf{G}$ ($\mathbf{c} = \mathbf{mG}$) and a parity-check matrix $\mathbf{H}$.

- Capacity is the maximum reliable rate; polar codes provably achieve it with $O(N \log N)$ complexity.

Important formulas

Rate and energy	$R = K/N, \quad E_s/N_0 = R\,E_b/N_0$
Error correction	$t = \lfloor (d_{\min} - 1)/2 \rfloor$
Capacities	$1 - \epsilon$ (BEC), $1 - H_2(p)$ (BSC), $\frac{1}{2}\log_2(1 + 2E_s/N_0)$ (AWGN)
Shannon limit	$E_b/N_0 \geq (2^{2R} - 1)/(2R)$

Visual recap • Chapter 2



distance between codewords ⇒ error correction • capacity C = speed limit • polar codes: provably reach C with $O(N \log N)$

Common student mistakes

! Common mistake

- Comparing a coded and an uncoded system at the same E_s/N_0 — this hides the energy cost of redundancy.
- Thinking capacity is a property of the code. It is a property of the *channel*; codes try to reach it.
- Using ordinary addition instead of XOR in $\mathbf{mG}$.
- Believing a code can correct $d_{\min}$ errors. It corrects $\lfloor (d_{\min} - 1)/2 \rfloor$ with a bounded-distance decoder.

Practice problems

Problem 2.1. A code has $N = 256$ and $K = 64$. What is its rate? At $E_b/N_0 = 1$ dB, what is σ for BPSK with $E_s = 1$?

Solution $R = 0.25$. $E_s/N_0 = 0.25 \times 10^{0.1} = 0.3147$, $N_0 = 3.177$, $\sigma = \sqrt{N_0/2} = 1.260$.

Problem 2.2. Find the capacity of a BSC with $p = 0.11$ and of a BEC with $\epsilon = 0.5$.

Solution $H_2(0.11) = 0.4999$, so $C_{\text{BSC}} \approx 0.500$; $C_{\text{BEC}} = 0.5$. Both channels can carry half a bit per use.

Problem 2.3. A linear code has $d_{\min} = 6$. How many errors can it detect and correct?

Solution Detect 5; correct $\lfloor 5/2 \rfloor = 2$.

Problem 2.4. Encode $\mathbf{m} = (0, 1, 1, 0)$ with the (7,4) Hamming code of this chapter.

Solution Rows 2 and 3: $0100011 \oplus 0010111 = 0110100$. Check: $\mathbf{H}\mathbf{c}^T = \mathbf{0}$.

Problem 2.5. What is the minimum E_b/N_0 for reliable communication at $R = 1/3$ with Gaussian inputs?

Solution $(2^{2/3} - 1)/(2/3) = 0.881 \Rightarrow -0.55\,\text{dB}$.

Self-assessment quiz

1. Is the set $\{000, 011, 101, 110\}$ a linear code? What is $d_{\min}$?
2. What happens to E_s/N_0 if we halve the code rate at fixed E_b/N_0?
3. What is the capacity of a BEC with $\epsilon = 0.2$?
4. Name the three properties that made polar codes historic.

Answers: 1) Yes (closed under XOR), $d_{\min} = 2$. 2) It drops by 3 dB. 3) 0.8. 4) Explicit construction, provably capacity-achieving, $O(N \log N)$ encoding and decoding.

PART II

Intermediate Level: Understanding Polar Codes

Channel polarization, the Kronecker generator matrix and the butterfly encoder, step by step with real numbers.

CHAPTER

3



Introduction to Polar Codes

IN THIS CHAPTER YOU WILL LEARN

- what a polar code is, in one picture;
- the history of Arıkan's 2009 discovery;
- channel polarization explained without equations;
- reliable and unreliable bit-channels;
- frozen bits and information bits;
- the parameters N, K, R and the sets $\mathcal{A}$, $\mathcal{F}$;
- how polar codes compare with LDPC and turbo codes.

3.1 What are polar codes?

The classroom analogy

A teacher has N seats in a classroom. Some seats are near the board and have a perfect view; others are behind a pillar and see nothing. If the teacher must place K important students, she puts them in the K best seats and leaves the pillar seats with dummy "students" whose answers she already knows. A polar code does exactly this with bits: it creates N "seats" (bit-channels) of very different quality, puts the K information bits in the best ones, and fills the bad ones with known values (zeros) called *frozen bits*.

Definition 3.1 • Polar code

A **polar code** of length $N = 2^n$ and dimension K is a linear block code defined by

1. the generator matrix $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ with kernel $\mathbf{F} = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$;
2. an **information set** $\mathcal{A} \subset \{0, \dots, N-1\}$ with $|\mathcal{A}| = K$; the complement $\mathcal{F} = \{0, \dots, N-1\} \setminus \mathcal{A}$ is the **frozen set**.

The encoder builds a vector $\mathbf{u}$ of length N with $u_i =$ message bits for $i \in \mathcal{A}$ and $u_i = 0$ for $i \in \mathcal{F}$, and sends $\mathbf{x} = \mathbf{u}\,\mathbf{G}_N$.

That is all! The magic is entirely in *which* positions are frozen and in the structure of $\mathbf{G}_N$, which makes the bit positions behave like channels of very different quality.

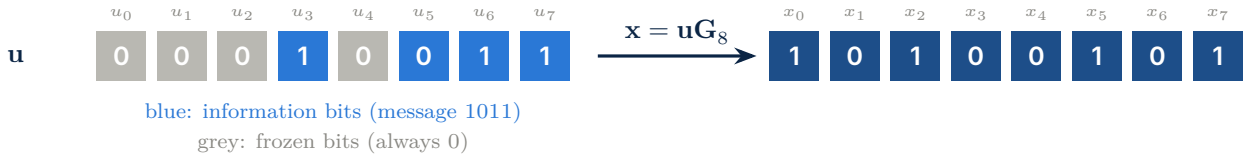
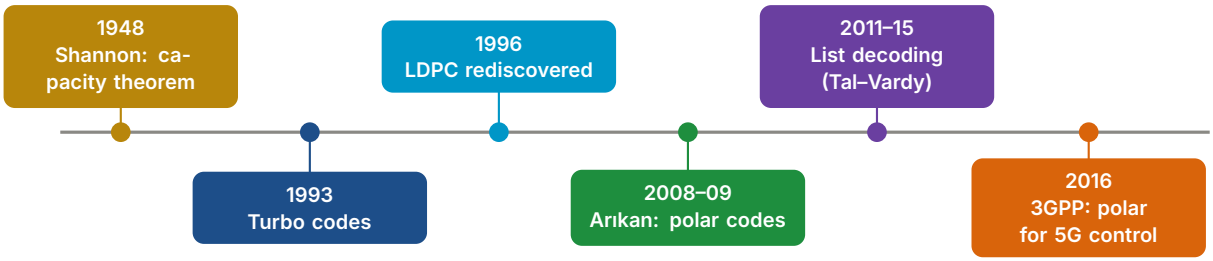


Figure 3.1. A complete $(8, 4)$ polar code: frozen set $\mathcal{F} = \{0, 1, 2, 4\}$, information set $\mathcal{A} = \{3, 5, 6, 7\}$. This code is used in many examples of this book.

3.2 History: Arıkan's 2009 breakthrough



- Erdal Arıkan (Bilkent University, Ankara) had worked for many years on sequential decoding and the "cutoff rate" of channels. While trying to boost the cutoff rate he noticed that combining two copies of a channel and splitting them again creates one better and one worse channel. Repeating the operation recursively, the channels *polarize*.
- His paper "Channel polarization: a method for constructing capacity-achieving codes for symmetric binary-input memoryless channels" appeared in the *IEEE Transactions on Information Theory* in July 2009 and received the IEEE Information Theory Society Paper Award.
- Early polar codes with the plain SC decoder were weaker than LDPC codes at practical lengths. The list decoder of Tal and Vardy (conference version 2011, journal 2015) combined with a CRC changed this completely: at short and medium lengths CRC-aided list decoding became competitive with or better than the best known codes.
- In November 2016 (3GPP RAN1 meeting #87) polar codes were agreed for the control channels of the 5G enhanced mobile broadband scenario. They are specified in 3GPP TS 38.212.

3.3 Why were polar codes invented?

Before 2009 the situation was strange: Shannon had proved that capacity can be reached, practical codes came close in simulations, but *no* code was proved to reach capacity with a practical decoder on general binary channels. Polar codes closed this gap with a beautifully simple idea: do not try to correct errors on a uniform channel; instead *transform* N identical channels into channels that are either almost perfect or almost useless, and then the coding problem becomes trivial — send data on the perfect ones.

3.4 Channel polarization explained intuitively

Two weak friends

Two friends each pass you a coin through a foggy window (each sees the coin correctly with probability 0.9). You ask the first friend to tell you $u_0 \oplus u_1$ and the second to tell you u_1. To learn u_0 you need *both* messages to be right — harder than before. But once you know u_0, you get *two* independent opinions about u_1 — easier than before. From two equal channels you have made one *worse* and one *better* channel. The total information is unchanged; it has only been re-distributed.

Repeat the trick: combine two "better" channels to make an even better one and a slightly worse one; combine two "worse" ones to make an even worse one and a slightly better one. After n levels there are $N = 2^n$ synthetic channels, called **bit-channels**, and almost all of them are extreme.

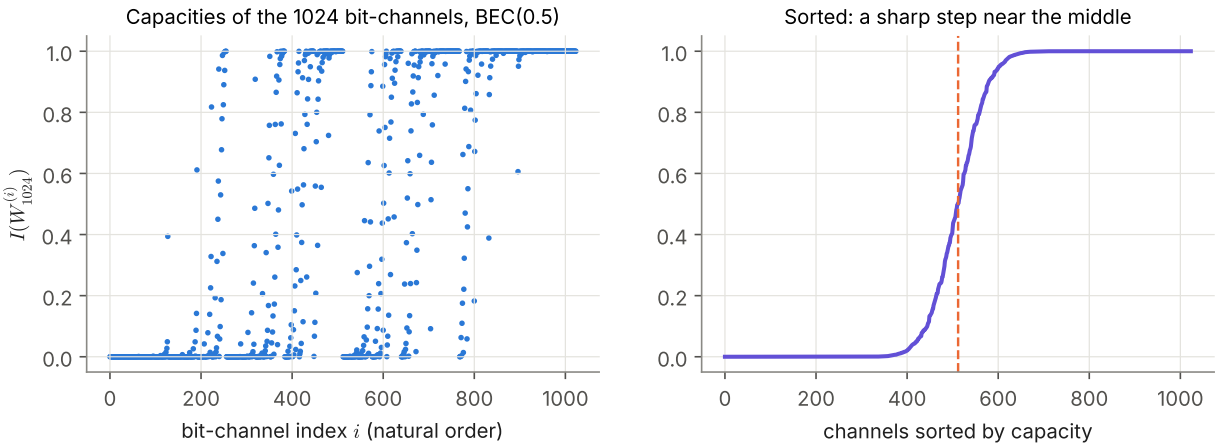


Figure 3.2. Polarization over BEC(0.5) with $N = 1024$. Left: capacity of each bit-channel in natural order. Right: sorted — about half the channels are almost perfect and half almost useless.

3.5 Reliable and unreliable bit-channels

Definition 3.2 • Bit-channel

The i-th **bit-channel** $W_N^{(i)}$ is the channel that bit u_i "sees" when the receiver knows the channel output $\mathbf{y}$ *and* the correct values of all earlier bits $u_0, \dots, u_{i-1}$. Its quality is measured by its capacity $I(W_N^{(i)}) \in [0,1]$ or by its Bhattacharyya parameter $Z(W_N^{(i)}) \in [0,1]$ (Chapter 4): small Z means reliable.

The "knows the earlier bits" part is essential. It is exactly what a *successive* decoder can offer: it decodes u_0 first, then u_1 using $\hat{u}_0$, then u_2 using $\hat{u}_0, \hat{u}_1$, and so on (Chapter 6).

Definition 3.3 • Polarization theorem (Arıkan 2009)

For any binary-input memoryless symmetric channel W and any $\delta \in (0,1)$, as $N \to \infty$,

$$\frac{\left|\{i : I(W_N^{(i)}) > 1-\delta\}\right|}{N} \to I(W), \qquad \frac{\left|\{i : I(W_N^{(i)}) < \delta\}\right|}{N} \to 1 - I(W).$$

In words: the fraction of almost-perfect channels tends to the capacity of the original channel, and all the other channels become almost useless.

3.6 Frozen bits and information bits

Definition 3.4 • Frozen and information bits

Information bits are placed on the K most reliable bit-channels (the set $\mathcal{A}$). **Frozen bits** are placed on the remaining $N-K$ least reliable channels (the set $\mathcal{F}$); their values are fixed in advance and known to the decoder. With symmetric channels we can take them all equal to 0.

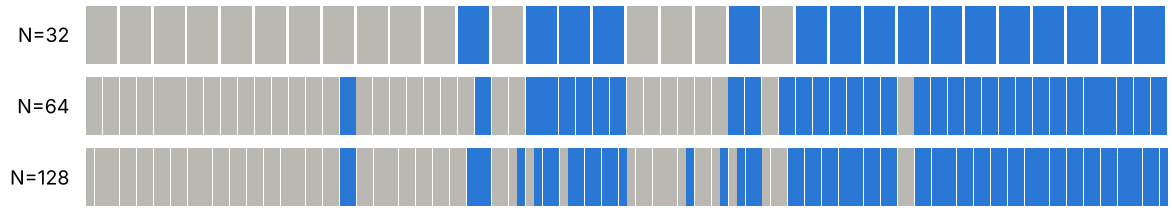


Figure 3.3. Information (blue) and frozen (grey) positions of rate-1/2 polar codes designed by Gaussian approximation. Frozen positions cluster at small indices, information at large indices, with a mixed zone in the middle.

Frozen bits are not wasted

The frozen positions do not carry data, but they are not useless: knowing them for certain is what allows the decoder to use them as "anchors". A bit that would arrive almost random is replaced by a bit the decoder knows perfectly. Choosing which positions to freeze is called **code construction** and is the heart of polar code design (Chapters 4 and 9).

Fig. 3.4 shows how the two sets are used in a polar transmitter. The construction block runs *once*, offline, for a given channel quality; its output (the sets $\mathcal{A}$ and $\mathcal{F}$) is stored at both the transmitter and the receiver. The encoder itself is the same for every code of length N.

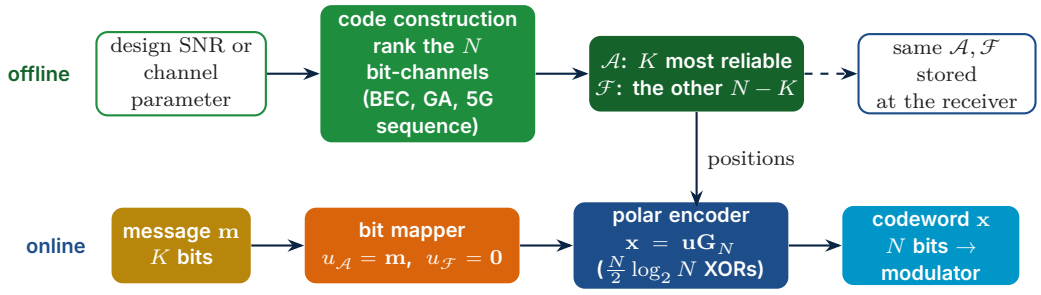


Figure 3.4. **Block diagram.** The polar-code transmitter. Construction (green) chooses the sets once; for every block the message is mapped onto $\mathcal{A}$ and encoded with the fixed matrix $\mathbf{G}_N$.

3.7 Block length, information length and code rate

$N = 2^n$	block length; always a power of two for the basic construction ($n = \log_2 N$ levels).
K	number of information bits (including CRC bits when a CRC is used).
$R = K/N$	code rate.
$\mathcal{A}$, $\mathcal{F}$	information set (K indices), frozen set ($N - K$ indices).

Lengths that are not powers of two are obtained by puncturing, shortening or repetition (Chapter 9); 5G uses this to reach any number of transmitted bits.

3.8 Capacity-achieving behaviour

Intuition

If we choose $K = \lfloor NR \rfloor$ with $R < I(W)$, the polarization theorem says that for large N there are more than K almost-perfect channels. All information bits sit on almost-perfect channels, so all of them are decoded correctly with high probability. The rate R can be as close to $I(W)$ as we like.

Arıkan proved the bound

$$P_e^{\mathrm{SC}}(N, K, \mathcal{A}) \;\le\; \sum_{i \in \mathcal{A}} Z(W_N^{(i)}),$$

and Arıkan and Telatar showed that, for any $\beta < 1/2$, the sum can be made smaller than 2^{-N^β} at any fixed rate below capacity. So the block error probability falls roughly like $2^{-\sqrt{N}}$ — fast, but slower than the exponential 2^{-cN} of the best (non-practical) random codes. This is one reason why plain polar codes need help (list decoding and CRC) at short lengths.

3.9 Polar codes versus LDPC and turbo codes

	Turbo codes	LDPC codes	Polar codes
Invented	Berrou et al., 1993	Gallager 1962; rediscovered 1990s	Arıkan 2009
Structure	two convolutional codes + interleaver	sparse parity-check matrix	Kronecker power $\mathbf{F}^{\otimes n}$ + frozen set
Typical decoder	iterative BCJR	belief propagation (iterative)	SC, SCL, CA-SCL (sequential)
Capacity proof	no	for special ensembles/channels (e.g. spatially coupled)	yes, for all BMS channels with SC
Strength	mature, flexible	very high throughput, long blocks	excellent at short lengths with CA-SCL; simple rate adaption
Weakness	error floors, latency	short-block performance, design of irregular graphs	sequential decoding limits throughput
Used in	3G, 4G LTE data	5G NR data, Wi-Fi, DVB-S2	5G NR control and broadcast channels

Professional insight

The 5G decision was not "polar is better than LDPC". It was a split by job: LDPC for large, high-speed data blocks, polar for small, critical control messages (typically tens to a few hundred bits) where very low BLER and good short-length performance matter most.

3.10 Chapter summary

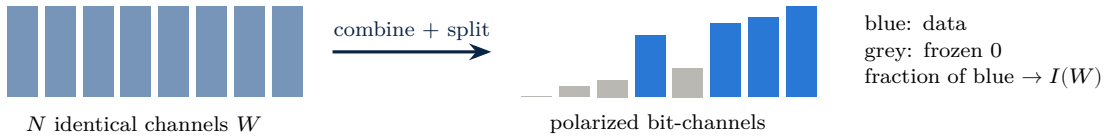
Key concepts

- A polar code is $\mathbf{x} = \mathbf{u}\mathbf{G}_N$ with $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ and a chosen frozen set.
- Recursive channel combining and splitting make bit-channels polarize into almost-perfect and almost-useless.
- Information bits go on the K most reliable bit-channels; the others are frozen to 0.
- The fraction of good channels tends to $I(W)$: polar codes achieve capacity.
- 5G NR uses polar codes for control information and LDPC codes for data.

Important formulas

Encoding	$\mathbf{x} = \mathbf{u}\,\mathbf{F}^{\otimes n}$, $u_i = 0$ for $i \in \mathcal{F}$
Rate	$R = K/N = \lvert\mathcal{A}\rvert/N$
SC error bound	$P_e \leq \sum_{i\in\mathcal{A}} Z(W_N^{(i)})$

Visual recap • Chapter 3



Common student mistakes

! Common mistake

- Thinking that frozen positions are the *first* $N-K$ indices. They are the *least reliable* indices, which are mostly but not only small (see Fig. 3.3).
- Forgetting that the bit-channel definition assumes the earlier bits are known. The decoder can only approximate this by using its own earlier decisions.
- Believing polar codes are good at every length with the SC decoder. At short lengths SC is weak; list decoding with CRC is needed.

Practice problems

Problem 3.1. A polar code has $N = 512$ and rate $R = 3/8$. How many frozen bits does it have?

Solution $K = 192$, so $N - K = 320$ frozen bits.

Problem 3.2. A BEC has $\epsilon = 0.3$. For very large N, roughly what fraction of bit-channels is almost perfect?

Solution $I(W) = 1 - \epsilon = 0.7$: about 70% of the bit-channels.

Problem 3.3. The Bhattacharyya parameters of the information bit-channels of a code are 0.004, 0.12, 0.19 and 0.32. Give an upper bound on the SC block error probability.

Solution $P_e \le 0.004 + 0.12 + 0.19 + 0.32 = 0.634$. (This is the $(8, 4)$ code over BEC(0.5); the bound is loose but illustrates the method.)

Self-assessment quiz

1. What is the kernel matrix of Arıkan's polar codes?
2. Which bits does the decoder already know before decoding?
3. In which 5G channels are polar codes used?
4. What happens to the fraction of "middle" (not polarized) channels as N grows?

Answers: 1) $\mathbf{F} = [1\,0; 1\,1]$. 2) The frozen bits. 3) Control and broadcast channels (DCI, UCI, PBCH). 4) It tends to zero.

CHAPTER

4



Channel Polarization — The Core Idea

IN THIS CHAPTER YOU WILL LEARN

- channel combining: building W_2, W_4, …, W_N;
- channel splitting: the synthetic bit-channels $W_N^{(i)}$;
- the exact BEC recursion $Z \mapsto 2Z - Z^2$, $Z \mapsto Z^2$;
- the Bhattacharyya parameter and its meaning;
- how to compute the reliability of every bit-channel;
- why channels polarize when N grows.

4.1 Channel combining

Intuition

We start with two independent uses of the same channel W. Instead of sending two independent bits, we *mix* them: the first channel carries $u_0 \oplus u_1$, the second carries u_1. Nothing is lost — from (x_0, x_1) we can always recover (u_0, u_1) — but the two bits are now protected in different ways.

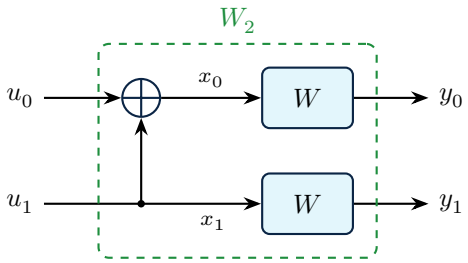


Figure 4.1. The basic combining step: two copies of W become the vector channel $W_2 : (u_0, u_1) \mapsto (y_0, y_1)$.

Definition 4.1 • Combined channel W_2

With $x_0 = u_0 \oplus u_1$ and $x_1 = u_1$, i.e. $\mathbf{x} = \mathbf{uF}$, the combined channel has transition probability

$$W_2(y_0, y_1 \mid u_0, u_1) = W(y_0 \mid u_0 \oplus u_1)\, W(y_1 \mid u_1).$$

4.1.1 Recursive combining

The construction is applied again to two copies of W_2. In general two independent copies of $W_{N/2}$ are combined into W_N:

$$W_N(\mathbf{y} \mid \mathbf{u}) = W^N(\mathbf{y} \mid \mathbf{uG}_N) = \prod_{j=0}^{N-1} W(y_j \mid x_j), \qquad \mathbf{x} = \mathbf{uG}_N.$$

Fig. 4.2 shows W_4. With our convention $\mathbf{G}_N = \mathbf{F} \otimes \mathbf{G}_{N/2}$, the *second* half of $\mathbf{u}$ is XORed into the first half, and each half is then encoded by $\mathbf{G}_{N/2}$:

$$\mathbf{x} = \big[(\mathbf{u}_{\text{first}} \oplus \mathbf{u}_{\text{second}})\mathbf{G}_{N/2} \;\big|\; \mathbf{u}_{\text{second}}\mathbf{G}_{N/2}\big].$$

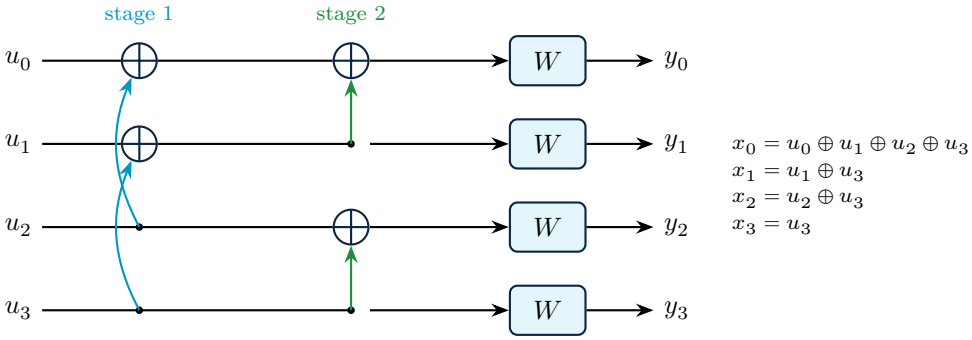


Figure 4.2. W_4 built from four copies of W. Each XOR node combines two signals; dots are copy points.

4.2 Channel splitting

Combining makes one big channel. *Splitting* cuts it again into N binary-input channels, one per bit u_i, in the order in which a successive decoder will meet them.

Definition 4.2 • Synthetic bit-channels

The i-th bit-channel has input u_i and output $(\mathbf{y}, u_0^{i-1})$ — the channel output plus all earlier bits:

$$W_N^{(i)}(\mathbf{y}, u_0^{i-1} \mid u_i) = \sum_{u_{i+1}^{N-1}} \frac{1}{2^{N-1}}\, W_N(\mathbf{y} \mid \mathbf{u}).$$

Later bits $u_{i+1}, \dots, u_{N-1}$ are unknown, so they are averaged out (treated as random).

For $N = 2$ we obtain the two most important channels in this book:

$$W^-(y_0, y_1 \mid u_0) = W_2^{(0)} = \tfrac{1}{2} \sum_{u_1 \in \{0,1\}} W(y_0 \mid u_0 \oplus u_1)\, W(y_1 \mid u_1) \qquad \text{(“minus”, worse)}$$

$$W^+(y_0, y_1, u_0 \mid u_1) = W_2^{(1)} = \tfrac{1}{2}\, W(y_0 \mid u_0 \oplus u_1)\, W(y_1 \mid u_1) \qquad \text{(“plus”, better)}$$

Intuition

To decide u_0 you must look through *both* channels at a mixture that also contains the unknown u_1: two noisy views multiplied together — worse. To decide u_1 you already know u_0, so y_0 tells you u_1 (after removing u_0) and y_1 tells you u_1 directly: two independent views — better. This is exactly the repetition code effect from Chapter 2.

Fig. 4.3 puts combining and splitting side by side. On the left is the physical system: N uses of W behind the encoder. On the right is the *same* system seen by a successive decoder: N separate binary channels, where channel i also "receives" the earlier bits.

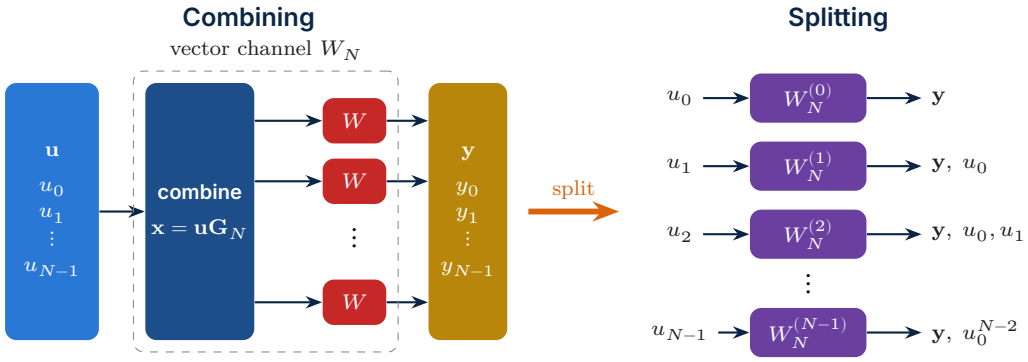


Figure 4.3. Block diagram. Channel combining (left) and splitting (right). Information is conserved: the capacities of the N bit-channels add up to $N\,I(W)$, but they become very unequal.

4.2.1 Conservation and polarization of information

By the chain rule of mutual information, nothing is created or destroyed:

$$I(W^-) + I(W^+) = 2\,I(W), \qquad I(W^-) \le I(W) \le I(W^+),$$

with equality only when $I(W) \in \{0, 1\}$. *Derivation sketch.* $(u_0, u_1) \leftrightarrow (x_0, x_1)$ is one-to-one, so $I(U_0U_1; Y_0Y_1) = I(X_0X_1; Y_0Y_1) = 2I(W)$. The chain rule gives $I(U_0U_1; Y_0Y_1) = I(U_0; Y_0Y_1) + I(U_1; Y_0Y_1 \mid U_0)$, and since U_0, U_1 are independent, $I(U_1; Y_0Y_1 \mid U_0) = I(U_1; Y_0Y_1U_0) = I(W^+)$. The first term is $I(W^-)$.

4.3 The binary erasure channel: an exact example

On the BEC every synthetic channel is again a BEC, and the recursion is exact and simple.

Worked Example 4.1 • Minus and plus channels of a BEC(ϵ)

Minus channel (bit u_0). We need $x_0 = u_0 \oplus u_1$ and $x_1 = u_1$; then $u_0 = x_0 \oplus x_1$. We learn u_0 only if *both* y_0 and y_1 are not erased:

$$P(\text{not erased}) = (1-\epsilon)^2 \;\Rightarrow\; \epsilon^- = 1 - (1-\epsilon)^2 = 2\epsilon - \epsilon^2.$$

Plus channel (bit u_1, with u_0 known). $u_1 = x_1$ from y_1, or $u_1 = x_0 \oplus u_0$ from y_0. It is lost only if *both* are erased:

$$\epsilon^+ = \epsilon^2.$$

Numbers for $\epsilon = 0.5$. $\epsilon^- = 0.75$, $\epsilon^+ = 0.25$. Capacities: $I(W^-) = 0.25$, $I(W^+) = 0.75$, and indeed $0.25 + 0.75 = 2 \times 0.5$.

BEC: $Z(W^-) = 2Z(W) - Z(W)^2, \qquad Z(W^+) = Z(W)^2, \qquad Z = \epsilon.$

4.3.1 Going to $N = 4$ and $N = 8$

Apply the same rules to W^- and W^+ (each with $\epsilon = 0.5$ start):

i	binary	path	erasure prob. $Z(W_4^{(i)})$	capacity	rank
0	00	$-,-$	$2(0.75) - 0.75^2 = 0.9375$	0.0625	4 (worst)
1	01	$-,+$	$0.75^2 = 0.5625$	0.4375	3
2	10	$+,-$	$2(0.25) - 0.25^2 = 0.4375$	0.5625	2
3	11	$+,+$	$0.25^2 = 0.0625$	0.9375	1 (best)

Reading the index in binary

With our natural-order convention, write i in binary with n bits, *most significant bit first*. Each bit tells which transform to apply, starting from the physical channel: $0 \Rightarrow$ minus ($Z \to 2Z - Z^2$), $1 \Rightarrow$ plus ($Z \to Z^2$). Example, $N = 8$, $i = 6 = 110_2$: $0.5 \xrightarrow{+} 0.25 \xrightarrow{+} 0.0625 \xrightarrow{-} 2(0.0625) - 0.0625^2 = 0.1211$.

i	0	1	2	3	4	5	6	7
binary	000	001	010	011	100	101	110	111
$Z(W_8^{(i)})$	0.9961	0.8789	0.8086	0.3164	0.6836	0.1914	0.1211	0.0039
$I(W_8^{(i)})$	0.0039	0.1211	0.1914	0.6836	0.3164	0.8086	0.8789	0.9961

The four best channels (shaded) are $\{3, 5, 6, 7\}$ — the information set of the $(8, 4)$ code of Fig. 3.1. Notice that channel 4 is *worse* than channel 3 although its index is larger.

4.4 Visualizing polarization for N = 2, 4, 8, 16

i	binary	Z	I	rank	i	binary	Z	I	rank
0	0000	0.99998	0.00002	16	8	1000	0.89989	0.10011	12
1	0001	0.99220	0.00780	15	9	1001	0.46730	0.53270	8
2	0010	0.98534	0.01466	14	10	1010	0.34618	0.65382	7
3	0011	0.77248	0.22752	11	11	1011	0.03664	0.96336	4
4	0100	0.96336	0.03664	13	12	1100	0.22752	0.77248	6
5	0101	0.65382	0.34618	10	13	1101	0.01466	0.98534	3
6	0110	0.53270	0.46730	9	14	1110	0.00780	0.99220	2
7	0111	0.10011	0.89989	5	15	1111	0.00002	0.99998	1

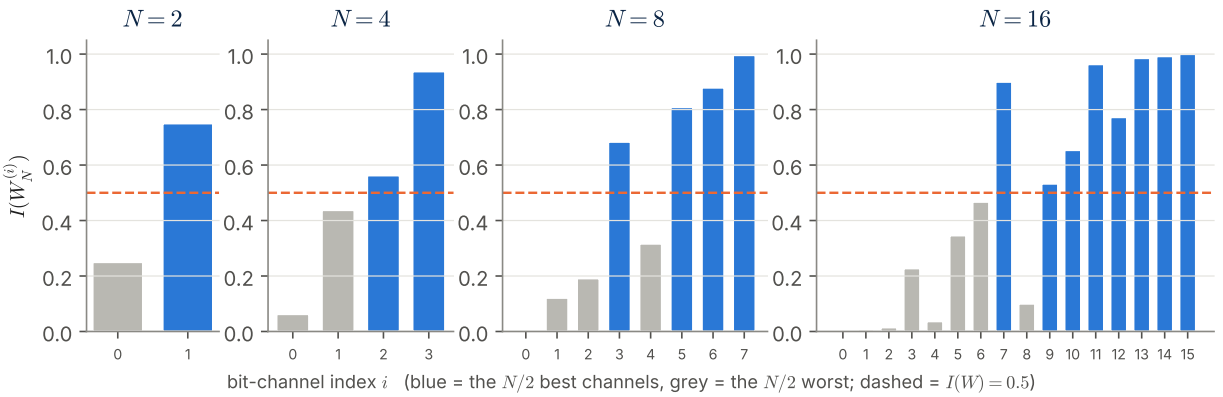


Figure 4.4. Capacities of the bit-channels for BEC(0.5). Already at $N = 16$ most bars are close to 0 or 1.

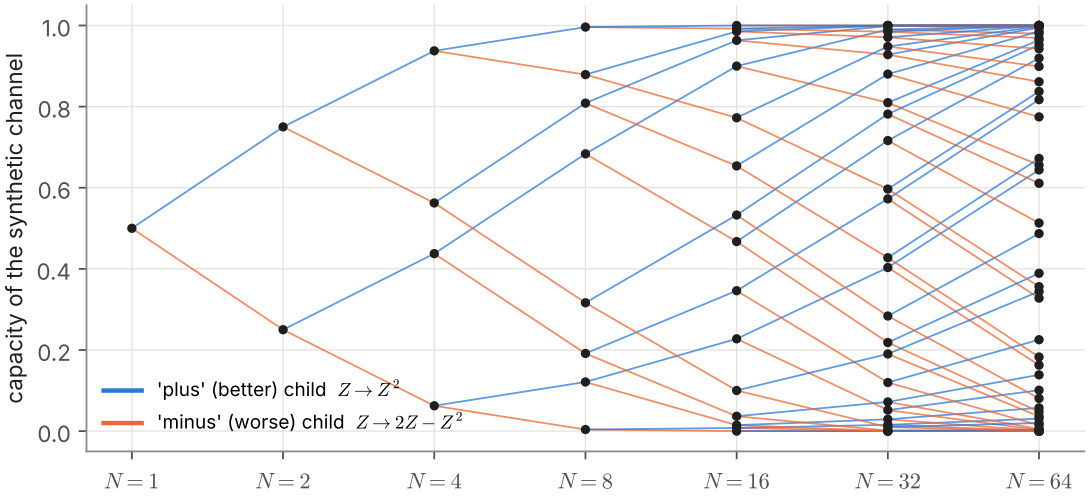


Figure 4.5. The polarization tree. Every channel splits into a better (blue) and a worse (orange) child. The paths spread towards 0 and 1.

For BEC(0.5) there is a nice symmetry: $Z(W_{16}^{(i)}) + Z(W_{16}^{(15-i)}) = 1$. The best eight channels are $\{7, 9, 10, 11, 12, 13, 14, 15\}$; their Z values add up to 1.2002 — still a poor SC bound at this tiny length.

4.5 The Bhattacharyya parameter

For general channels (BSC, AWGN) the synthetic channels are no longer of the same family, and we need a single number that measures reliability.

Definition 4.3 • Bhattacharyya parameter

For a binary-input channel W with output alphabet $\mathcal{Y}$,

$$Z(W) = \sum_{y \in \mathcal{Y}} \sqrt{W(y \mid 0)\, W(y \mid 1)} \qquad \Big(\text{an integral for continuous } y\Big).$$

$Z(W) \in [0, 1]$. $Z = 0$: the two inputs give disjoint outputs (perfect channel). $Z = 1$: the outputs have the same distribution for both inputs (useless channel).

Meaning. $Z(W)$ is an upper bound on the error probability of the best (maximum-likelihood) decision about one bit sent over W: $P_e(W) \le Z(W)$. It also bounds capacity: $I(W) \ge \log_2 \frac{2}{1+Z(W)}$ and $I(W) \le \sqrt{1-Z(W)^2}$.

Worked Example 4.2 • Bhattacharyya parameters of the three channels

BEC(ϵ). Only the output "?" has non-zero probability under both inputs: $Z = \sqrt{\epsilon \cdot \epsilon} = \epsilon$.
BSC(p). $Z = \sqrt{(1-p)p} + \sqrt{p(1-p)} = 2\sqrt{p(1-p)}$. For $p = 0.1$: $Z = 0.6$.
BI-AWGN. Completing the square in the Gaussian integral gives $Z = \int \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-1)^2+(y+1)^2}{4\sigma^2}} dy = e^{-1/(2\sigma^2)} = e^{-E_s/N_0}$. At $E_b/N_0 = 2\,\text{dB}$ and $R = 1/2$: $E_s/N_0 = 0.792$, $Z = e^{-0.792} = 0.453$.

4.5.1 Recursive reliability calculations

Definition 4.4 • Bhattacharyya recursion (Arıkan)

For any binary-input memoryless symmetric channel,

$$Z(W^+) = Z(W)^2, \qquad Z(W) \le Z(W^-) \le 2Z(W) - Z(W)^2,$$

with equality on the right for the BEC. Using the right-hand side as an approximation for all channels gives a simple (pessimistic) construction method.

```python
def bec_bhattacharyya(N, eps):
    """Z of the N bit-channels of a BEC(eps), natural index order."""
    if N == 1:
        return np.array([eps])
    zm = 2 * eps - eps ** 2              # minus channel (worse)
    zp = eps ** 2                        # plus  channel (better)
    return np.concatenate([bec_bhattacharyya(N // 2, zm),
                           bec_bhattacharyya(N // 2, zp)])
```

The function returns the first half for the minus branch and the second half for the plus branch, which matches the MSB-first rule in the note above. The cost is $O(N)$ operations.

4.6 Why do channels polarize?

Intuition

Each transform pushes a channel's capacity away from the middle: the plus child gains exactly what the minus child loses. A channel at $I = 0.5$ moves to 0.25 and 0.75; a channel at 0.9 moves to 0.81 and 0.99. Only channels already at 0 or 1 stay fixed. After many steps, nothing can stay in the middle.

Advanced • The martingale argument

Pick a random path: at each level choose minus or plus with probability 1/2. Let I_n be the

capacity of the channel reached after n steps. Because $I(W^-) + I(W^+) = 2I(W)$,

$$\mathbb{E}[I_{n+1} \mid I_n] = \tfrac{1}{2}I(W^-) + \tfrac{1}{2}I(W^+) = I_n,$$

so $\{I_n\}$ is a bounded martingale and converges almost surely to a limit I_∞. At the limit the transform must not move the capacity any more, which happens only for $I \in \{0, 1\}$. Since $\mathbb{E}[I_\infty] = I_0 = I(W)$, we get $P(I_\infty = 1) = I(W)$. This is exactly the polarization theorem: a fraction $I(W)$ of the channels becomes perfect. A similar argument on Z_n (with $Z_{n+1} = Z_n^2$ half of the time) gives the speed of polarization, $Z_n \approx 2^{-2^{n/2}} = 2^{-\sqrt{N}}$.

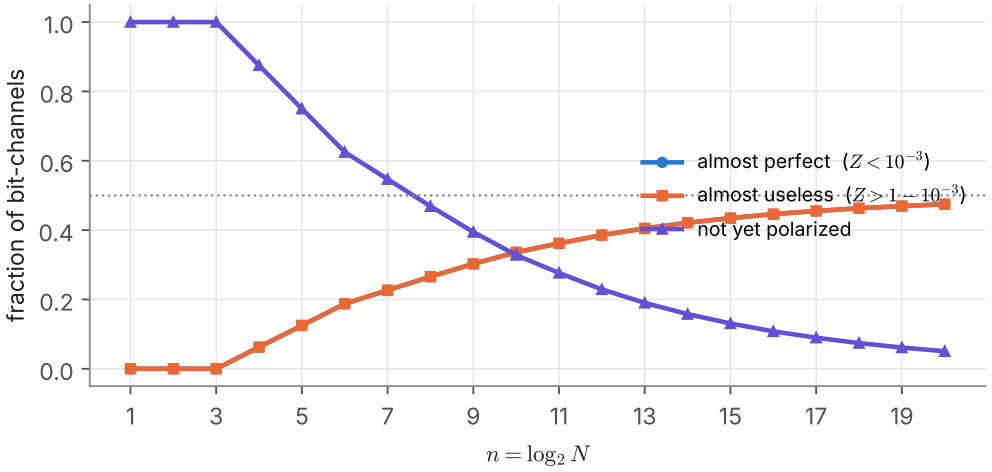


Figure 4.6. Fraction of polarized channels for BEC(0.5) as $N = 2^n$ grows. The "not yet polarized" fraction goes to zero, but slowly — even at $N = 2^{20} \approx 10^6$ it is not zero.

Professional insight

The slow disappearance of the middle channels (Fig. 4.6) explains the *finite-length gap*: at $N = 1024$ a noticeable fraction of channels is neither good nor bad. Choosing them as information bits causes errors; freezing them costs rate. Practical designs handle this with list decoding and CRC (Chapters 7–8) and with careful construction (Chapter 9).

4.7 Chapter summary

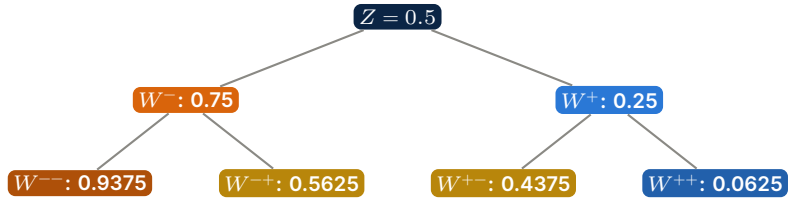
Key concepts

- Combining: $W_N(\mathbf{y}|\mathbf{u}) = W^N(\mathbf{y}|\mathbf{u}\mathbf{G}_N)$, built recursively from two copies of $W_{N/2}$.
- Splitting: $W_N^{(i)}$ has input u_i and output $(\mathbf{y}, u_0^{i-1})$; later bits are averaged out.
- One step creates a worse channel W^- and a better channel W^+ with the same total capacity.
- On the BEC: $Z^- = 2Z - Z^2$, $Z^+ = Z^2$, exact. In general these give bounds.
- Reliability of index i: read i in binary MSB first; 0 = minus, 1 = plus.

Important formulas

Bit-channel	$W_N^{(i)}(\mathbf{y}, u_0^{i-1}\|u_i) = \sum_{u_{i+1}^{N-1}} 2^{-(N-1)} W_N(\mathbf{y}\|\mathbf{u})$
Conservation	$I(W^-) + I(W^+) = 2I(W)$
Bhattacharyya	$Z(W) = \sum_y \sqrt{W(y\|0)W(y\|1)}$; BEC: ϵ, BSC: $2\sqrt{p(1-p)}$, AWGN: e^{-E_s/N_0}
Recursion	$Z^+ = Z^2$, $Z^- \le 2Z - Z^2$ (equality for BEC)

Visual recap • Chapter 4



minus: $Z \to 2Z - Z^2$ plus: $Z \to Z^2$ indices $0, 1, 2, 3$ from left to right

Common student mistakes

! Common mistake

- Applying the recursion LSB-first with the natural-order encoder of this book. With bit-reversed conventions (Arıkan's original $\mathbf{B}_N\mathbf{F}^{\otimes n}$) the rule changes; always check the convention.
- Thinking $Z(W^-) = 2Z - Z^2$ is exact for AWGN. It is only an upper bound outside the BEC.
- Confusing Z (small is good) with I (large is good) when ranking channels.

Practice problems

Problem 4.1. For BEC(0.4), compute Z for the four bit-channels of $N = 4$ and choose the information set for $K = 2$.

Solution $\epsilon^- = 0.64$, $\epsilon^+ = 0.16$. $Z_0 = 2(0.64) - 0.64^2 = 0.8704$, $Z_1 = 0.64^2 = 0.4096$, $Z_2 = 2(0.16) - 0.16^2 = 0.2944$, $Z_3 = 0.0256$. $\mathcal{A} = \{2, 3\}$. Check: capacities add to $4 \times 0.6 = 2.4$.

Problem 4.2. Compute $Z(W_8^{(5)})$ for BEC(0.5) using the binary rule.

Solution $5 = 101_2$: $0.5 \xrightarrow{+} 0.25 \xrightarrow{-} 0.4375 \xrightarrow{+} 0.1914$.

Problem 4.3. A BI-AWGN channel has $E_s/N_0 = 1$ (linear). Find $Z(W)$ and the upper bound for $Z(W^-)$.

Solution $Z = e^{-1} = 0.368$. $Z(W^-) \le 2(0.368) - 0.368^2 = 0.600$; $Z(W^+) = 0.135$.

Problem 4.4. Show that $I(W^-) + I(W^+) = 2I(W)$ holds for BEC(ϵ) using the erasure formulas.

Solution $(1 - 2\epsilon + \epsilon^2) + (1 - \epsilon^2) = 2 - 2\epsilon = 2(1 - \epsilon)$.

Self-assessment quiz

1. Which bit sees the better channel in W_2: u_0 or u_1?
2. What extra information is included in the output of $W_N^{(i)}$?
3. For the BEC, is the minus channel also a BEC? With which erasure probability?
4. True or false: $Z(W^+) = Z(W)^2$ for every symmetric channel.

Answers: 1) u_1. 2) The previous bits $u_0, \ldots, u_{i-1}$. 3) Yes, $2\epsilon - \epsilon^2$. 4) True.

CHAPTER

5



Polar Code Encoding

IN THIS CHAPTER YOU WILL LEARN

- the polar kernel $\mathbf{F}$ and the Kronecker product;
- how to write $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ for any N;
- recursive and stage-by-stage (butterfly) encoding;
- frozen-bit placement and information-bit selection;
- complete $N = 4$ and $N = 8$ examples with every XOR;
- why the encoder costs only $\frac{N}{2}\log_2 N$ XORs.

5.1 The polar encoding kernel

Definition 5.1 • Kernel

The 2×2 **polar kernel** is

$$\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \qquad [u_0\ u_1]\,\mathbf{F} = [\,u_0 \oplus u_1 \;\; u_1\,].$$

It is exactly the combining step of Fig. 4.1. Over GF(2), $\mathbf{F}\mathbf{F} = \mathbf{I}$, so the kernel is its own inverse.

Intuition

The kernel is the smallest non-trivial way to mix two bits: one output is the XOR of both, the other is a copy of one of them. Everything in polar coding is this little operation repeated.

5.2 The Kronecker product

Definition 5.2 • Kronecker product

For an $m \times n$ matrix $\mathbf{A}$ and any matrix $\mathbf{B}$,

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix}.$$

Each entry of $\mathbf{A}$ is replaced by that entry times the whole matrix $\mathbf{B}$. The n-fold power is $\mathbf{F}^{\otimes n} = \mathbf{F} \otimes \mathbf{F}^{\otimes(n-1)}$ with $\mathbf{F}^{\otimes 1} = \mathbf{F}$.

Because $\mathbf{F}$ has a 0 in the top-right corner, the recursion has a very simple block form:

$$\mathbf{G}_N = \mathbf{F} \otimes \mathbf{G}_{N/2} = \begin{bmatrix} \mathbf{G}_{N/2} & \mathbf{0} \\ \mathbf{G}_{N/2} & \mathbf{G}_{N/2} \end{bmatrix}.$$

Worked Example 5.1 • G_4 and G_8 step by step

$$\mathbf{G}_4 = \mathbf{F} \otimes \mathbf{F} = \begin{bmatrix} 1\cdot\mathbf{F} & 0\cdot\mathbf{F} \\ 1\cdot\mathbf{F} & 1\cdot\mathbf{F} \end{bmatrix} = \left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{array}\right],$$

$$\mathbf{G}_8 = \begin{bmatrix} \mathbf{G}_4 & \mathbf{0} \\ \mathbf{G}_4 & \mathbf{G}_4 \end{bmatrix} = \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{array}\right].$$

Row weights of $\mathbf{G}_8$: $1, 2, 2, 4, 2, 4, 4, 8$. In general the weight of row i is $2^{w(i)}$, where $w(i)$ is the number of ones in the binary form of i.

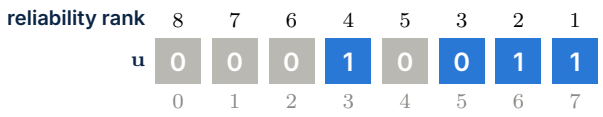
Advanced • Row weights, minimum distance and Reed–Muller codes

A polar code is spanned by the rows of $\mathbf{G}_N$ with indices in $\mathcal{A}$. Its minimum distance equals the smallest weight among those rows: $d_{\min} = \min_{i\in\mathcal{A}} 2^{w(i)}$. Reed–Muller codes use the same matrix but choose rows by *weight* (largest first); polar codes choose them by *bit-channel reliability*. For the $(8, 4)$ code with $\mathcal{A} = \{3, 5, 6, 7\}$ both rules agree and $d_{\min} = 4$; for long codes they differ. Polar codes have better SC performance, Reed–Muller codes larger distance; modern designs (e.g. PAC codes and 5G's sequence) mix both ideas.

5.3 Frozen-bit placement and information-bit selection

To encode we first build the length-N vector $\mathbf{u}$:

1. compute a reliability for each index i (Bhattacharyya parameter, GA, or a standard sequence);
2. put the K message bits, in increasing index order, on the K most reliable indices ($\mathcal{A}$);
3. set the other $N-K$ indices ($\mathcal{F}$) to the frozen value 0.



$K = 4$: the 4 most reliable indices $\{3,5,6,7\}$ receive the message $m = (1,0,1,1)$ in order: $u_3 = 1, u_5 = 0, u_6 = 1, u_7 = 1$.

Important note

The frozen value does not have to be 0 — any value known to both sides works, and 5G uses the freedom of "frozen" positions for parity-check bits in some cases (Chapter 10). For symmetric channels the error probability does not depend on the frozen values, so 0 is the standard choice.

The code itself is $\{\mathbf{u}\mathbf{G}_N : u_{\mathcal{F}} = \mathbf{0}\}$, i.e. the generator matrix of the (N,K) code is the sub-matrix $\mathbf{G}_{\mathcal{A}}$ made of the rows of $\mathbf{G}_N$ with indices in $\mathcal{A}$: $\mathbf{x} = \mathbf{u}_{\mathcal{A}}\mathbf{G}_{\mathcal{A}}$.

5.4 Recursive encoding

The block form of $\mathbf{G}_N$ gives a recursion. Split $\mathbf{u} = [\mathbf{u}_a \,|\, \mathbf{u}_b]$ into halves:

$$\mathbf{x} = \mathbf{u}\mathbf{G}_N = [\mathbf{u}_a\ \mathbf{u}_b]\begin{bmatrix}\mathbf{G}_{N/2} & \mathbf{0}\\ \mathbf{G}_{N/2} & \mathbf{G}_{N/2}\end{bmatrix} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \,|\, \mathbf{u}_b\mathbf{G}_{N/2}].$$

```python
def polar_encode_recursive(u):
    """x = u G_N with G_N = F^(x)n (no bit reversal)."""
    N = len(u)
    if N == 1:
        return u.copy()
    a, b = u[:N // 2], u[N // 2:]
    return np.concatenate([polar_encode_recursive(a ^ b),   # (u_a xor u_b) G_{N/2}
                           polar_encode_recursive(b)])      #  u_b G_{N/2}
```

Fig. 5.1 draws this recursion as a block diagram. One encoder of length N is one row of $N/2$ XOR gates followed by two encoders of length $N/2$; each of them is built in the same way, down to length 1, where $\mathbf{x} = \mathbf{u}$. Unrolling the recursion gives the $\log_2 N$ butterfly stages of the next section.

5.5 Encoding stages and butterfly diagrams

Unrolling the recursion gives $n = \log_2 N$ **stages**. In stage s ($s = 1, \dots, n$) the bits are grouped in blocks of size $2h$ with $h = N/2^s$; inside each block the second half is XORed into the first half:

$$v_j \leftarrow v_j \oplus v_{j+h} \quad \text{for the upper positions } j \text{ of each block.}$$

Drawn as a graph, each stage looks like a set of crossing "butterfly" wings.

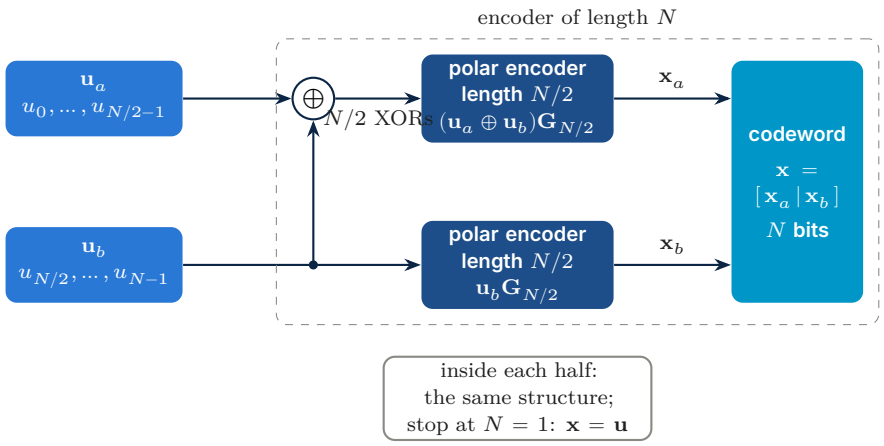


Figure 5.1. Block diagram. Recursive structure of the polar encoder: $\mathbf{x} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \,|\, \mathbf{u}_b\mathbf{G}_{N/2}]$.

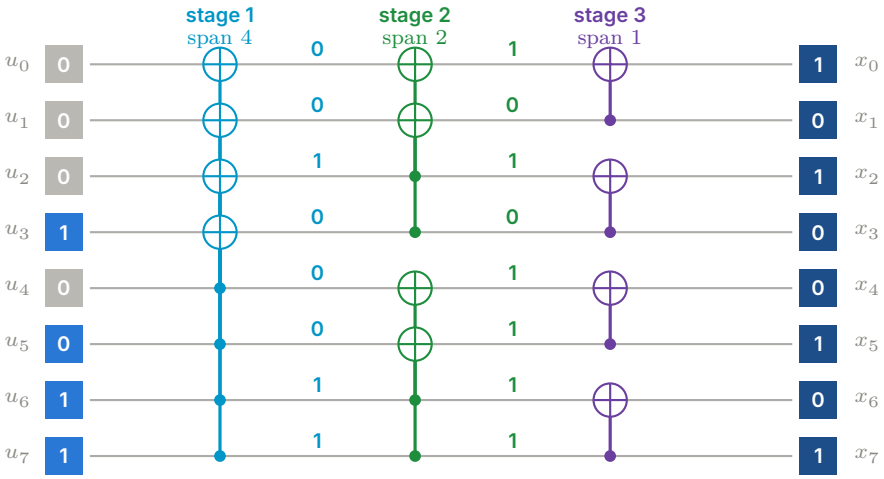


Figure 5.2. Butterfly encoder for $N = 8$ with the values of Worked Example 5.3. Small numbers are the intermediate vectors after stage 1 (cyan) and stage 2 (green). Each XOR node adds the bit coming up from the dot below.

Worked Example 5.2 • $N = 4$ encoding, every XOR shown

Code: $N = 4$, $K = 2$, BEC(0.5) ranking from Chapter 4 gives $\mathcal{A} = \{2, 3\}$, $\mathcal{F} = \{0, 1\}$. Message $m = (1, 1)$, so $\mathbf{u} = (0, 0, 1, 1)$.

Stage 1 (span 2): $v_0 = u_0 \oplus u_2 = 0 \oplus 1 = 1$, $v_1 = u_1 \oplus u_3 = 0 \oplus 1 = 1$, $v_2 = u_2 = 1$, $v_3 = u_3 = 1$.

Stage 2 (span 1): $x_0 = v_0 \oplus v_1 = 1 \oplus 1 = 0$, $x_1 = v_1 = 1$, $x_2 = v_2 \oplus v_3 = 1 \oplus 1 = 0$, $x_3 = v_3 = 1$.

Result: $\mathbf{x} = (0, 1, 0, 1)$.

Check with the matrix: rows 2 and 3 of $\mathbf{G}_4$ are 1010 and 1111; $1010 \oplus 1111 = 0101$.

All four codewords of this code: $m = 00 \to 0000$, $01 \to 1111$, $10 \to 1010$, $11 \to 0101$. Minimum distance 2.

Worked Example 5.3 • $N = 8$ encoding, every XOR shown

Code: $(8, 4)$, $\mathcal{A} = \{3, 5, 6, 7\}$, message $m = (1, 0, 1, 1)$, so $\mathbf{u} = (0, 0, 0, 1, 0, 0, 1, 1)$.

Stage 1 (span 4) — XOR u_{j+4} into u_j for $j = 0, 1, 2, 3$:

$$v_0 = u_0 \oplus u_4 = 0 \oplus 0 = 0 \qquad v_1 = u_1 \oplus u_5 = 0 \oplus 0 = 0$$
$$v_2 = u_2 \oplus u_6 = 0 \oplus 1 = 1 \qquad v_3 = u_3 \oplus u_7 = 1 \oplus 1 = 0$$
$$v_4 = u_4 = 0, \quad v_5 = u_5 = 0 \quad v_6 = u_6 = 1, \quad v_7 = u_7 = 1$$

$\mathbf{v} = (0, 0, 1, 0, 0, 0, 1, 1)$.
Stage 2 (span 2) — inside blocks $\{0..3\}$ and $\{4..7\}$:

$$w_0 = v_0 \oplus v_2 = 0 \oplus 1 = 1 \quad w_1 = v_1 \oplus v_3 = 0 \oplus 0 = 0 \quad w_2 = v_2 = 1 \quad w_3 = v_3 = 0$$
$$w_4 = v_4 \oplus v_6 = 0 \oplus 1 = 1 \quad w_5 = v_5 \oplus v_7 = 0 \oplus 1 = 1 \quad w_6 = v_6 = 1 \quad w_7 = v_7 = 1$$

$\mathbf{w} = (1, 0, 1, 0, 1, 1, 1, 1)$.
Stage 3 (span 1) — pairs $(0, 1), (2, 3), (4, 5), (6, 7)$:

$$x_0 = w_0 \oplus w_1 = 1 \quad x_1 = w_1 = 0 \quad x_2 = w_2 \oplus w_3 = 1 \quad x_3 = w_3 = 0$$
$$x_4 = w_4 \oplus w_5 = 0 \quad x_5 = w_5 = 1 \quad x_6 = w_6 \oplus w_7 = 0 \quad x_7 = w_7 = 1$$

Result: $\mathbf{x} = (1, 0, 1, 0, 0, 1, 0, 1)$, exactly as in Fig. 5.2.
Check with the matrix: $u_3 = u_6 = u_7 = 1$, so $\mathbf{x} = \text{row}_3 \oplus \text{row}_6 \oplus \text{row}_7 = 11110000 \oplus 10101010 \oplus 11111111 = 10100101$.

Stage order does not matter

Because XOR is linear and the stages act on different "bit positions" of the index, the stages may be executed in any order (span $1, 2, 4$ or $4, 2, 1$) and give the same $\mathbf{x}$. Hardware designers use this freedom.

5.6 Encoding complexity

Each stage performs $N/2$ XORs and there are $\log_2 N$ stages:

$$\text{XOR count} = \frac{N}{2} \log_2 N, \qquad \text{e.g. } N = 8 : \ 12, \quad N = 1024 : \ 5120.$$

A direct matrix product $\mathbf{u}\mathbf{G}_N$ would need up to $N^2/2 \approx 5 \times 10^5$ operations for $N = 1024$. The butterfly also shows that the encoder can be built with $\log_2 N$ pipeline stages of simple XOR gates.

```python
def polar_encode(u):
    """Butterfly encoder, works on a vector or on a (B, N) batch of vectors."""
    x = np.array(u, dtype=np.uint8, copy=True)
    one_d = (x.ndim == 1)
    if one_d:
        x = x[None, :]
    B, N = x.shape
    half = N // 2
    while half >= 1:                        # spans N/2, N/4, ..., 1
        x = x.reshape(B, -1, 2, half)       # blocks of 2*half bits
        x[:, :, 0, :] ^= x[:, :, 1, :]      # upper half ^= lower half
        x = x.reshape(B, N)
```

```
        half //= 2
    return x[0] if one_d else x
```

Inverse encoding is the same circuit

Since $\mathbf{G}_N\mathbf{G}_N = \mathbf{I}$ over GF(2), the same butterfly maps $\mathbf{x}$ back to $\mathbf{u} = \mathbf{x}\mathbf{G}_N$. Decoders use this to re-encode partial decisions (the "partial sums" of Chapter 6), and systematic encoders use it twice (Chapter 9).

5.7 Chapter summary

Key concepts

- $\mathbf{G}_N = \mathbf{F}^{\otimes n} = \begin{bmatrix} \mathbf{G}_{N/2} & 0 \\ \mathbf{G}_{N/2} & \mathbf{G}_{N/2} \end{bmatrix}$.
- $\mathbf{u}$ holds message bits on $\mathcal{A}$ and zeros on $\mathcal{F}$; $\mathbf{x} = \mathbf{u}\mathbf{G}_N$.
- The encoder has $\log_2 N$ stages of $N/2$ XORs (butterflies).
- Row i of $\mathbf{G}_N$ has weight $2^{w(i)}$; $d_{\min} = \min_{i\in\mathcal{A}} 2^{w(i)}$.

Important formulas

Kronecker recursion	$\mathbf{G}_N = \mathbf{F} \otimes \mathbf{G}_{N/2}$
Recursive encoding	$\mathbf{x} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \,\vert\, \mathbf{u}_b\mathbf{G}_{N/2}]$
Stage rule	$v_j \leftarrow v_j \oplus v_{j+h}$, $h = N/2^s$
Complexity	$\frac{N}{2}\log_2 N$ XORs

Visual recap • Chapter 5



$\mathbf{u} = 0011 \Rightarrow \mathbf{x} = 0101$
$\frac{N}{2}\log_2 N = 4$ XORs

Common student mistakes

! Common mistake

- Writing $\mathbf{F} = \left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$ and using row vectors — this is the transposed convention; results differ unless you also change the order of $\mathbf{u}$.
- Mixing the 5G (natural order) and Arıkan bit-reversed ($\mathbf{B}_N\mathbf{G}_N$) conventions in one design. The two give the same codes after re-indexing, but reliability tables are *not* interchangeable.
- Placing message bits on $\mathcal{A}$ in reliability order instead of index order — both sides must

use the same mapping; this book (and 5G) uses increasing index order.

Practice problems

Problem 5.1. Encode $\mathbf{u} = (1, 0, 1, 1)$ with $N = 4$, showing both stages.

Solution Stage 1: $v = (1 \oplus 1, 0 \oplus 1, 1, 1) = (0, 1, 1, 1)$. Stage 2: $x = (0 \oplus 1, 1, 1 \oplus 1, 1) = (1, 1, 0, 1)$. Matrix check: rows $0, 2, 3$: $1000 \oplus 1010 \oplus 1111 = 1101$.

Problem 5.2. For the $(8, 4)$ code with $\mathcal{A} = \{3, 5, 6, 7\}$, encode $m = (0, 1, 0, 1)$.

Solution $\mathbf{u} = (0, 0, 0, 0, 0, 1, 0, 1)$; rows 5 and 7: $11001100 \oplus 11111111 = 00110011$.

Problem 5.3. How many XORs does the butterfly encoder need for $N = 512$?

Solution $256 \times 9 = 2304$.

Problem 5.4. What is $d_{\min}$ of the polar code $N = 16$, $\mathcal{A} = \{7, 9, 10, 11, 12, 13, 14, 15\}$?

Solution Binary weights: $7 \to 3$, $9 \to 2$, $10 \to 2$, $11 \to 3$, $12 \to 2$, $13 \to 3$, $14 \to 3$, $15 \to 4$. Smallest row weight $2^2 = 4$, so $d_{\min} = 4$.

Self-assessment quiz

1. What is $\mathbf{F} \otimes \mathbf{I}_2$? Is it equal to $\mathbf{I}_2 \otimes \mathbf{F}$?
2. What is the inverse of $\mathbf{G}_N$ over GF(2)?
3. In stage 1 of an $N = 16$ encoder, which bit is XORed into v_3?
4. True or false: the weight of row 12 of $\mathbf{G}_{16}$ is 4.

Answers: 1) $\left[\begin{smallmatrix} \mathbf{I} & 0 \\ \mathbf{I} & \mathbf{I} \end{smallmatrix}\right]$; no, $\mathbf{I}_2 \otimes \mathbf{F} = \left[\begin{smallmatrix} \mathbf{F} & 0 \\ 0 & \mathbf{F} \end{smallmatrix}\right]$. 2) $\mathbf{G}_N$ itself. 3) u_{11}. 4) True ($12 = 1100_2$, two ones).

PART III

Advanced Level: Decoding Algorithms

Successive cancellation, list decoding, CRC aid, and the professional toolbox: fast decoders, construction methods and rate matching.

CHAPTER

6



Successive Cancellation (SC) Decoding

IN THIS CHAPTER YOU WILL LEARN

- why a smart decoder is needed;
- likelihood ratios and log-likelihood ratios (LLRs);
- the f and g functions, derived from first principles;
- the SC decoding tree and its message flow;
- how frozen bits are handled;
- a complete $N = 8$ decoding example, number by number;
- complexity, latency, strengths and weaknesses of SC.

6.1 Why decoding is necessary

The receiver observes $\mathbf{y} = (y_0, \dots, y_{N-1})$, a noisy version of the codeword. The best possible decision — maximum likelihood (ML) — compares all 2^K codewords and picks the most likely one. For $K = 100$ this is $2^{100} \approx 10^{30}$ comparisons: impossible. A practical decoder must exploit the structure of the code. Arıkan's **successive cancellation** (SC) decoder uses the recursive structure of $\mathbf{G}_N$ and needs only $O(N \log N)$ operations.

6.2 The successive cancellation concept

Intuition

Remember the bit-channels: $W_N^{(i)}$ is a good channel *if the earlier bits are known*. So the decoder decides the bits one after another, $u_0, u_1, \dots, u_{N-1}$, and when deciding u_i it pretends its own earlier decisions $\hat{u}_0, \dots, \hat{u}_{i-1}$ are correct. "Cancellation" means the effect of the already-decided bits is removed (cancelled) from the received signal before the next bit is decided — just like peeling layers off an onion.

Definition 6.1 • SC decoding rule

For $i = 0, 1, \dots, N-1$:

$$\hat{u}_i = \begin{cases} 0 \text{ (frozen value)} & i \in \mathcal{F}, \\ 0 & i \in \mathcal{A} \text{ and } L_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}) \geq 0, \\ 1 & i \in \mathcal{A} \text{ and } L_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}) < 0, \end{cases}$$

where $L_N^{(i)}$ is the LLR of the i-th bit-channel computed with the previous *decisions*.

Fig. 6.1 shows the decoder as hardware engineers draw it. The channel LLRs enter once. The tree processor computes the LLR of the current bit with f and g units; the leaf unit decides the bit (or forces 0 on a frozen position); and the decision goes back to the tree as a *partial sum*, which the next g operations need. This feedback loop, run N times, is the "successive" part of the name.

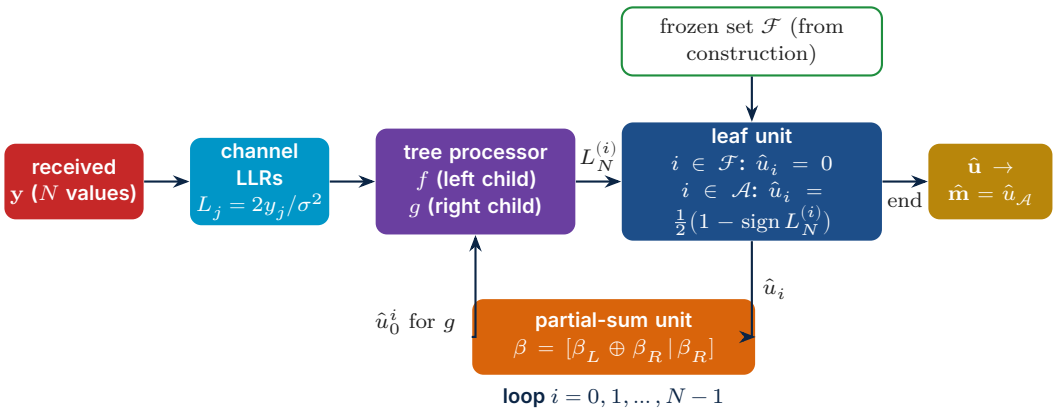


Figure 6.1. Block diagram. Architecture of the successive cancellation decoder. The decision of bit i is fed back (partial sums) before bit $i+1$ is computed.

6.3 Likelihood ratios and log-likelihood ratios

Definition 6.2 • LR and LLR

For a binary variable c and an observation y,

$$\Lambda(y) = \frac{P(y \mid c=0)}{P(y \mid c=1)} \quad \text{(likelihood ratio)}, \qquad L(y) = \ln \Lambda(y) \quad \text{(log-likelihood ratio, LLR)}.$$

With equally likely bits, $L = \ln \frac{P(c=0|y)}{P(c=1|y)}$. The **sign** of L is the hard decision ($L > 0 \Rightarrow 0$); the **magnitude** $|L|$ is the confidence.

6.3.1 LLR of BPSK over AWGN

With $s = 1 - 2c$ and $y = s + w$:

$$L(y) = \ln \frac{\exp\left(-\frac{(y-1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(y+1)^2}{2\sigma^2}\right)} = \frac{(y+1)^2 - (y-1)^2}{2\sigma^2} = \frac{4y}{2\sigma^2}.$$

$$L(y) = \frac{2y}{\sigma^2} \qquad (\text{BPSK}, 0 \mapsto +1,\ \sigma^2 = N_0/2).$$

The demodulator only scales y; the scale $2/\sigma^2$ tells the decoder how much to trust the channel.

Worked Example 6.1 • From received value to LLR and probability

$\sigma = 0.8$, $y = 0.4$. Then $L = 2(0.4)/0.64 = 1.25$, so $P(c = 0|y)/P(c = 1|y) = e^{1.25} = 3.49$ and $P(c = 0 \mid y) = 3.49/4.49 = 0.78$. A weak "0". With $y = -1.6$: $L = -5.0$, $P(c = 1 \mid y) = 0.993$ — a strong "1".

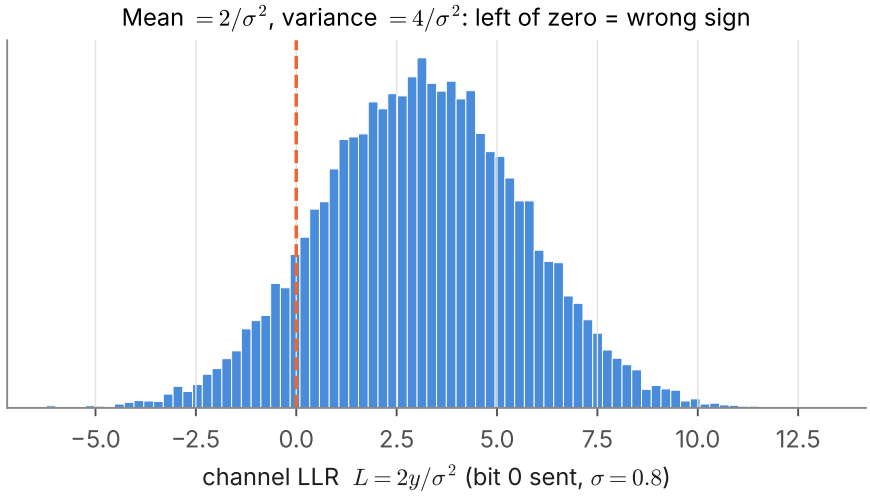


Figure 6.2. Histogram of channel LLRs when bit 0 is sent. Values left of zero would be hard-decision errors.

Useful LLR facts

- $P(c = 0) = \dfrac{1}{1 + e^{-L}}$, $P(c = 1) = \dfrac{1}{1 + e^{L}}$, and $\tanh(L/2) = P(c = 0) - P(c = 1)$.
- Independent observations of the same bit: LLRs *add*.
- The LLR of the XOR of two independent bits: tanh values *multiply* (next section).

6.4 The f and g functions

Everything in SC decoding follows from decoding the basic $N = 2$ code: $x_0 = u_0 \oplus u_1$, $x_1 = u_1$, with channel LLRs L_0 (for x_0) and L_1 (for x_1).

6.4.1 Deciding u_0: the f function

$u_0 = x_0 \oplus x_1$ and u_1 is unknown. Let $p_j = P(x_j = 0 \mid y_j)$. The XOR is 0 when both are 0 or both are 1:

$$P(u_0 = 0) = p_0 p_1 + (1 - p_0)(1 - p_1) \ \Rightarrow\ P(u_0 = 0) - P(u_0 = 1) = (2p_0 - 1)(2p_1 - 1).$$

Using $2p_j - 1 = \tanh(L_j/2)$:

$$L(u_0) = f(L_0, L_1) = 2\tanh^{-1}\Big(\tanh\tfrac{L_0}{2}\,\tanh\tfrac{L_1}{2}\Big) \;\approx\; \operatorname{sign}(L_0)\operatorname{sign}(L_1)\min\left(|L_0|, |L_1|\right).$$

The approximation is the **min-sum** rule. It is very accurate when one input is much larger than the other, never wrong in sign, and slightly overestimates the magnitude.

6.4.2 Deciding u_1 when u_0 is known: the g function

Now $u_1 = x_1$ is seen directly through L_1, and also through $x_0 = u_0 \oplus u_1$: if $u_0 = 0$, L_0 is another view of u_1; if $u_0 = 1$, it is a view of $\bar{u}_1$, so its sign flips. Independent views add:

$$L(u_1) = g(L_0, L_1, \hat{u}_0) = L_1 + (1 - 2\hat{u}_0)\, L_0.$$

Worked Example 6.2 • f and g on numbers

$L_0 = +1.2$, $L_1 = -0.4$.
Exact f: $\tanh(0.6) = 0.5370$, $\tanh(-0.2) = -0.1974$, product -0.1060, $f = 2\tanh^{-1}(-0.1060) = -0.2128$. Check with probabilities: $p_0 = 0.7685$, $p_1 = 0.4013$, $P(u_0 = 0) = 0.4470$, $\ln(0.4470/0.5530) = -0.2128$.
Min-sum f: $(+)(-)\min(1.2, 0.4) = -0.4$ (same sign, larger magnitude).
g: if $\hat{u}_0 = 0$, $g = -0.4 + 1.2 = +0.8$; if $\hat{u}_0 = 1$, $g = -0.4 - 1.2 = -1.6$.

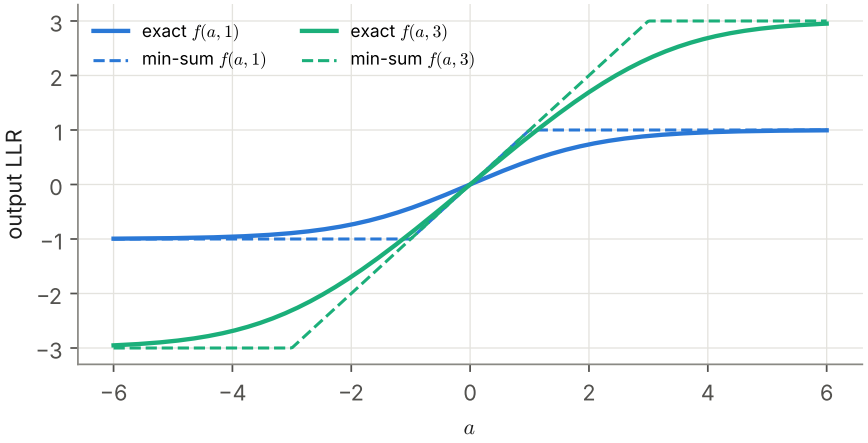


Figure 6.3. Exact f (solid) and its min-sum approximation (dashed) for $L_1 = 1$ and $L_1 = 3$. The output can never be more reliable than the less reliable input.

6.5 The SC decoding tree

For general N the same two operations are applied recursively. Recall the recursive encoder: $\mathbf{x} = [(\mathbf{u}_a \oplus \mathbf{u}_b)\mathbf{G}_{N/2} \,|\, \mathbf{u}_b\mathbf{G}_{N/2}]$. Writing $\mathbf{c}_a = \mathbf{u}_a\mathbf{G}_{N/2}$ and $\mathbf{c}_b = \mathbf{u}_b\mathbf{G}_{N/2}$, the left half of $\mathbf{x}$ is $\mathbf{c}_a \oplus \mathbf{c}_b$ and the right half is $\mathbf{c}_b$ — exactly the $N = 2$ code applied element by element. Hence:

Definition 6.3 • SC node operation

A node of size n receives an LLR vector α of length n (split into halves α_a, α_b) and returns a bit vector β (its re-encoded decisions, the **partial sums**):

1. left child input: $\alpha_L = f(\alpha_a, \alpha_b)$ (element-wise); it returns β_L;
2. right child input: $\alpha_R = g(\alpha_a, \alpha_b, \beta_L)$; it returns β_R;
3. output: $\beta = [\beta_L \oplus \beta_R \mid \beta_R]$.

A leaf (size 1) is one bit u_i: it decides $\hat{u}_i$ by the rule above and returns $\beta = \hat{u}_i$.

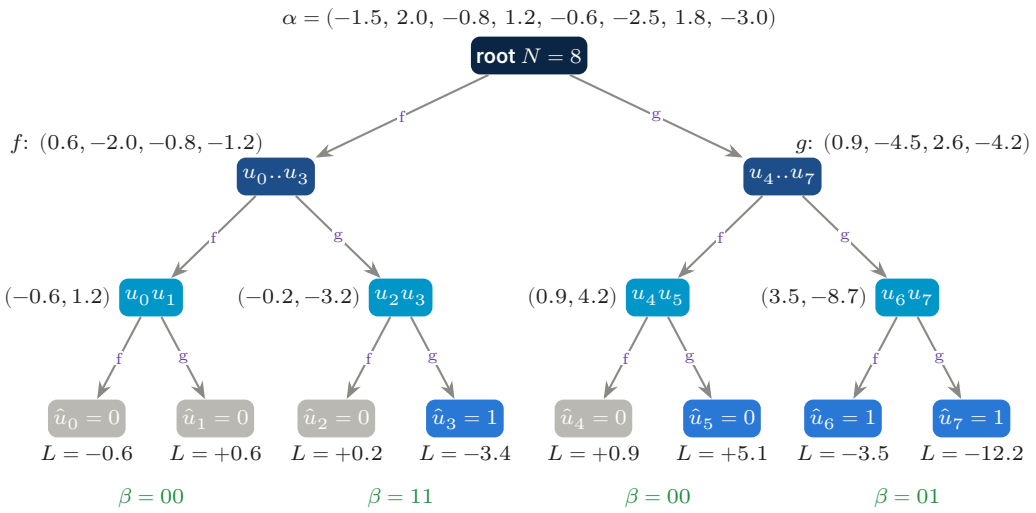


Figure 6.4. SC decoding tree for $N = 8$ with the numbers of Worked Example 6.3. LLRs travel down (f to the left child, g to the right child); partial sums β travel up. Leaves are visited left to right: $u_0, u_1, \ldots, u_7$. Grey leaves are frozen, blue leaves are information bits.

6.6 Hard decisions and frozen-bit handling

At a leaf:

- **information bit** ($i \in \mathcal{A}$): $\hat{u}_i = 0$ if $L \geq 0$, else 1 (hard decision on the bit-channel LLR);
- **frozen bit** ($i \in \mathcal{F}$): $\hat{u}_i = 0$ *whatever* L says. The LLR is still computed (it is needed for nothing else) — but the decoder trusts its prior knowledge.

! Do not "decode" a frozen bit

A frozen leaf may have a negative LLR (in our example u_0 has $L = -0.6$). Setting $\hat{u}_0 = 1$ because of the sign would be a serious bug: the partial sums would be wrong and every later decision would be corrupted. Frozen bits are *known*; this certainty is where the coding gain comes from.

6.7 Step-by-step numerical decoding example

Worked Example 6.3 • Complete SC decoding, $N = 8$, $K = 4$

Setting. The $(8, 4)$ code with $\mathcal{F} = \{0, 1, 2, 4\}$ sends $m = (1, 0, 1, 1)$, i.e. $\mathbf{u} = (0, 0, 0, 1, 0, 0, 1, 1)$ and $\mathbf{x} = (1, 0, 1, 0, 0, 1, 0, 1)$ (Chapter 5). BPSK gives $\mathbf{s} = (-1, +1, -1, +1, +1, -1, +1, -1)$. The received LLRs are

$$\alpha = (-1.5,\ 2.0,\ -0.8,\ 1.2,\ \mathbf{-0.6},\ -2.5,\ 1.8,\ -3.0).$$

Position 4 has the wrong sign (sent $+1$): a hard-decision receiver would make one error. We use min-sum f.

Step 1 — root, f to the left child ($\alpha_a = (-1.5, 2.0, -0.8, 1.2)$, $\alpha_b = (-0.6, -2.5, 1.8, -3.0)$):

$$f(-1.5, -0.6) = +0.6, \quad f(2.0, -2.5) = -2.0, \quad f(-0.8, 1.8) = -0.8, \quad f(1.2, -3.0) = -1.2.$$

Step 2 — node $u_0..u_3$, f to node u_0u_1: $f(0.6, -0.8) = -0.6$, $f(-2.0, -1.2) = +1.2$.
Step 3 — leaf u_0: $L = f(-0.6, 1.2) = -0.6$. Frozen $\Rightarrow \hat{u}_0 = 0$.
Step 4 — leaf u_1: $L = g(-0.6, 1.2, 0) = 1.2 + (-0.6) = +0.6$. Frozen $\Rightarrow \hat{u}_1 = 0$. Node u_0u_1 returns $\beta = (\hat{u}_0 \oplus \hat{u}_1, \hat{u}_1) = (0, 0)$.
Step 5 — g to node u_2u_3 with $\beta_L = (0, 0)$: $g(0.6, -0.8, 0) = -0.2$, $g(-2.0, -1.2, 0) = -3.2$.
Step 6 — leaf u_2: $L = f(-0.2, -3.2) = +0.2$. Frozen $\Rightarrow \hat{u}_2 = 0$.
Step 7 — leaf u_3: $L = g(-0.2, -3.2, 0) = -3.4$. Information, $L < 0 \Rightarrow \hat{u}_3 = 1$. Node returns $\beta = (0 \oplus 1, 1) = (1, 1)$.
Step 8 — node $u_0..u_3$ returns $[\beta_L \oplus \beta_R \,|\, \beta_R] = [(0, 0) \oplus (1, 1) \,|\, (1, 1)] = (1, 1, 1, 1)$.
Step 9 — root, g to the right child with $\beta_L = (1, 1, 1, 1)$, so $g = \alpha_b - \alpha_a$:

$$-0.6 + 1.5 = 0.9, \quad -2.5 - 2.0 = -4.5, \quad 1.8 + 0.8 = 2.6, \quad -3.0 - 1.2 = -4.2.$$

Step 10 — node $u_4..u_7$, f to node u_4u_5: $f(0.9, 2.6) = 0.9$, $f(-4.5, -4.2) = 4.2$.
Step 11 — leaf u_4: $L = f(0.9, 4.2) = 0.9$. Frozen $\Rightarrow \hat{u}_4 = 0$.
Step 12 — leaf u_5: $L = g(0.9, 4.2, 0) = 5.1$. Information $\Rightarrow \hat{u}_5 = 0$. Node returns $(0, 0)$.
Step 13 — g to node u_6u_7: the node inputs are $\alpha_a = (0.9, -4.5)$ and $\alpha_b = (2.6, -4.2)$ with $\beta_L = (0, 0)$, so $g(0.9, 2.6, 0) = 3.5$ and $g(-4.5, -4.2, 0) = -8.7$.
Step 14 — leaf u_6: $L = f(3.5, -8.7) = -3.5 \Rightarrow \hat{u}_6 = 1$.
Step 15 — leaf u_7: $L = g(3.5, -8.7, 1) = -8.7 - 3.5 = -12.2 \Rightarrow \hat{u}_7 = 1$. Node returns $(0, 1)$.
Step 16 — results. $\hat{\mathbf{u}} = (0, 0, 0, 1, 0, 0, 1, 1)$, message $\hat{m} = (\hat{u}_3, \hat{u}_5, \hat{u}_6, \hat{u}_7) = (1, 0, 1, 1)$ — **correct**. The root returns $\hat{\mathbf{x}} = (1, 0, 1, 0, 0, 1, 0, 1)$: the channel error at position 4 has been corrected.

Watch the reliabilities grow

The information bits were decided with $|L| = 3.4, 5.1, 3.5, 12.2$, much larger than most channel LLRs. The frozen bits had small or even negative LLRs $(-0.6, 0.6, 0.2, 0.9)$ — exactly the polarization predicted in Chapter 4.

6.8 A simple SC decoder in Python

This version mirrors the tree exactly and is written for clarity (the fast, vectorized version is in Chapter 11).

```python
def f(a, b):                        # min-sum check-node rule
    return np.sign(a) * np.sign(b) * np.minimum(np.abs(a), np.abs(b))

def g(a, b, c):                     # c = partial sums from the left child
    return b + (1 - 2 * c) * a

def sc_node(alpha, frozen, start, u_hat):
    """Decode the leaves start..start+len(alpha)-1; return the partial sums beta."""
    n = len(alpha)
    if n == 1:                                      # leaf: one bit u_start
        bit = 0 if frozen[start] else int(alpha[0] < 0)
        u_hat[start] = bit
        return np.array([bit])
    h = n // 2
    a, b = alpha[:h], alpha[h:]
    beta_L = sc_node(f(a, b), frozen, start, u_hat)            # left  child
    beta_R = sc_node(g(a, b, beta_L), frozen, start + h, u_hat) # right child
    return np.concatenate([beta_L ^ beta_R, beta_R])

def sc_decode(llr, frozen):
    u_hat = np.zeros(len(llr), dtype=int)
    sc_node(np.asarray(llr, float), frozen, 0, u_hat)
    return u_hat

frozen = np.array([1, 1, 1, 0, 1, 0, 0, 0], bool)       # F = {0,1,2,4}
print(sc_decode([-1.5, 2.0, -0.8, 1.2, -0.6, -2.5, 1.8, -3.0], frozen))
```

```
OUTPUT
[0 0 0 1 0 0 1 1]
```

6.9 Computational complexity

- At each of the $\log_2 N$ tree levels the decoder computes $N/2$ f values and $N/2$ g values in total (level with node size n: N/n nodes $\times$ $n/2$ f and $n/2$ g). Total: $N \log_2 N$ LLR operations, i.e. $O(N \log N)$.
- Partial sums: also $\frac{N}{2} \log_2 N$ XORs (it is a re-encoding).
- Memory: one LLR vector per tree level suffices if the nodes are visited depth-first: $N + N/2 + \cdots + 1 \approx 2N$ LLRs, plus $\approx 2N$ partial-sum bits: $O(N)$.
- Latency: a straightforward hardware tree decoder needs $2N - 2$ clock cycles (one per node visit when each level computes in one cycle). Fast simplifications reduce this a lot (Chapter 9).

Advanced • Why SC is exactly "bit-channel" decoding

One can prove that the LLR computed at leaf i by the recursion (with exact f) equals $\ln \frac{W_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}|0)}{W_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}|1)}$, the LLR of the synthetic channel of Chapter 4. We verified this numerically: for $N = 8$ and 200 random LLR vectors, the decoder in Chapter 11 reproduced the brute-force bit-channel LLRs (summing over all 2^{N-i-1} future bit patterns) for every leaf.

6.10 Advantages and limitations

Advantages	Limitations
$O(N \log N)$ complexity, tiny memory	**Error propagation**: one wrong decision corrupts all later partial sums
Achieves capacity as $N \to \infty$	Weak at short and medium lengths
Simple, regular hardware (f, g, XOR)	Inherently sequential: bit i waits for bits $0..i{-}1$
Only needs LLRs and the frozen set	No soft output (hard decisions only)

Professional insight

Error propagation is the key weakness. An SC decoder never reconsiders a decision: if $\hat{u}_3$ is wrong, the g functions for $u_4, \dots, u_7$ use wrong partial sums. The cure is to keep *several* candidate decisions alive — the list decoder of the next chapter.

6.11 Chapter summary

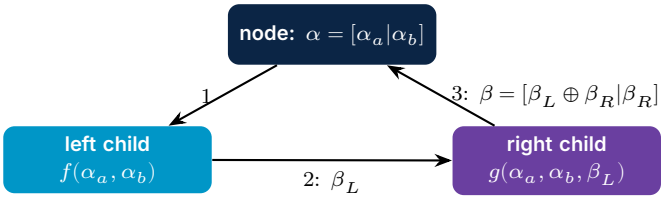
Key concepts

- SC decodes $u_0, \dots, u_{N-1}$ in order, using its own earlier decisions as if correct.
- BPSK/AWGN channel LLR: $L = 2y/\sigma^2$.
- f combines two LLRs of an XOR; g adds two views of the same bit after cancelling the known bit.
- The tree sends LLRs down and partial sums up; frozen leaves always decide 0.
- Complexity $O(N \log N)$, memory $O(N)$, but sequential and sensitive to error propagation.

Important formulas

Channel LLR	$L = 2y/\sigma^2$
f (exact / min-sum)	$2\tanh^{-1}(\tanh\frac{a}{2}\tanh\frac{b}{2}) \approx \operatorname{sign}(a)\operatorname{sign}(b)\min(\lvert a\rvert, \lvert b\rvert)$
g	$b + (1-2c)\,a$
Partial sums	$\beta = [\beta_L \oplus \beta_R \,\vert\, \beta_R]$
Complexity	$N \log_2 N$ f/g operations

Visual recap • Chapter 6



leaf: frozen $\to 0$; info $\to$ sign of L
cost $O(N \log N)$; error propagation!

Common student mistakes

! Common mistake

- Feeding the decoder y instead of $L = 2y/\sigma^2$. With min-sum f the decisions do not change under a positive scale factor, but the exact f, exact list metrics (Chapter 7) and fixed-point hardware need correct scaling.
- Using u bits instead of partial sums β in g. The g function needs the re-encoded left child, e.g. $(\hat{u}_0 \oplus \hat{u}_1, \hat{u}_1)$, not $(\hat{u}_0, \hat{u}_1)$.
- Mixing the order of halves: with this book's convention, f and g combine element j of the first half with element j of the second half.
- Making a hard decision on frozen bits.

Practice problems

Problem 6.1. Compute exact and min-sum $f(2.0, -3.0)$.

Solution $\tanh(1) = 0.7616$, $\tanh(-1.5) = -0.9051$, product -0.6893, $f = 2\tanh^{-1}(-0.6893) = -1.693$. Min-sum: -2.0.

Problem 6.2. $N = 2$, $K = 1$, $\mathcal{A} = \{1\}$. Received LLRs $(L_0, L_1) = (-0.5, +0.3)$. Decode with SC.

Solution u_0 frozen $= 0$. $L(u_1) = g(-0.5, 0.3, 0) = -0.2 \Rightarrow \hat{u}_1 = 1$, $\hat{\mathbf{x}} = (1, 1)$. This is the repetition code: the sum of the two LLRs decides.

Problem 6.3. $N = 4$, $\mathcal{A} = \{2, 3\}$, LLRs $(1.0, -2.0, -1.5, 0.5)$. Decode with min-sum SC.

Solution f to left: $f(1.0, -1.5) = -1.0$, $f(-2.0, 0.5) = -0.5$. u_0: frozen. u_1: frozen; $\beta_L = (0, 0)$. g to right: $(-1.5 + 1.0, 0.5 - 2.0) = (-0.5, -1.5)$. u_2: $f(-0.5, -1.5) = +0.5 \Rightarrow \hat{u}_2 = 0$ (two negative inputs give a positive XOR!). u_3: $g(-0.5, -1.5, 0) = -1.5 - 0.5 = -2.0 \Rightarrow \hat{u}_3 = 1$. $\hat{\mathbf{u}} = (0, 0, 0, 1)$, $\hat{\mathbf{x}} = (1, 1, 1, 1)$. Verified with the code above.

Problem 6.4. How many f and g evaluations does SC need for $N = 1024$?

Solution $\frac{N}{2}\log_2 N = 5120$ of each, 10240 in total.

Self-assessment quiz

1. What does the sign of an LLR mean? And its magnitude?

2. Why can the magnitude of $f(a, b)$ never exceed $\min(|a|, |b|)$?
3. What information flows up the SC tree?
4. Why is SC decoding hard to parallelize?

Answers: 1) Sign = hard decision, magnitude = confidence. 2) An XOR is only as reliable as its weakest input. 3) Partial sums (re-encoded decisions). 4) Each bit needs all earlier decisions.

CHAPTER

7



Successive Cancellation List (SCL) Decoding

IN THIS CHAPTER YOU WILL LEARN

- a real example where SC fails;
- path splitting and the list size L;
- the LLR-based path metric and its derivation;
- how candidates are pruned;
- a complete $L = 2$ example that corrects SC's mistake;
- SCL versus SC: simulation results;
- complexity and memory.

7.1 Why SC decoding is sometimes insufficient

Intuition

A chess player who must commit to every move without ever looking ahead will sometimes play a move that looks good now and is a disaster three moves later. SC is like this: it commits to each bit at once. A *list* decoder looks ahead a little: at every uncertain bit it keeps both possibilities alive and lets the *future* — in particular the frozen bits, whose values are known — decide which was right.

Worked Example 7.1 • SC makes a wrong decision

Same $(8, 4)$ code and same transmitted codeword as Worked Example 6.3 ($\mathbf{u} = (0, 0, 0, 1, 0, 0, 1, 1)$, $\mathbf{s} = (-1, +1, -1, +1, +1, -1, +1, -1)$), but a different noise realization:

$$\alpha = (-2.0,\ 0.3,\ -0.4,\ \mathbf{-2.0},\ 1.0,\ -5.8,\ 3.2,\ -4.8).$$

Only position 3 has the wrong sign, but it is a *strong* error (-2.0). Running SC:

u_0	u_1	u_2	u_3	u_4	u_5	u_6	u_7
frozen	frozen	frozen	info	frozen	info	info	info
-0.3	$+0.1$	-1.4	$\mathbf{+0.3}$	$\mathbf{-1.0}$	$+4.5$	-1.8	-14.1
0	0	0	**0** ×	0	0	1	1

At u_3 the LLR is $+0.3$: weak, but positive, so SC decides $\hat{u}_3 = 0$ — wrong. The decision is never reconsidered. Note the warning sign one step later: frozen bit u_4 (known to be 0) gets LLR -1.0, i.e. the decoder "believes" $u_4 = 1$. SC ignores this hint. The output $\hat{m} = (0, 0, 1, 1)$ is a block error.

7.2 Path splitting

Definition 7.1 • Decoding path

A **decoding path** is a candidate partial vector $\hat{u}_0^i = (\hat{u}_0, \dots, \hat{u}_i)$ together with its own copy of the internal decoder state (LLRs and partial sums) and a reliability score, the **path metric**.

At an information bit, each path **splits** into two children: one with $\hat{u}_i = 0$, one with $\hat{u}_i = 1$. At a frozen bit, each path has only one child ($\hat{u}_i = 0$). Without control the number of paths would double at every information bit and reach 2^K — this is ML decoding, which is too expensive.

Definition 7.2 • SCL decoder

The **successive cancellation list** decoder of list size L runs SC on up to L paths in parallel. After each split it keeps only the L paths with the best (smallest) path metrics. At the end it outputs the best path (or, with a CRC, the best path that passes the CRC — Chapter 8).

Fig. 7.1 shows one step of the SCL decoder as a block diagram. The list of L paths is a bank of L SC decoders that share the same channel LLRs. At an information bit every path splits, the metrics are updated, and a sorter keeps the L best candidates; the survivors may copy the internal state of their parent path.

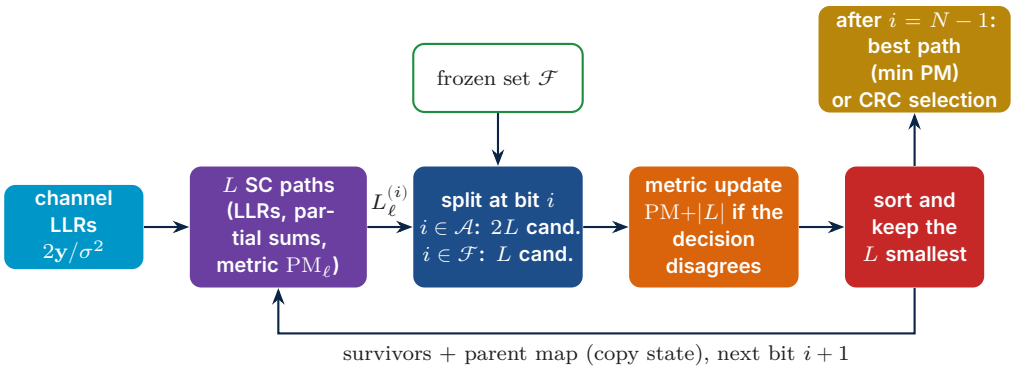


Figure 7.1. Block diagram. Successive cancellation list decoder: L parallel SC paths, path splitting, metric update and pruning. With $L = 1$ it is the SC decoder of Chapter 6.

7.3 Path metrics

7.3.1 Derivation

For path l with decisions $\hat{u}_0^i[l]$ we would like to rank by the probability $P(\hat{u}_0^i[l] \mid \mathbf{y})$. Following Balatsoukas-Stimming, Bastani Parizi and Burg (2015), define the path metric as the negative log

of this probability (up to a common constant). Using the chain rule,

$$P(\hat{u}_0^i \mid \mathbf{y}) = \prod_{j=0}^{i} P(\hat{u}_j \mid \mathbf{y}, \hat{u}_0^{j-1}), \qquad P(\hat{u}_j \mid \mathbf{y}, \hat{u}_0^{j-1}) = \frac{1}{1 + e^{-(1-2\hat{u}_j)L_j}},$$

where L_j is the bit-channel LLR computed on that path. Taking $-\ln$:

$$\mathrm{PM}_l^{(i)} = \sum_{j=0}^{i} \ln\left(1 + e^{-(1-2\hat{u}_j[l])\,L_j[l]}\right) \approx \sum_{j=0}^{i} \begin{cases} 0 & \text{if } \hat{u}_j \text{ agrees with } \operatorname{sign}(L_j), \\ |L_j| & \text{otherwise.} \end{cases}$$

The approximation uses $\ln(1 + e^x) \approx \max(0, x)$. It is the form used in this book and in most hardware.

A penalty system

Each path starts with PM $= 0$. Whenever a path takes a decision *against* the LLR, it pays a penalty equal to $|L|$ — the confidence of the LLR it contradicted. Contradicting a weak LLR is cheap; contradicting a strong one is expensive. Frozen bits also cost a penalty when their LLR says "1": this is how a wrong path gets punished later.

7.3.2 Update rule in practice

For path l at bit i with LLR L:

- frozen: PM $\leftarrow$ PM $+$ ($|L|$ if $L < 0$ else 0);
- information, child $\hat{u}_i = 0$: PM $+$ ($|L|$ if $L < 0$ else 0);
- information, child $\hat{u}_i = 1$: PM $+$ ($|L|$ if $L > 0$ else 0).

The child that follows the LLR keeps the parent's metric; the other child pays $|L|$.

7.4 Candidate pruning

After splitting, there are up to $2L$ candidates. The decoder sorts their metrics and keeps the L smallest; the others are deleted. Two details matter:

1. At the start there is one path. For the first $\log_2 L$ information bits the list grows $(1, 2, 4, \dots)$ and no pruning is necessary.
2. When a parent has *both* children kept, its state must be copied. When neither child is kept, the parent's memory is freed. Tal and Vardy's *lazy copy* shares memory between paths until a copy is really needed, keeping the complexity at $O(LN \log N)$.

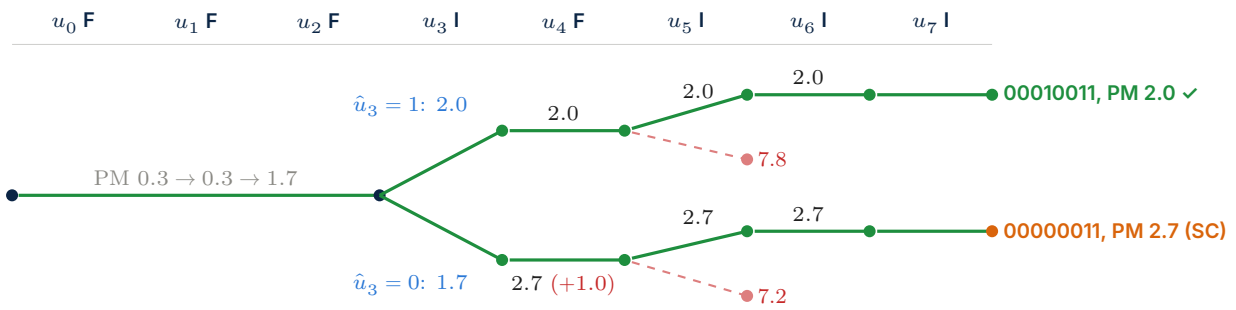


Figure 7.2. The surviving paths of the $L = 2$ decoder of Worked Example 7.2 (pruned candidates at u_5 shown dashed; at u_6 and u_7 the pruned candidates have metrics above 4.5 and are not drawn). The SC path (bottom) pays 1.0 at frozen bit u_4 and loses.

7.5 Worked SCL decoding example

Worked Example 7.2 • SCL with $L = 2$ corrects the SC error

Same received LLRs as Worked Example 7.1. For each candidate we show (decisions, PM, LLR of the bit on that path).

bit	type	candidates after the update (kept, pruned)
u_0	F	0 (0.3) $L = -0.3$, frozen decided 0 against the sign: $+0.3$
u_1	F	00 (0.3) $L = +0.1$
u_2	F	000 (1.7) $L = -1.4$: $+1.4$
u_3	I	$L = +0.3$: 0000 (1.7), 0001 (2.0) — only 2 candidates, both kept
u_4	F	path 0001: $L = +2.8 \Rightarrow$ 00010 (2.0); path 0000: $L = -1.0 \Rightarrow$ 00000 (2.7)
u_5	I	path 00010 ($L = +5.8$): 000100 (2.0), 000101 (7.8); path 00000 ($L = +4.5$): 000000 (2.7), 000001 (7.2)
u_6	I	path 000100 ($L = -6.6$): 0001001 (2.0), 0001000 (8.6); path 000000 ($L = -1.8$): 0000001 (2.7), 0000000 (4.5)
u_7	I	path 0001001 ($L = -15.5$): 00010011 (2.0), 00010010 (17.5); path 0000001 ($L = -14.1$): 00000011 (2.7), 00000010 (16.8)

Result. The best final path is $\hat{\mathbf{u}} = (0, 0, 0, 1, 0, 0, 1, 1)$ with PM = 2.0: message $(1, 0, 1, 1)$ — **correct**. The SC path $(0, 0, 0, 0, 0, 0, 1, 1)$ ends with PM = 2.7.

Why it worked. At u_3 the wrong choice was *slightly* cheaper (1.7 vs 2.0). But on the wrong path the frozen bit u_4 had $L = -1.0$ and cost a penalty of 1.0, while on the correct path it had $L = +2.8$ and cost nothing. The frozen bit acted as a free parity check. This is the essence of list decoding.

At the end, PM is a distance to the received signal

With the approximate metric, the final PM of a complete path equals the sum of $|\alpha_j|$ over the channel positions where its codeword $\hat{\mathbf{u}}\mathbf{G}_N$ disagrees with the hard decisions $\text{sign}(\alpha_j)$ (the *correlation discrepancy*). Here the hard decisions are $1, 0, 1, 1, 0, 1, 0, 1$.

- Correct path: codeword 10100101 differs only at position 3: PM $= |-2.0| = 2.0$
- SC path: codeword 01010101 differs at positions $0, 1, 2$: PM $= 2.0 + 0.3 + 0.4 = 2.7$

So with a large list the decoder picks the codeword closest to the received signal: SCL

approaches ML decoding.

7.6 SCL versus SC performance

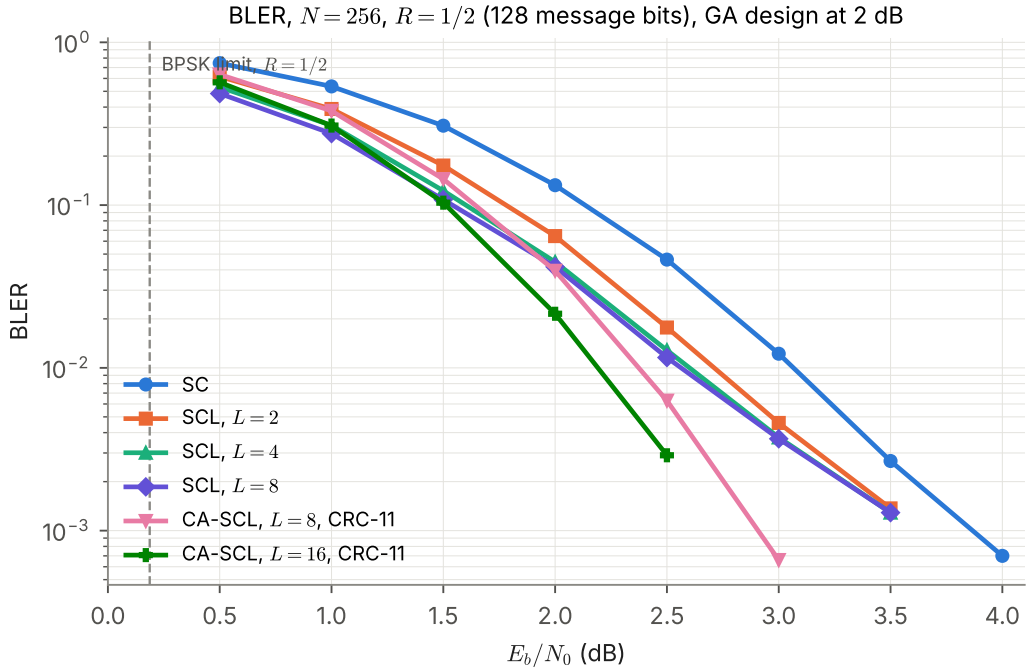


Figure 7.3. Monte-Carlo BLER of the $(256, 128)$ polar code over BPSK-AWGN (code of Chapter 11, GA construction at 2 dB, min-sum metrics, at least 150 block errors per point where possible).

BLER at $E_b/N_0 =$	2.0 dB	2.5 dB	3.0 dB	3.5 dB
SC	1.3×10^{-1}	4.6×10^{-2}	1.2×10^{-2}	2.7×10^{-3}
SCL, $L = 2$	6.5×10^{-2}	1.8×10^{-2}	4.6×10^{-3}	1.4×10^{-3}
SCL, $L = 4$	4.5×10^{-2}	1.3×10^{-2}	3.7×10^{-3}	1.3×10^{-3}
SCL, $L = 8$	4.2×10^{-2}	1.2×10^{-2}	3.7×10^{-3}	1.3×10^{-3}

Reading the simulation

- $L = 2$ already gains about 0.3–0.4 dB over SC at BLER 10^{-2}.
- Going from $L = 4$ to $L = 8$ gives almost nothing. The list decoder is already close to ML decoding of *this* code; the remaining errors are codewords that are genuinely closer to the received signal — the code's small minimum distance is the limit, not the decoder.
- The way forward is not a larger list but a better *code*: adding a CRC (next chapter) gives a large extra gain (green and pink curves).

7.7 Complexity and memory requirements

Computation	$O(LN \log N)$: L SC decoders in parallel (with lazy copy).
Sorting	at each of the K information bits, select the best L of $2L$ metrics.
Memory	$O(LN)$: L copies of the LLR and partial-sum memories (plus pointers).
Latency	similar to SC per bit, plus sorting delay; grows slowly with L.
Hardware cost	roughly linear in L; $L = 8$ is a common practical choice and was widely assumed in 3GPP performance evaluations.

Advanced • LLR-based versus probability-based SCL

Tal and Vardy's original decoder propagated pairs of probabilities and needed careful normalization. The LLR-based form above (Balatsoukas-Stimming et al., 2015) is numerically stable, uses the same f/g hardware as SC and needs only one extra adder per path for the metric. It is the standard in practice.

7.8 Chapter summary

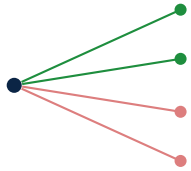
Key concepts

- SC's commit-at-once policy causes error propagation; SCL keeps up to L candidate paths.
- Information bits split paths; frozen bits do not split but can penalise wrong paths.
- Path metric: add $|L|$ when the decision disagrees with the LLR; keep the L smallest metrics.
- With large L, SCL approaches ML decoding of the polar code.
- Complexity $O(LN \log N)$, memory $O(LN)$.

Important formulas

Exact metric	$\mathrm{PM} \leftarrow \mathrm{PM} + \ln(1 + e^{-(1-2\hat{u}_i)L_i})$
Approximate metric	$\mathrm{PM} \leftarrow \mathrm{PM} + \lvert L_i \rvert \cdot [\hat{u}_i \neq \frac{1}{2}(1 - \operatorname{sign} L_i)]$
Pruning	keep L smallest of $\leq 2L$ candidates

Visual recap • Chapter 7



penalty = $|L|$ when a decision disagrees with the LLR
frozen bits punish wrong paths
L large ⇒ ML ⇒ limited by $d_{\min}$ ⇒ add CRC

Common student mistakes

! Common mistake

- Not penalising frozen bits. Without the frozen-bit penalty the list decoder loses most of its power.
- Sorting by *largest* metric. With the $-\ln P$ definition, small is good.
- Forgetting to copy the partial sums together with the LLRs when a path is duplicated.
- Feeding unscaled y values. With min-sum f and the approximate metric a common positive scale does not change any decision, but the exact f, the exact metric $\ln(1+e^{x})$ and fixed-point hardware (quantization ranges) all need correctly scaled LLRs ($2/\sigma^2$).

Practice problems

Problem 7.1. A path has PM $= 1.2$ and reaches an information bit with $L = -0.7$. Give the metrics of its two children.

Solution $\hat{u} = 1$ agrees with $L < 0$: 1.2. $\hat{u} = 0$ disagrees: $1.2 + 0.7 = 1.9$.

Problem 7.2. With $L = 2$, the four candidates after a split have metrics 3.1, 0.4, 2.2, 0.9. Which survive?

Solution 0.4 and 0.9.

Problem 7.3. Using the note on correlation discrepancy, compute the final approximate PM of codeword 11111111 for the LLRs of Example 7.1.

Solution Codeword bit 1 disagrees where $\alpha_j > 0$: positions $1, 4, 6$ with $0.3 + 1.0 + 3.2 = 4.5$.

Self-assessment quiz

1. What does SCL with $L = 1$ reduce to?
2. Do frozen bits split paths?
3. Why does going from $L = 4$ to $L = 8$ give little gain in Fig. 7.3?
4. What is the memory order of SCL?

Answers: 1) SC. 2) No, but they add penalties. 3) SCL is already near ML; the code's distance limits performance. 4) $O(LN)$.

CHAPTER

8



CRC-Aided SCL Decoding

IN THIS CHAPTER YOU WILL LEARN

- what a cyclic redundancy check (CRC) is;
- how to compute CRC bits by polynomial division;
- why a CRC makes list decoding so much stronger;
- where CRC bits are placed in a polar code;
- how the decoder selects the final candidate;
- a full transmitter/receiver chain and a real example;
- the price: rate loss and reduced error detection.

8.1 What is a CRC?

Intuition

A CRC is a short "fingerprint" of a message. The transmitter computes it and sends it with the message. The receiver recomputes the fingerprint from what it received; if the two do not match, something went wrong. Bank account numbers and ISBN numbers use the same idea with a single check digit; a CRC uses r check bits and catches almost all errors.

Definition 8.1 • Cyclic redundancy check

Write the A-bit message as a polynomial $m(D) = m_0 D^{A-1} + \cdots + m_{A-1}$ over GF(2). For a fixed **generator polynomial** $g(D)$ of degree r, the CRC bits are the coefficients of the remainder

$$p(D) = \left(m(D)\, D^r\right) \bmod g(D).$$

The transmitted word $[\mathbf{m}\,|\,\mathbf{p}]$ corresponds to $m(D)D^r + p(D)$, which is divisible by $g(D)$. The receiver accepts a word if its division by $g(D)$ leaves remainder 0.

Worked Example 8.1 • CRC by long division

Message $\mathbf{m} = 1101$, generator $g(D) = D^3 + D + 1$ (1011), $r = 3$. Append three zeros and divide (XOR, no carries):

$$\begin{array}{r} 1101000 \\ \oplus\ 1011 \\ \hline 0110000 \\ \oplus\ \ 1011 \\ \hline 0011100 \\ \oplus\ \ \ 1011 \\ \hline 0001010 \\ \oplus\ \ \ \ 1011 \\ \hline 0000001 \end{array}$$

Remainder $\mathbf{p} = 001$; transmitted word 1101 001. Check: $1101001 \div 1011$ leaves remainder 000. If any single bit is flipped the remainder is non-zero.

8.1.1 CRC polynomials used with polar codes in 5G NR

Name	$g(D)$ (TS 38.212, clause 5.1)	Used for
CRC6	$D^6 + D^5 + 1$	UCI, $12 \le A \le 19$ bits
CRC11	$D^{11} + D^{10} + D^9 + D^5 + 1$	UCI, $A \ge 20$ bits
CRC24C	$D^{24} + D^{23} + D^{21} + D^{20} + D^{17} + D^{15} + D^{13} + D^{12} + D^8 + D^4 + D^2 + D + 1$	DCI and PBCH

8.1.2 The CRC encoder as a shift register

Long division is easy on paper, but hardware computes the CRC with a **linear feedback shift register** (LFSR): r one-bit memory cells and one XOR gate for every non-zero coefficient of $g(D)$. Fig. 8.1 shows the encoder of CRC6, $g(D) = D^6 + D^5 + 1$. The registers start at zero, the A message bits enter one per clock, and after the last bit the registers hold the 6 CRC bits (read r_5 first). No zeros need to be appended: this "pre-multiplied" form computes $m(D)D^6 \bmod g(D)$ directly. We checked that it gives the same CRC as the long division of `crc_bits` for 200 random messages; for example, $\mathbf{m} = 1011001011$ gives $\mathbf{p} = 011011$.

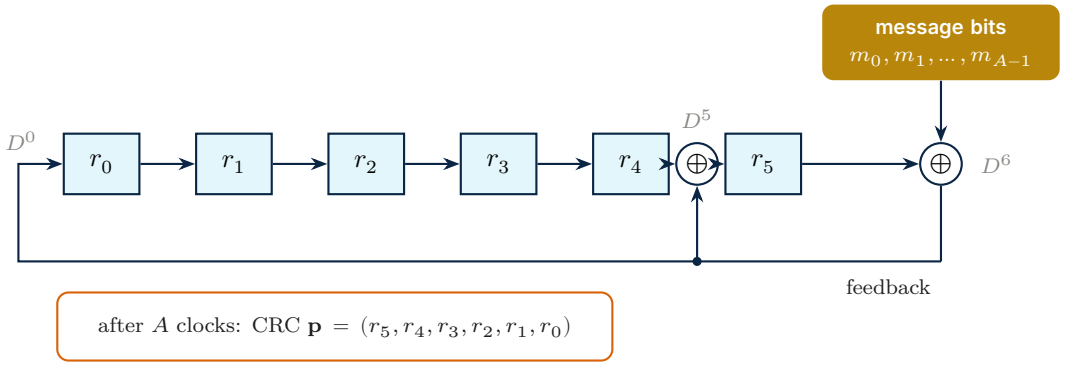


Figure 8.1. Block diagram. LFSR encoder of CRC6 ($g(D) = D^6 + D^5 + 1$). The XOR gates sit at the positions of the non-zero coefficients D^0, D^5 and D^6.

8.1.3 Properties

- A CRC is a *linear* code: $\mathbf{p} = \mathbf{mP}$ for an $A \times r$ binary matrix $\mathbf{P}$. Our Python class builds $\mathbf{P}$ once and then computes all CRCs with one matrix product.
- It detects every single error, every burst of length $\leq r$, and (if $(D+1)$ divides $g(D)$) every odd number of errors.
- For a random wrong word, the probability that it passes the check is about 2^{-r}.

8.2 How a CRC helps polar decoding

Intuition

In Chapter 7 the list decoder often *had* the correct message in its list — just not at the top. The path metric alone cannot always tell which candidate is right. A CRC is an independent judge: among the L final candidates, choose the best one that also has a correct fingerprint.

Seen as a code, the CRC and the polar code form a **concatenated code**: outer CRC, inner polar. The concatenation has a much larger minimum distance than the polar code alone, and the list decoder is an efficient (approximate) ML decoder for the concatenation. This is why CA-SCL with moderate L beats SCL with any L.

8.3 CRC attachment

1. The A message bits get r CRC bits: payload length $K = A + r$.
2. The K payload bits are placed on the K most reliable positions $\mathcal{A}$ (in index order, CRC last).
3. The rest is a normal polar encoding of length N.



$K = 16$ payload bits on $\mathcal{A}$ of an $N = 32$ polar code. Message rate $A/N = 0.3125$.

! Account for the CRC in E_b/N_0

The CRC bits carry no user information. A fair comparison uses the *message* rate $R = A/N$ when converting E_b/N_0 to noise variance. In our simulations, SCL uses $K = 128$ message bits and CA-SCL uses 128 message bits $+11$ CRC bits $= 139$ payload bits, so both have $R = 1/2$.

8.4 CRC-based candidate selection

Definition 8.2 • CA-SCL decoder

Run SCL with list size L. At the end, sort the L surviving paths by path metric. Output the first (smallest PM) path whose payload passes the CRC. If no path passes, declare a failure (or output the best path and flag it).

```
payload = u[:, :, info]   # u: (B, L, N), paths sorted by PM; payload = message + CRC
ok = crc.check(payload) & np.isfinite(PM)
pick = np.where(ok.any(axis=1), ok.argmax(axis=1), 0)   # first passing path
u_hat = u[np.arange(B), pick, :]
```

Fig. 8.2 draws this rule as a flow chart. The CRC is checked on at most L candidates, starting with the most likely one.

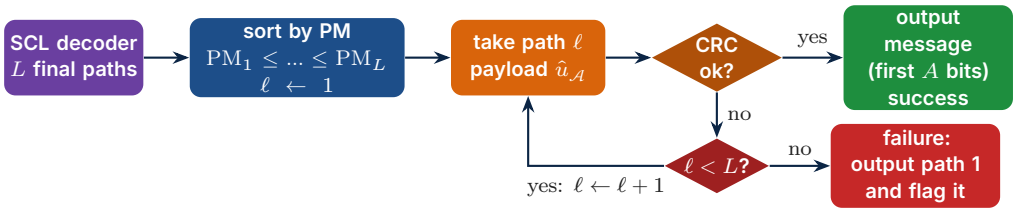


Figure 8.2. Block diagram. Candidate selection in the CRC-aided SCL decoder.

8.5 Complete transmitter and receiver workflow

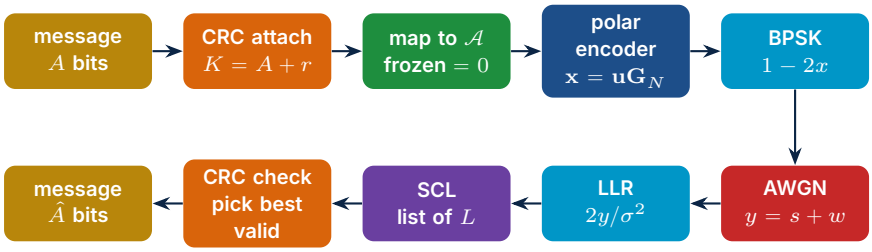


Figure 8.3. CRC-aided polar coding chain used in the simulations of this book.

8.6 Practical examples

Worked Example 8.2 • The CRC picks the third path

Code: $N = 32$, $A = 10$, CRC6, $K = 16$, GA construction at 2 dB: $\mathcal{A} = \{11, 13, 14, 15, 19, 21, \ldots, 31\}$. Message 1011001001; CRC6 bits 011011. One noisy reception at $E_b/N_0 = 1$ dB (LLRs rounded to one decimal) gives this final list ($L = 4$):

rank	payload (message | CRC)	PM	CRC	correct?
1	`1011001000 | 010111`	5.6	fail	no
2	`0000100100 | 100100`	6.3	fail	no
3	`1011001001 | 011011`	7.7	pass	**yes**
4	`0111011010 | 100011`	8.0	fail	no

Plain SCL would output rank 1 (one message bit wrong). CA-SCL skips the two paths that fail the CRC and outputs rank 3 — the transmitted message. (Produced with the library of Chapter 11, random seed 11.)

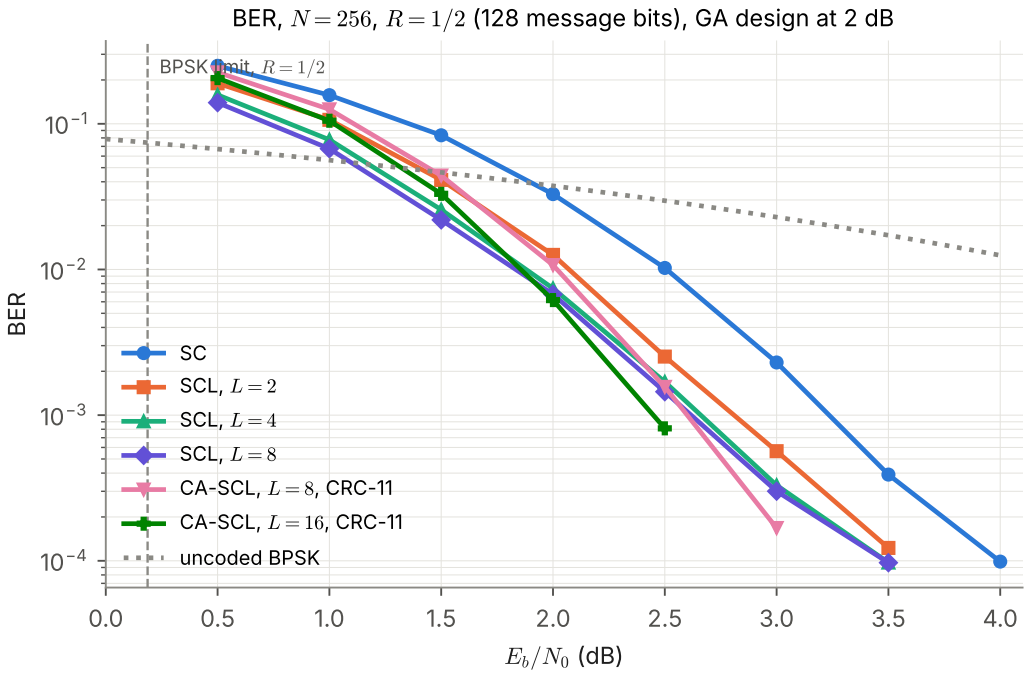


Figure 8.4. BER for the same simulation as Fig. 7.3. CA-SCL with CRC-11 gives the steepest curves.

Worked Example 8.3 • Reading the simulation results

E_b/N_0 needed to reach a given BLER, $N = 256$, $R = 1/2$ (Fig. 7.3, log-linear interpolation):

Decoder	BLER 10^{-1}	BLER 10^{-2}	BLER 10^{-3}
SC	2.13 dB	3.07 dB	3.87 dB
SCL, $L = 8$	1.54 dB	2.56 dB	> 3.5 dB
CA-SCL, $L = 8$, CRC11	1.64 dB	2.37 dB	2.91 dB
CA-SCL, $L = 16$, CRC11	1.51 dB	2.19 dB	> 2.5 dB

- At high BLER (10^{-1}) the CRC costs a little: 11 extra payload bits raise the inner code rate from 0.500 to 0.543.
- At BLER 10^{-3} the CRC gains about 1 dB over SC, and the curve is much steeper than plain SCL.

- Doubling the list from 8 to 16 with the CRC still helps ($\approx 0.2\,\text{dB}$ at 10^{-2}), unlike without CRC.

8.7 The price of the CRC

1. **Rate loss.** r payload bits carry no data. For small blocks this matters: CRC24 on a 40-bit DCI is a large overhead. 5G therefore uses short CRCs (6, 11 bits) for small UCI.
2. **Less error detection.** The receiver uses the CRC both to *choose* among L paths and to *detect* errors. With L candidates the chance that some wrong path passes is about $L \cdot 2^{-r}$, i.e. roughly $\log_2 L$ CRC bits are "spent" on selection. With CRC24 and $L = 8$, about 21 bits of protection remain.

Distributed CRC in 5G downlink

In 5G DCI the CRC bits are *interleaved* among the information bits so that each CRC bit is placed right after the information bits it depends on. The decoder can then check CRC bits *during* decoding and kill wrong paths early (early termination), which saves power when a UE blindly tries many PDCCH candidates (Chapter 10).

8.8 Chapter summary

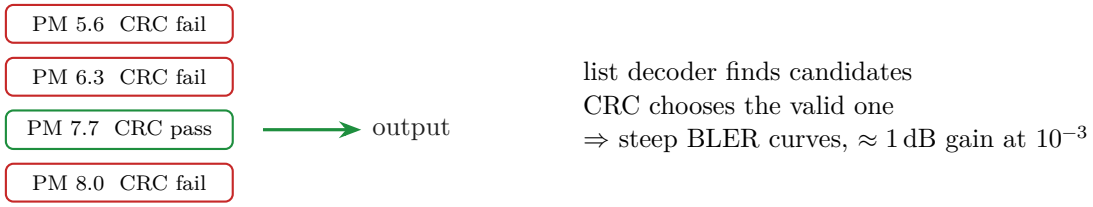
Key concepts

- A CRC appends the remainder of $m(D)D^r$ divided by $g(D)$; it is a linear code.
- CA-SCL selects the best path that passes the CRC; this effectively decodes a concatenated code with large distance.
- Use the message rate A/N for E_b/N_0; CRC bits are overhead.
- The CRC gives large gains at low BLER and makes larger lists useful.

Important formulas

CRC bits	$p(D) = m(D)D^r \bmod g(D)$
Payload	$K = A + r$, message rate $R = A/N$
False pass with a list	$\approx L \cdot 2^{-r}$

Visual recap • Chapter 8



Common student mistakes

! Common mistake

- Comparing CA-SCL and SCL at the same *payload* rate K/N instead of the same message rate A/N.
- Initialising the CRC register differently at transmitter and receiver (5G DCI starts with 24 ones prepended; our simple examples start from zero).
- Assuming CRC-r still detects errors with probability $1-2^{-r}$ after it has been used for list selection.

Practice problems

Problem 8.1. Compute the CRC of $\mathbf{m} = 1011$ with $g(D) = D^3 + D + 1$.

Solution $1011000 \oplus 1011000 = 0000000$: remainder 000. The message itself is $g(D)$, so its multiple $m(D)D^3$ is divisible by $g(D)$.

Problem 8.2. A DCI of $A = 40$ bits uses CRC24C and $N = 128$. What are K and the message rate?

Solution $K = 64$, payload rate 0.5, message rate $40/128 = 0.3125$.

Problem 8.3. With CRC6 and $L = 8$, estimate the probability that a wrong list contains a path passing the CRC.

Solution $\approx 8 \cdot 2^{-6} = 0.125$ — CRC6 alone is weak for detection; this is why 5G adds parity-check bits for short UCI.

Self-assessment quiz

1. Why does a CRC help list decoding more than SC decoding?
2. Where are the CRC bits placed in the polar input vector?
3. What does CA-SCL output when no path passes the CRC?
4. Which CRC protects 5G DCI?

Answers: 1) SC has only one candidate to check. 2) On information positions, after (or interleaved with) the message bits. 3) A failure flag or the best-PM path. 4) CRC24C.

CHAPTER

9



Additional Professional Topics

IN THIS CHAPTER YOU WILL LEARN

- systematic polar encoding;
- belief propagation (BP) and SCAN decoding;
- Fast-SSC: decoding whole sub-trees at once;
- adaptive SCL;
- code construction: Bhattacharyya, density evolution, Gaussian approximation, polarization weight;
- puncturing, shortening, repetition and rate matching;
- finite-length behaviour and hardware aspects.

9.1 Systematic polar encoding

Intuition

In the normal (non-systematic) code, the message bits are hidden inside $\mathbf{u}$ and do not appear directly in $\mathbf{x}$. In a *systematic* code the message appears unchanged in the codeword. The block error rate is the same, but the *bit* error rate is lower, because a decoding error usually changes only a few of the visible message bits.

Definition 9.1 • Systematic polar code (Arıkan, 2011)

A systematic polar code uses the same codebook $\{\mathbf{u}\mathbf{G}_N : u_{\mathcal{F}} = \mathbf{0}\}$ but a different mapping from message to codeword: the message is placed in the codeword positions $\mathcal{A}$, i.e. $x_{\mathcal{A}} = \mathbf{m}$.

A simple encoder uses the butterfly twice (valid for frozen sets produced by the usual constructions):

1. put $\mathbf{m}$ on $\mathcal{A}$ and zeros on $\mathcal{F}$, and encode: $\mathbf{v} = \mathbf{u}\mathbf{G}_N$;
2. set the frozen positions of $\mathbf{v}$ to zero: $v_{\mathcal{F}} = \mathbf{0}$;
3. encode again: $\mathbf{x} = \mathbf{v}\mathbf{G}_N$.

Fig. 9.1 shows the same three steps as a block diagram.

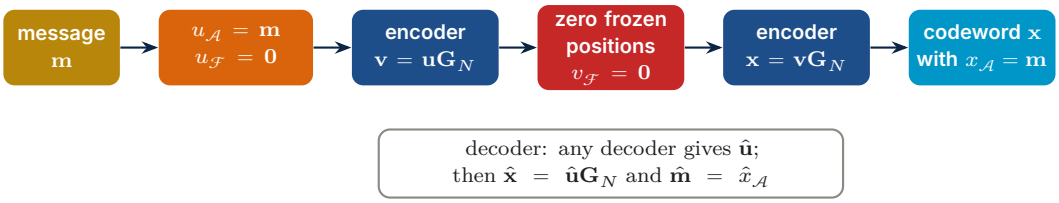


Figure 9.1. Block diagram. Systematic polar encoder built from two ordinary (non-systematic) encoders.

Worked Example 9.1 • Systematic encoding of the $(8, 4)$ code

$\mathcal{A} = \{3, 5, 6, 7\}$, $\mathbf{m} = (1, 0, 1, 1)$.
Step 1: $\mathbf{u} = 00010011 \Rightarrow \mathbf{v} = 10100101$. Step 2: zero positions $0, 1, 2, 4$: $\mathbf{v} = 00000101$.
Step 3: $\mathbf{x} = \mathbf{v}\mathbf{G}_8 = 00110011$.
Check: $x_3, x_5, x_6, x_7 = 1, 0, 1, 1 = \mathbf{m}$. It is a valid codeword: $\mathbf{x}\mathbf{G}_8 = 00000101$ has zeros on $\mathcal{F}$. (We also verified this procedure for GA-designed codes with $N = 64$ and 256.)
Decoding. Run any decoder to get $\hat{\mathbf{u}}$, re-encode $\hat{\mathbf{x}} = \hat{\mathbf{u}}\mathbf{G}_N$, and read $\hat{x}_{\mathcal{A}}$.

9.2 Belief propagation (BP) decoding

BP decoding runs message passing on the butterfly graph of Fig. 5.2. Each butterfly is a small *processing element* with inputs (a, b) on the left and outputs $(a \oplus b,\ b)$ on the right. Messages L travel from right (channel) to left; messages R travel from left (frozen side) to right.

Definition 9.2 • BP update of one processing element

With $L_{\text{up}}, L_{\text{low}}$ arriving from the right and R_a, R_b from the left:

$$L_a = f(L_{\text{up}},\, L_{\text{low}} + R_b), \quad R_{\text{up}} = f(R_a,\, L_{\text{low}} + R_b),$$
$$L_b = f(L_{\text{up}},\, R_a) + L_{\text{low}}, \quad R_{\text{low}} = f(R_a,\, L_{\text{up}}) + R_b.$$

Initialization: channel LLRs on the right; $R = +\infty$ (a very large number) for frozen inputs and 0 for information inputs on the left. Iterate until the hard decisions satisfy $\hat{\mathbf{x}} = \hat{\mathbf{u}}\mathbf{G}_N$ or a maximum number of iterations is reached.

Derivation in one line. The element enforces $x_{\text{up}} = a \oplus b$ and $x_{\text{low}} = b$: each output message combines the other messages with f when an XOR relation is involved and adds them when two views of the same bit are available — exactly the f/g logic of Chapter 6, but without a fixed schedule.

BP advantages	BP drawbacks
fully parallel, high throughput; soft outputs (useful for iterative receivers)	needs many iterations (tens); performance usually below SCL with CRC; larger memory

9.3 SCAN decoding

The **soft cancellation** (SCAN) decoder of Fayyaz and Barry (2014) keeps the SC schedule (left to right through the tree) but, like BP, sends *soft* messages back instead of hard partial sums. One or

two iterations already give most of the gain; it produces soft outputs with far fewer computations than BP and is attractive for turbo-equalization and iterative detection.

9.4 Fast-SSC decoding

Intuition

SC visits every leaf, even when a whole sub-tree is "easy". If a node contains only frozen bits, we already know its output: all zeros. If it contains only information bits, it is an uncoded block: just take hard decisions. Fast decoders recognise such special nodes and decode them in one step.

Node type	frozen pattern	one-step decoding of the node's partial sums β
Rate-0	all frozen	$\beta = \mathbf{0}$ (no computation)
Rate-1	all information	$\beta_j = \frac{1}{2}(1 - \operatorname{sign}\alpha_j)$ (hard decisions)
Repetition (Rep)	only the last bit is information	$\beta_j = b$ for all j, with $b = 0$ if $\sum_j \alpha_j \geq 0$, else 1
Single parity check (SPC)	only the first bit is frozen	hard decisions; if their XOR is 1, flip the bit with the smallest $\lvert\alpha_j\rvert$

Simplified SC (SSC) by Alamdar-Yazdi and Kschischang (2011) introduced Rate-0 and Rate-1 nodes; Fast-SSC by Sarkis et al. (2014) added Rep and SPC nodes and more. The Rate-0, Rate-1 and Rep rules give exactly the SC decisions (with min-sum f); the SPC rule is the maximum-likelihood (Wagner) decision for the node, so the overall performance is the same as SC or marginally better — with far fewer steps.

Fig. 9.2 shows the node processor of a Fast-SSC decoder. Before decoding, every node of the tree is classified once from the frozen pattern; during decoding a special node is finished in one step, and only the other nodes are split further with f and g.

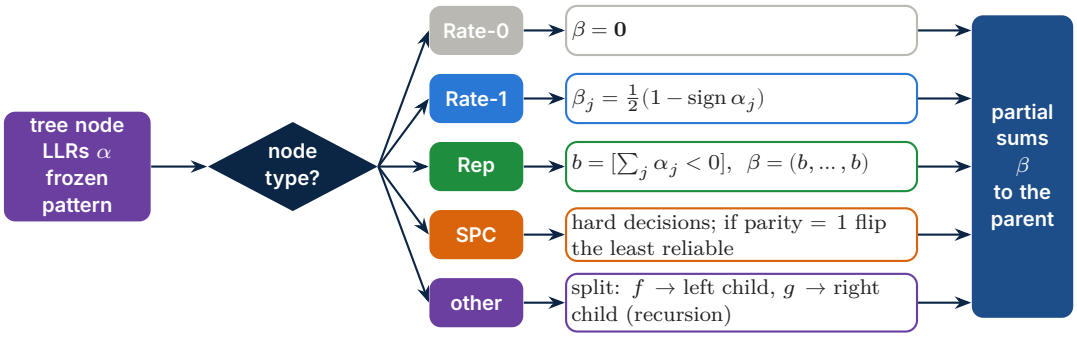


Figure 9.2. Block diagram. Fast-SSC: a node classifier sends each tree node to a one-step decoder.

Worked Example 9.2 • Fast-SSC on the $(8, 4)$ code

Frozen pattern: $u_0..u_3$ = F F F I (a **Rep** node), $u_4..u_7$ = F I I I (an **SPC** node). The whole code is two special nodes!
LLRs of Worked Example 6.3. Root f: $\alpha_L = (0.6, -2.0, -0.8, -1.2)$.
Rep node: $\sum \alpha_j = 0.6 - 2.0 - 0.8 - 1.2 = -3.4 < 0 \Rightarrow \beta_L = (1, 1, 1, 1)$ (so $\hat{u}_3 = 1$).

Root g: $\alpha_R = (0.9, -4.5, 2.6, -4.2)$.
SPC node: hard decisions $(0,1,0,1)$, XOR $= 0$, nothing to flip: $\beta_R = (0,1,0,1)$.
Output $\hat{\mathbf{x}} = [\beta_L \oplus \beta_R \,|\, \beta_R] = (1,0,1,0,0,1,0,1)$, and $\hat{\mathbf{u}} = \hat{\mathbf{x}}\mathbf{G}_8 = (0,0,0,1,0,0,1,1)$ — identical to the 16-step SC decoding of Chapter 6, in 4 steps.

9.5 Adaptive SCL decoding

Most received blocks at operating SNR are easy. Li, Shen and Tse (2012) proposed: decode with a small list (even $L = 1$); if the CRC fails, decode again with $2L$; repeat until the CRC passes or $L_{\max}$ is reached. The error rate is that of CA-SCL with $L_{\max}$, while the *average* complexity at high SNR is close to that of SC. The cost is a variable (worst-case large) latency, which matters for real-time systems.

Fig. 9.3 shows this control loop.

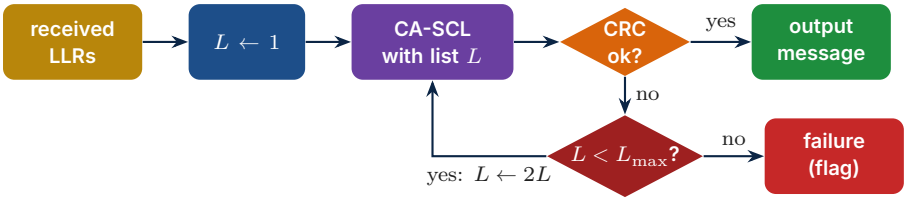


Figure 9.3. Block diagram. Adaptive SCL decoding: most blocks finish with a small list.

9.6 Polar-code construction methods

Definition 9.3 • Code construction

Construction is the choice of the information set $\mathcal{A}$: rank the N bit-channels by reliability for a given channel (or design SNR) and take the K best.

Method	Idea	Cost / accuracy
Bhattacharyya (BEC)	treat the channel as a BEC with $\epsilon = Z(W)$ and use $2Z - Z^2$, Z^2	$O(N)$; approximate
Monte-Carlo	simulate SC with a genie and count errors per bit-channel	slow; accurate at the simulated SNR
Density evolution (DE)	track the full LLR density through f and g (Mori–Tanaka 2009)	heavy convolutions; exact in principle
Tal–Vardy	quantized DE with upgraded/degraded channels; gives bounds	$O(N\mu^2 \log \mu)$; very accurate
Gaussian approximation (GA)	LLR densities assumed Gaussian; track only the mean (Trifonov 2012)	$O(N)$; very good for AWGN
Polarization weight (PW)	SNR-independent formula $W_i = \sum_j b_j \beta^j$, $\beta = 2^{1/4}$	$O(N \log N)$; nested; close to 5G
Fixed sequence (5G)	one standard ordering of 0..1023, nested for all $N \le 1024$	table look-up

9.6.1 Density evolution

If the channel LLR has density $a(\ell)$ (given that 0 is sent), the g node ("plus") simply adds two independent LLRs, so its density is the convolution $a * a$. The f node ("minus") has a more complex density (the "check-node convolution"). Applying these n times along each path gives the exact LLR density of every bit-channel, and the bit-channel error probability is $P(\ell < 0)$. In practice densities are quantized, and care is needed so that quantization errors are controlled (this is what Tal–Vardy's method does rigorously).

9.6.2 Gaussian approximation

For BPSK over AWGN, the channel LLR is Gaussian with mean m and variance $2m$ (a "consistent" Gaussian, $m = 2/\sigma^2$). GA assumes every bit-channel LLR stays consistent Gaussian and tracks only m:

$$m^+ = 2m, \qquad m^- = \phi^{-1}\Big(1 - \big(1 - \phi(m)\big)^2\Big), \qquad P_e(W_N^{(i)}) \approx Q\big(\sqrt{m_i/2}\big),$$

with $\phi(x) = 1 - \frac{1}{\sqrt{4\pi x}} \int_{-\infty}^{\infty} \tanh\frac{u}{2}\, e^{-\frac{(u-x)^2}{4x}}\, du$ for $x > 0$, $\phi(0) = 1$. A popular closed-form approximation (Chung et al.) is $\phi(x) \approx e^{-0.4527 x^{0.86} + 0.0218}$ for $0 < x < 10$ and $\phi(x) \approx \sqrt{\pi/x}\, e^{-x/4}(1 - \frac{10}{7x})$ for $x \geq 10$.

! Numerical traps in GA

The closed-form ϕ for small x is slightly larger than 1, and for large x the value $1 - (1-\phi)^2$ rounds to 0 in double precision. Both errors silently produce wrong rankings for long codes (we observed a BLER more than four times too high at $N = 1024$ before fixing them). Our implementation evaluates ϕ by numerical integration for $x < 10$, works with $\ln\phi$, and uses $1 - (1-p)^2 = 2p - p^2$. See `_log_phi` and `_ga_minus` in Chapter 11.

Worked Example 9.3 • GA for $N = 4$ at design $E_b/N_0 = 2\,\mathrm{dB}$, $R = 1/2$

$E_s/N_0 = 0.792$, so $m_0 = 4E_s/N_0 = 3.17$.
Level 1: $m^- = \phi^{-1}(1 - (1 - \phi(3.17))^2) = 1.63$, $m^+ = 6.34$.
Level 2: from 1.63: $m^{--} = 0.60$, $m^{-+} = 3.26$; from 6.34: $m^{+-} = 4.27$, $m^{++} = 12.68$.
Bit-channel error estimates $Q(\sqrt{m/2})$: 0.29, 0.10, 0.072, 0.0059 for $i = 0, 1, 2, 3$. The ranking $3 > 2 > 1 > 0$ is the same as for the BEC in Chapter 4.

Fig. 9.4 summarises the GA construction as a chain of blocks; it is exactly what `ga_construct` (Python, Chapter 11) and `ga_construct.m` (MATLAB, Appendix D) do.

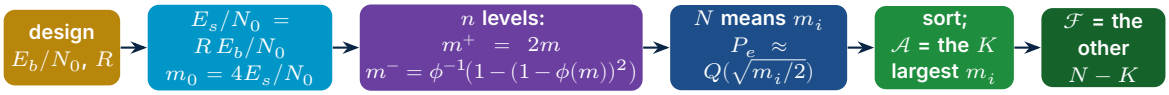


Figure 9.4. Block diagram. Gaussian-approximation construction of the information set.

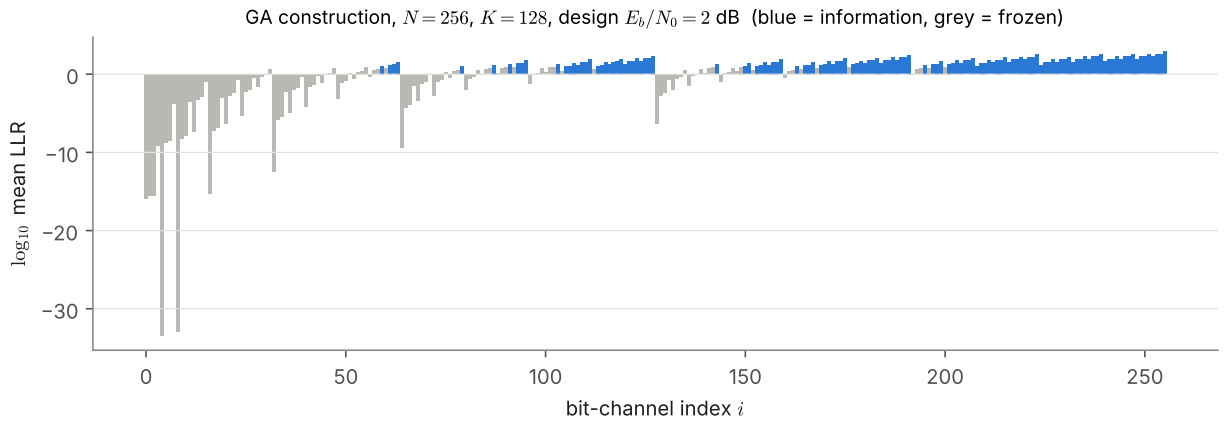


Figure 9.5. GA mean LLRs of the 256 bit-channels (log scale). The construction used in all our simulations.

9.6.3 Polarization weight

Advanced • An SNR-independent ordering

Write $i = \sum_{j=0}^{n-1} b_j 2^j$. The polarization weight (He et al., 2017) is $W_i = \sum_j b_j \, 2^{j/4}$; larger W_i means more reliable. For $N = 16$ it gives the order (least to most reliable) $0, 1, 2, 4, 8, 3, 5, 6, 9, 10, 12, 7, 11, 13, 14, 15$; the 5G sequence restricted to $i < 16$ is $0, 1, 2, 4, 8, 3, 5, 9, 6, 10, 12, 7, 11, 13, 14, 15$ — only 6 and 9 swap. For $N = 1024$, $R = 1/2$ the two information sets share 507 of 512 positions.

9.7 Puncturing, shortening and repetition

Real systems need any number E of transmitted bits, while the mother polar code has $N = 2^n$ bits.

Definition 9.4 • Rate-matching operations

- **Puncturing** ($E < N$): some coded bits are *not transmitted*. The receiver knows nothing about them: their LLR is set to 0.
- **Shortening** ($E < N$): some coded bits are forced to a value known to the receiver (zero) and not transmitted. Their LLR is set to $+\infty$ (a large number).
- **Repetition** ($E > N$): some coded bits are sent twice; the receiver *adds* their LLRs.

Worked Example 9.4 • Which u positions must be frozen?

Because $\mathbf{G}_N$ is lower triangular, x_j depends only on u_i with $i \geq j$, and $x_{N-1} = u_{N-1}$.

- **Shortening the last k bits:** freeze $u_{N-k}, \ldots, u_{N-1}$. Then $x_{N-k}, \ldots, x_{N-1}$ are always 0 — known to the receiver. (These are reliable positions, so this freezing costs something; shortening is preferred at high rate.)
- **Puncturing the first k bits:** the bit-channels $u_0, \ldots, u_{k-1}$ become useless (capacity 0); freeze them. They are unreliable anyway, so puncturing is cheap at low rate.

Example $N = 8$, $E = 6$: puncture x_0, x_1 (LLRs 0) and freeze u_0, u_1 — already frozen in our

$(8, 4)$ code.

9.8 Rate matching

The full rate-matching chain (5G version, Chapter 10) is: sub-block interleaving of the N coded bits, then bit selection into a circular buffer (take the first or last E bits, or wrap around for repetition), and finally a channel interleaver. The choice in 5G is:

$E \geq N$: repetition; $E < N$, $\frac{K}{E} \leq \frac{7}{16}$: puncturing; $E < N$, $\frac{K}{E} > \frac{7}{16}$: shortening.

9.9 Finite-length performance

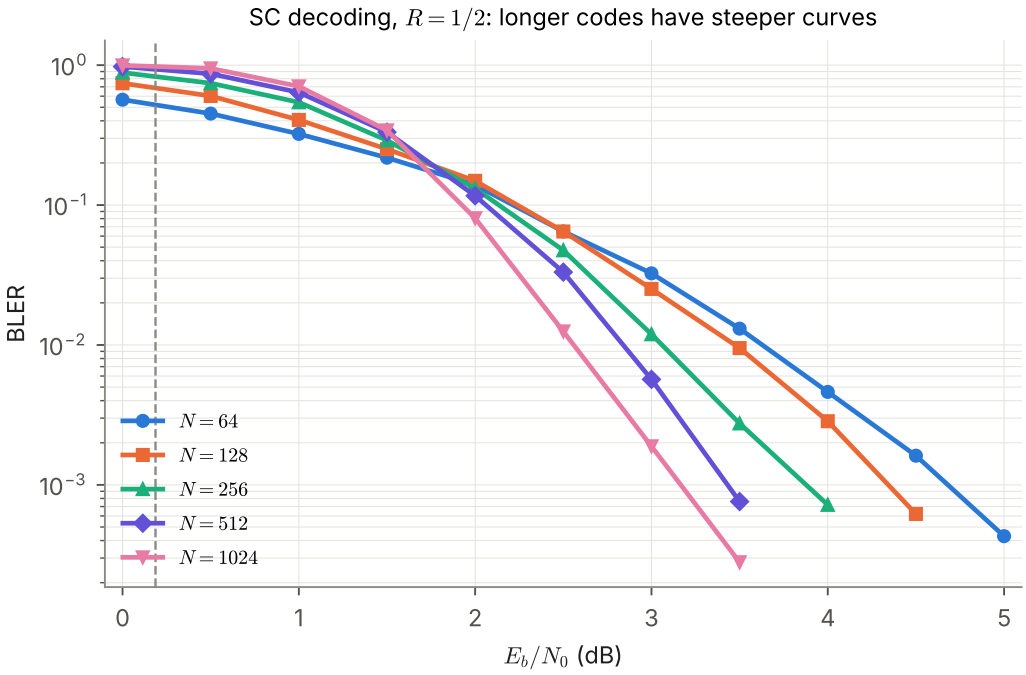


Figure 9.6. SC decoding at $R = 1/2$ for $N = 64$ to 1024 (GA at 2 dB). Points with fewer than 20 block errors are not shown.

E_b/N_0 for BLER $= 10^{-3}$, SC, $R = 1/2$	$N = 64$	128	256	512	1024
simulated	4.68 dB	4.34 dB	3.88 dB	3.43 dB	3.17 dB
gap to BPSK limit (0.19 dB)	4.5 dB	4.2 dB	3.7 dB	3.2 dB	3.0 dB

Scaling law

Each doubling of N gains roughly 0.3–0.45 dB here, and the gain shrinks slowly. Theory predicts the gap to capacity at fixed error probability to shrink like $N^{-1/\mu}$, with a *scaling*

exponent $\mu \approx 3.6$ for polar codes with SC on the BEC (Hassani, Alishahi and Urbanke), compared with $\mu = 2$ for the best possible codes. List decoding and CRC (Chapters 7–8) recover a large part of this finite-length loss — which is why 5G uses CA-SCL.

9.10 Hardware implementation considerations

- **Quantization.** LLRs of 5–6 bits (internal 6–8 bits with saturation) lose only about 0.1 dB in typical studies. Min-sum f needs only comparators and sign logic.
- **Semi-parallel SC** (Leroux et al.): P processing elements shared over the tree; memory, not logic, dominates the area.
- **Latency.** Conventional SC needs $2N - 2$ cycles; Fast-SSC style node processing reduces it many times. Unrolled, fully pipelined decoders reach very high throughput for a fixed code.
- **SCL.** L copies of LLR memory with pointer-based (lazy) copying; a sorter selects L of $2L$ metrics each information bit. Sorter delay and memory copying are the main bottlenecks.
- **Flexibility.** 5G requires many (K, E) pairs; the frozen set is computed on the fly from the reliability sequence, which favours sequence-based construction over SNR-dependent methods.

9.11 Chapter summary

Key concepts

- Systematic encoding: same codebook, message visible in $x_{\mathcal{A}}$; better BER, same BLER.
- BP and SCAN give soft outputs; BP is parallel but iterative.
- Fast-SSC decodes Rate-0, Rate-1, Rep and SPC nodes in one step with identical results to SC.
- Construction ranks bit-channels: BEC bound, DE, Tal–Vardy, GA, PW, or a fixed sequence.
- Puncturing (LLR 0) suits low rates; shortening (LLR $+\infty$) suits high rates; repetition adds LLRs.

Important formulas

GA	$m^+ = 2m$, $m^- = \phi^{-1}(1-(1-\phi(m))^2)$, $P_e \approx Q(\sqrt{m/2})$
PW	$W_i = \sum_j b_j 2^{j/4}$
Rep node	$b = \frac{1}{2}(1 - \operatorname{sign}\sum_j \alpha_j)$
5G mode	repetition if $E \geq N$; puncture if $K/E \leq 7/16$; else shorten

Visual recap • Chapter 9



Common student mistakes

! Common mistake

- Setting punctured LLRs to $+\infty$ or shortened LLRs to 0 — it is the other way round.
- Using a construction designed for one N after puncturing/shortening without re-freezing the affected positions.
- Believing Fast-SSC is a lossy approximation of SC. Its node rules are exact (or ML for SPC nodes).
- Comparing systematic and non-systematic codes by BLER and expecting a difference.

Practice problems

Problem 9.1. Classify the nodes [F,F,F,F], [I,I,I,I], [F,F,F,I], [F,I,I,I].

Solution Rate-0, Rate-1, Repetition, SPC.

Problem 9.2. An SPC node receives $\alpha = (1.2, -0.3, -2.0, 0.8)$. Decode it.

Solution Hard decisions $(0,1,1,0)$, XOR $= 0 \Rightarrow$ keep: $\beta = (0,1,1,0)$. If instead $\alpha_1 = +0.3$: decisions $(0,0,1,0)$, XOR $= 1$, flip the least reliable ($|0.3|$): $(0,1,1,0)$.

Problem 9.3. $N = 1024$, $K = 100$, $E = 600$. Which rate-matching mode does 5G use?

Solution $E < N$ and $K/E = 0.167 \leq 7/16$: puncturing.

Problem 9.4. Compute the polarization weights of $i = 6$ and $i = 9$ for $N = 16$. Which is more reliable?

Solution $6 = 0110_2$: $2^{1/4}+2^{2/4} = 1.189+1.414 = 2.603$. $9 = 1001_2$: $1+2^{3/4} = 1+1.682 = 2.682$. PW ranks 9 above 6.

Self-assessment quiz

1. Which two operations of SC are replaced by the BP processing-element equations?
2. Why does adaptive SCL have variable latency?
3. What is the mean and variance of the BI-AWGN channel LLR?
4. Why is a fixed reliability sequence attractive for standards?

Answers: 1) f and g (with soft instead of hard partial sums). 2) It may decode the same block several times. 3) $m = 2/\sigma^2$, variance $2m$. 4) No SNR estimate needed, nested for all lengths, cheap to store, identical at

both ends.

PART IV

Professional Level: 5G and Practical Implementation

How 3GPP NR really uses polar codes, a complete tested Python implementation, and engineering analysis of real simulation results.

CHAPTER

10



Polar Codes in 5G NR

IN THIS CHAPTER YOU WILL LEARN

- why 3GPP chose polar codes for control information;
- which NR physical channels use them (DCI, UCI, PBCH);
- CRC processing, interleaving and parity-check bits;
- the NR reliability sequence and frozen-set selection;
- how NR chooses N and performs rate matching;
- the complete transmitter and receiver chains;
- textbook polar coding versus real NR processing.

About this chapter

This chapter explains the *structure* of the NR polar coding chain for learning. Section numbers refer to 3GPP TS 38.212 (Release 15 onwards). Implementation work must follow the exact text and tables of the release you target; some details (for example segmentation thresholds and all table values) are summarised, not reproduced.

10.1 Why polar codes are used in 5G

During the 5G standardisation (2016) three candidate families were compared for the enhanced mobile broadband (eMBB) scenario: turbo, LDPC and polar codes. The decision was split by the job:

- **Data channels** (PDSCH, PUSCH): large blocks (up to thousands of bits), very high throughput ⇒ LDPC.
- **Control channels**: small blocks (tens to a few hundred bits), very low BLER targets, many different sizes and rates, and blind decoding by the phone ⇒ **polar** codes with CA-SCL decoding.

Polar codes are strong at these short lengths with CA-SCL, their rate is adapted simply by puncturing, shortening and repetition from one nested reliability sequence, and early termination with

distributed CRC saves power in blind decoding.

10.2 Polar coding of control information

Information	Physical channel	Link	Polar-related features
DCI	PDCCH	downlink	CRC24C with 24 ones prepended and RNTI-scrambled last 16 CRC bits; distributed CRC (input interleaver); $n_{\max} = 9$
UCI	PUCCH, PUSCH	uplink	CRC6 ($12 \le A \le 19$) plus 3 parity-check bits, or CRC11 ($A \ge 20$); channel interleaver; $n_{\max} = 10$; segmentation for large payloads
MIB + timing	PBCH	downlink broadcast	$A = 32$, CRC24C, $K = 56$, $E = 864$, $N = 512$, $n_{\max} = 9$

- **DCI** (downlink control information) tells a UE where its data is, which modulation and coding scheme, HARQ process, power control and so on.
- **UCI** (uplink control information) carries HARQ-ACK, scheduling requests and channel state information. Very small UCI ($A \le 11$ bits) does not use polar codes (repetition and Reed–Muller-type block codes are used instead).
- **PBCH** carries the master information block that a phone needs before anything else.

10.3 CRC processing

- **DCI:** the CRC24C is computed over the payload with 24 ones prepended (this avoids the all-zero ambiguity), and the last 16 CRC bits are XORed with the 16-bit RNTI (the UE identity). A UE checks the CRC with its own RNTI: if the DCI is for someone else, the CRC fails. This is how a phone finds its own control message among many candidates (**blind decoding**).
- **UCI:** CRC6 or CRC11 for small payloads (overhead matters); see Chapter 8.
- **PBCH:** CRC24C.

10.4 Interleaving

NR uses three different interleavers in the polar chain. Do not confuse them:

Interleaver	Used in	Purpose
Input bit interleaver ($I_{IL} = 1$)	DCI, PBCH	spreads CRC bits between information bits (distributed CRC) for early termination; defined for up to $K_{\max} = 164$ bits
Sub-block interleaver	all	part of rate matching: permutes 32 equal sub-blocks of the N coded bits so that puncturing/shortening remove good positions
Channel interleaver ($I_{BIL} = 1$)	UCI	triangular interleaver on the E transmitted bits; improves performance with higher-order modulation

10.4.1 Sub-block interleaver

The N coded bits are split into 32 sub-blocks of $N/32$ bits. Sub-block i is moved to position given by the pattern $P(i)$:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	1	2	4	3	5	6	7	8	16	9	17	10	18	11	19
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P(i)$	12	20	13	21	14	22	15	23	24	25	26	28	27	29	30	31

Output bit n is $y_n = d_{J(n)}$ with $J(n) = P(\lfloor 32n/N \rfloor) \cdot \frac{N}{32} + (n \bmod \frac{N}{32})$.

10.5 Reliability sequences

Definition 10.1 • NR reliability sequence

TS 38.212 Table 5.3.1.2-1 lists one sequence $\mathbf{Q}_0^{N_{\max}-1} = (Q_0, \ldots, Q_{1023})$ of all indices $0, \ldots, 1023$ in *increasing* order of reliability. For a code of length $N < 1024$ the sequence for N is obtained by keeping, in the same order, only the indices smaller than N (the sequence is **nested**).

Worked Example 10.1 • Short NR sub-sequences

Taking the indices $< N$ from the NR sequence gives (least reliable first):

- $N = 8$: $0, 1, 2, 4, 3, 5, 6, 7$ (same as the BEC(0.5) ranking of Chapter 4);
- $N = 16$: $0, 1, 2, 4, 8, 3, 5, 9, 6, 10, 12, 7, 11, 13, 14, 15$;
- $N = 32$: 0, 1, 2, 4, 8, 16, 3, 5, 9, 6, 17, 10, 18, 12, 20, 24, 7, 11, 19, 13, 14, 21, 26, 25, 22, 28, 15, 23, 27, 29, 30, 31.

For $N = 32$, $K = 16$ the NR information set is the last 16 entries: $\{7, 11, 13, 14, 15, 19, 21, 22, 23, 25, 26, 27, 28, 29, 30, 31\}$. Our GA design at 2 dB (Chapter 8) chose $\{11, 13, 14, 15, 19, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31\}$: the two differ in one position (7 versus 24).

10.5.1 Frozen-set selection

1. Take the sequence for N.
2. Remove positions that rate matching makes unusable (pre-frozen set $\mathcal{Q}_{F,\text{tmp}}$, next sections).
3. From the remaining, take the $K+n_{PC}$ most reliable as $\mathcal{A}$ (information, CRC and parity-check bits); the rest is frozen.

10.6 Rate matching

10.6.1 Choosing the mother code length N

Given K (payload including CRC) and E (number of bits the physical channel can carry):

$$n_1 = \begin{cases} \lceil \log_2 E \rceil - 1 & \text{if } E \le \frac{9}{8}\, 2^{\lceil \log_2 E \rceil - 1} \text{ and } K/E < \frac{9}{16}, \\ \lceil \log_2 E \rceil & \text{otherwise,} \end{cases} \qquad n_2 = \left\lceil \log_2 \frac{K}{R_{\min}} \right\rceil, \; R_{\min} = \frac{1}{8},$$

$$n = \max\big(\min(n_1, n_2, n_{\max}),\, n_{\min}\big), \quad n_{\min} = 5, \quad N = 2^n.$$

The first rule avoids a very long mother code when E is only slightly above a power of two (it prefers a little repetition); n_2 avoids mother codes with rate below 1/8; $n_{\max}$ limits decoder size.

Worked Example 10.2 • Three NR-style length calculations

PBCH: $K = 56$, $E = 864$, $n_{\max} = 9$. $\lceil \log_2 864 \rceil = 10$; is $864 \le \frac{9}{8} \cdot 512 = 576$? No $\Rightarrow n_1 = 10$. $n_2 = \lceil \log_2 448 \rceil = 9$. $n = \min(10, 9, 9) = 9$, $N = 512$. $E > N \Rightarrow$ **repetition** of 352 bits.
DCI: $A = 40$, $K = 64$, aggregation level 2 $\Rightarrow$ $E = 2 \times 108 = 216$. $\lceil \log_2 216 \rceil = 8$; $216 \le \frac{9}{8} \cdot 128 = 144$? No $\Rightarrow n_1 = 8$. $n_2 = \lceil \log_2 512 \rceil = 9$. $N = 256$. $E < N$ and $K/E = 0.296 \le 7/16 \Rightarrow$ **puncturing** of 40 bits.
Small E above a power of two: $K = 64$, $E = 140$. $\lceil \log_2 140 \rceil = 8$; $140 \le 144$ and $K/E = 0.457 < 0.5625 \Rightarrow n_1 = 7$, $N = 128$, **repetition** of 12 bits instead of puncturing a 256-bit code.

10.6.2 Bit selection: puncturing, shortening, repetition

After sub-block interleaving, the N bits $y_0, \ldots, y_{N-1}$ are put in a circular buffer and E bits are taken:

$$\begin{aligned} &\text{repetition } (E \ge N): && e_k = y_{k \bmod N}, \\ &\text{puncturing } (E < N,\ K/E \le \tfrac{7}{16}): && e_k = y_{k+N-E}, \\ &\text{shortening } (E < N,\ K/E > \tfrac{7}{16}): && e_k = y_k, \qquad k = 0, \ldots, E-1. \end{aligned}$$

Puncturing removes the *first* $N - E$ interleaved bits, shortening the *last* $N - E$.

Fig. 10.1 shows the rate-matching block and what each of the three modes does to the circular buffer.

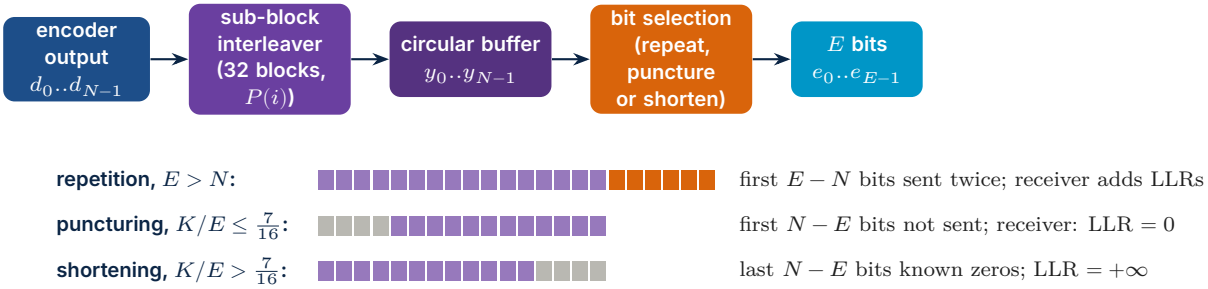


Figure 10.1. Block diagram. NR rate matching for polar codes. Purple: transmitted buffer bits; orange: repeated bits; grey: bits that are not transmitted.

10.6.3 Pre-frozen positions

- **Puncturing:** the indices $J(n)$, $n = 0, \ldots, N - E - 1$, are frozen, and in addition the first $\lceil \frac{3N}{4} - \frac{E}{2} \rceil$ indices if $E \ge \frac{3N}{4}$, otherwise the first $\lceil \frac{9N}{16} - \frac{E}{4} \rceil$ indices.

- **Shortening:** the indices $J(n)$, $n = E, \ldots, N-1$, are frozen (their coded bits are known to be zero, as explained in Chapter 9).

For the DCI example ($N = 256$, $E = 216 \geq 192$): the first $\lceil 192 - 108 \rceil = 84$ indices are pre-frozen.

10.7 Parity-check bits for small UCI

For UCI with $12 \leq A \leq 19$, the CRC6 alone is too weak for both list selection and error detection. NR adds $n_{PC} = 3$ **parity-check (PC) bits** placed on information positions. Each PC bit is computed during encoding by a cyclic shift register of length 5 that accumulates earlier information bits; the decoder checks them *during* SCL decoding (like extra frozen bits whose value depends on earlier decisions) and prunes paths that violate them. One of the three PC bits may be placed on a low-weight row of $\mathbf{G}_N$ to increase the minimum distance.

10.8 The complete encoding and decoding chain

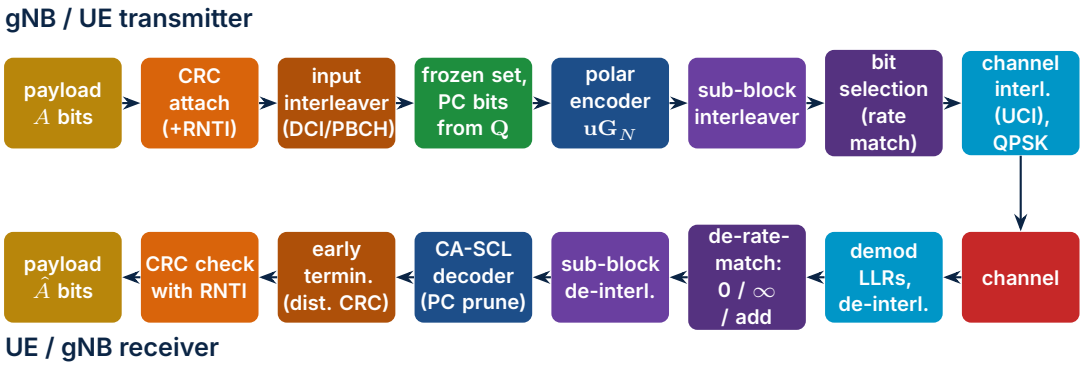


Figure 10.2. The NR polar coding chain (simplified). Orange: CRC related; green: construction; navy: polar core; purple: rate matching; cyan: modulation and channel interleaving.

Receiver de-rate-matching: punctured positions get LLR 0, shortened positions get a large positive LLR, repeated positions have their LLRs added. Then the sub-block interleaver is inverted and the CA-SCL decoder runs on the N mother-code LLRs.

10.9 Textbook polar coding versus 5G NR processing

Aspect	Textbook (Chapters 3–8)	5G NR (TS 38.212)
Length	$N = 2^n$ transmitted	any E; mother code $N \leq 512$ (DL) or 1024 (UL)
Construction	BEC / GA at a design SNR	one fixed nested sequence of length 1024
CRC	optional, appended	mandatory; distributed (DCI/PBCH), RNTI-masked (DCI)
Extra bits	none	parity-check bits for small UCI
Interleaving	none	input, sub-block and channel interleavers
Frozen set	$N - K$ least reliable	also pre-frozen positions due to puncturing/shortening
Decoder	SC / SCL / CA-SCL	implementation choice (typically CA-SCL, $L \approx 8$, with early termination)
Generator	$\mathbf{G}_N = \mathbf{F}^{\otimes n}$	the same, no bit reversal

10.10 Relevant 3GPP specifications

TS 38.212	NR; Multiplexing and channel coding. Clause 5.1 (CRC), 5.3.1 (polar coding, reliability sequence Table 5.3.1.2-1), 5.4.1 (rate matching for polar codes), 6.3.1 / 6.3.2 (UCI on PUCCH / PUSCH), 7.1 (BCH), 7.3 (DCI).
TS 38.211	NR; Physical channels and modulation (PDCCH, PUCCH, PBCH structure and modulation).
TS 38.213	NR; Physical layer procedures for control (search spaces, blind decoding, UCI reporting).

Blind decoding and early termination

A UE does not know where its DCI is or what aggregation level was used. It tries many candidates in each slot (the exact limits are given in TS 38.213). Every candidate is a full polar decoding! The distributed CRC lets the decoder stop a candidate as soon as an embedded CRC bit fails, which is one of the reasons the NR polar design is power-efficient.

10.11 Chapter summary

Key concepts

- NR uses polar codes for DCI, UCI ($A \geq 12$) and PBCH; LDPC for data.
- One nested reliability sequence of length 1024 gives the frozen set for every N.
- N is chosen from E, K, $R_{\min} = 1/8$ and $n_{\max}$; rate matching uses repetition, puncturing ($K/E \leq 7/16$) or shortening.
- CRC24C with RNTI masking enables blind decoding; distributed CRC enables early termination; PC bits help small UCI.

Important formulas

Sub-block interleaver	$J(n) = P(\lfloor 32n/N \rfloor)\frac{N}{32} + (n \bmod \frac{N}{32})$
Mother length	$n = \max(\min(n_1, n_2, n_{\max}), 5)$
Bit selection	$e_k = y_{k \bmod N},\ y_{k+N-E},\ y_k$
PDCCH size	$E = 108\times$ aggregation level

Visual recap • Chapter 10



Common student mistakes

! Common mistake

- Using Arıkan's bit-reversed generator with the NR sequence. NR uses $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ in natural order.
- Choosing the frozen set from the sequence without first removing the pre-frozen (punctured/shortened) positions.
- Forgetting the 24 prepended ones or the RNTI masking when testing DCI CRCs.
- Mixing up the three interleavers.

Practice problems

Problem 10.1. A DCI uses aggregation level 4. How many coded bits E does it get? For $K = 64$, find N and the rate-matching mode.

Solution $E = 432$. $\lceil \log_2 432 \rceil = 9$; $432 \le \frac{9}{8} \cdot 256 = 288$? No $\Rightarrow n_1 = 9$; $n_2 = 9$; $n_{\max} = 9 \Rightarrow N = 512$. $E < N$, $K/E = 0.148 \le 7/16$: puncturing 80 bits.

Problem 10.2. Compute $J(n)$ for $N = 64$ and $n = 18$.

Solution $N/32 = 2$, $\lfloor 32 \cdot 18/64 \rfloor = 9$, $P(9) = 16$; $J(18) = 16 \cdot 2 + (18 \bmod 2) = 32$.

Problem 10.3. Using the $N = 16$ NR sub-sequence, find the information set for $K = 6$.

Solution The last six entries: $\{7, 11, 13, 14, 15, 12\}$, i.e. $\{7, 11, 12, 13, 14, 15\}$.

Self-assessment quiz

1. Which control payloads in NR are *not* polar coded?
2. Why does the DCI CRC contain the RNTI?
3. What LLR does the receiver use for a punctured bit?

4. What is $n_{\max}$ in the downlink?

Answers: 1) Very small UCI ($A \leq 11$). 2) So that only the intended UE passes the CRC (blind detection). 3) 0. 4) 9 ($N \leq 512$).

CHAPTER

11



Python Implementation and Simulation

IN THIS CHAPTER YOU WILL LEARN

- a complete polar-code library in about 350 lines;
- encoder, frozen-bit selection (BEC and GA);
- BPSK modulation and the AWGN channel;
- one decoder class for SC, SCL and CA-SCL;
- CRC attachment and checking;
- BER/BLER simulation and plotting;
- how we tested the code for correctness.

11.1 Organisation of the software

Everything is in one file, `polar.py`, which needs only NumPy (and SciPy/Matplotlib for plots). It is printed here in full, section by section, with the real line numbers. Three small scripts use it:

`polar.py`	the library: encoder, construction, channel, decoders, CRC, simulation
`demo_ch11.py`	small demonstrations with printed outputs (this chapter)
`run_sims.py`	the Monte-Carlo runs behind every BER/BLER figure of the book
`plot_sims.py`	turns the saved results into figures

Fig. 11.1 shows how the files work together. The same structure exists in MATLAB: every Python function has a MATLAB file with the same name and the same arguments (Appendix D), and the two libraries give identical decisions on the same LLRs.

Conventions repeated

$\mathbf{G}_N = \mathbf{F}^{\otimes n}$ without bit reversal; $\mathbf{x} = \mathbf{u}\mathbf{G}_N$; bit $0 \mapsto +1$; $L = \ln \frac{P(0)}{P(1)}$; indices in natural order. Arrays of decisions are `uint8`; LLRs are `float64`.

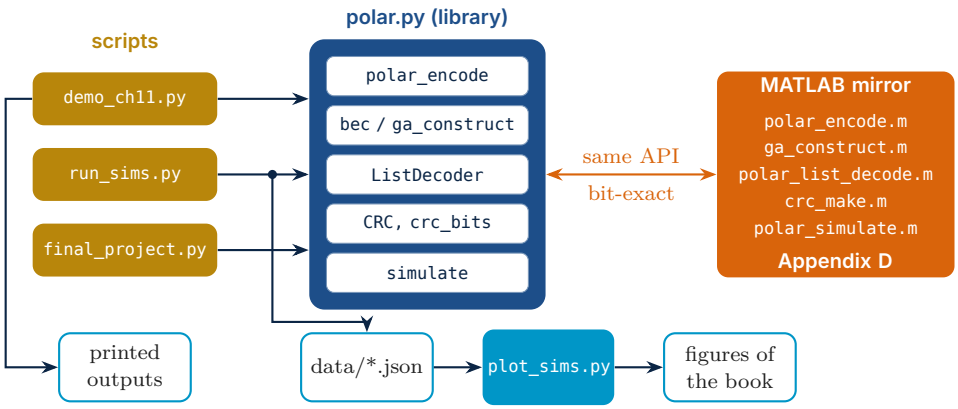


Figure 11.1. Block diagram. Structure of the book's software: one library, three scripts, the plotting chain and the equivalent MATLAB library.

11.2 Module header

```
"""
polar.py  --  A small, readable polar-code library for the textbook
"Polar Codes: From Zero to Expert".

Conventions used everywhere in this book
----------------------------------------
* Kernel  F = [[1, 0],
               [1, 1]]
* Generator matrix  G_N = F^{(x)n}  (n-fold Kronecker power, NO bit reversal).
  This is the same convention used by 3GPP NR (TS 38.212).
* Encoding is a row-vector product over GF(2):   x = u G_N.
* Bit indices run from 0 to N-1 in natural order.
* BPSK mapping:  bit 0 -> +1,  bit 1 -> -1.
* LLR definition:  L = ln( P(bit = 0 | y) / P(bit = 1 | y) ).
  A positive LLR therefore favours bit 0.
"""
import numpy as np
```

11.3 Program 1 — Polar encoder

```
# ---------------------------------------------------------------------------
# 1. Encoder
# ---------------------------------------------------------------------------
def generator_matrix(N):
    """Return G_N = F^{(x)n} as a 0/1 integer matrix."""
    F = np.array([[1, 0], [1, 1]], dtype=np.uint8)
    G = np.array([[1]], dtype=np.uint8)
    while G.shape[0] < N:
        G = np.kron(F, G) % 2          # F (x) G  -> [[G,0],[G,G]]
    return G

```

```python

def polar_encode(u):
    """
    Butterfly (fast) encoder.  u may be a 1-D vector of length N or a 2-D
    array of shape (B, N) holding B blocks.  Returns x = u G_N (mod 2).

    Each of the log2(N) stages XORs the "upper" half of every butterfly
    into the "lower" half, exactly as in the butterfly diagram.
    """
    x = np.array(u, dtype=np.uint8, copy=True)
    one_d = (x.ndim == 1)
    if one_d:
        x = x[None, :]
    B, N = x.shape
    half = N // 2
    while half >= 1:                        # stages: span N/2, N/4, ..., 1
        x = x.reshape(B, -1, 2, half)       # groups of 2*half bits
        x[:, :, 0, :] ^= x[:, :, 1, :]      # upper part ^= lower part
        x = x.reshape(B, N)
        half //= 2
    return x[0] if one_d else x
```

How it works. `generator_matrix` builds $\mathbf{F}^{\otimes n}$ with `np.kron` and is used only for testing. `polar_encode` is the butterfly of Chapter 5: in each stage it views the bits as blocks of size $2h$ and XORs the second half of every block into the first half. Thanks to the `reshape` trick, one line processes a whole batch of B blocks at once — important for fast simulations.

11.4 Program 2 — Frozen-bit selection

```python
# ---------------------------------------------------------------------------
# 2. Code construction (reliability of the bit-channels)
# ---------------------------------------------------------------------------
def bec_bhattacharyya(N, eps):
    """
    Exact Bhattacharyya parameters (= erasure probabilities) of the N
    synthetic bit-channels of a BEC(eps).
        Z(W-) = 2Z - Z^2      (worse  channel)
        Z(W+) = Z^2           (better channel)
    Natural index order, consistent with polar_encode().
    """
    if N == 1:
        return np.array([eps], dtype=float)
    zm = 2 * eps - eps ** 2
    zp = eps ** 2
    return np.concatenate([bec_bhattacharyya(N // 2, zm),
                           bec_bhattacharyya(N // 2, zp)])


# Table of log(phi(x)) for 0 < x < 10, computed by Gauss-Hermite quadrature of
#     phi(x) = 1 - E[ tanh(U/2) ],   U ~ N(x, 2x)
# (this is the exact definition used in Gaussian approximation).
```

```python
_GH_T, _GH_W = np.polynomial.hermite.hermgauss(120)
_XGRID = np.logspace(-8, 1, 4000)
_LPHI_TAB = np.log(np.clip(1 - np.array(
    [np.sum(_GH_W * np.tanh((xx + 2 * np.sqrt(xx) * _GH_T) / 2)) / np.sqrt(np.pi)
     for xx in _XGRID]), 1e-300, None))


def _log_phi(x):
    """
    Natural log of the GA function phi(x).
      * 0 < x < 10 : exact value (table built by numerical integration)
      * x >= 10    : Chung's asymptotic form
                     phi(x) ~ sqrt(pi/x) exp(-x/4) (1 - 10/(7x))
    Working with log(phi) avoids underflow for very reliable channels.
    """
    if x <= _XGRID[0]:
        return np.log1p(-x / 2)            # phi(x) ~ 1 - x/2 near zero
    if x < 10:
        return float(np.interp(np.log(x), np.log(_XGRID), _LPHI_TAB))
    return 0.5 * np.log(np.pi / x) - x / 4 + np.log(1 - 10 / (7 * x))


def _ga_minus(m):
    """Mean LLR of the 'minus' (check-node) channel: phi^-1(1-(1-phi(m))^2)."""
    lp = _log_phi(m)
    p = np.exp(lp)
    target = lp + np.log(2 - p)          # log(2p - p^2) = log(1-(1-p)^2)
    lo, hi = 0.0, m                      # the minus channel is never better
    for _ in range(80):                  # bisection: log_phi is decreasing
        mid = 0.5 * (lo + hi)
        if _log_phi(mid) > target:
            lo = mid
        else:
            hi = mid
    return 0.5 * (lo + hi)


def ga_llr_means_recursive(N, design_snr_db, rate):
    """
    Gaussian Approximation (GA) construction for BPSK over AWGN.
    Returns the mean LLR of every bit-channel (larger = more reliable),
    in natural index order (consistent with polar_encode()).
        minus child:  m- = phi^-1( 1 - (1 - phi(m))^2 )
        plus  child:  m+ = 2 m
    """
    esn0 = 10 ** (design_snr_db / 10) * rate
    m0 = 4 * esn0                        # channel LLR mean = 2/sigma^2
    def rec(n, m):
        if n == 1:
            return np.array([m])
        return np.concatenate([rec(n // 2, _ga_minus(m)), rec(n // 2, 2 * m)])
    return rec(N, m0)
```

```python
def ga_construct(N, K, design_snr_db=2.0, rate=None):
    """Return a boolean mask: True = information position (K most reliable)."""
    if rate is None:
        rate = K / N
    m = ga_llr_means_recursive(N, design_snr_db, rate)
    info = np.zeros(N, dtype=bool)
    info[np.argsort(m)[-K:]] = True
    return info


def bec_construct(N, K, eps=0.5):
    z = bec_bhattacharyya(N, eps)
    info = np.zeros(N, dtype=bool)
    info[np.argsort(z)[:K]] = True
    return info
```

How it works. `bec_bhattacharyya` is the exact BEC recursion. The GA functions implement Chapter 9: `_log_phi` returns $\ln \phi(x)$ (numerically integrated table for $x < 10$, asymptotic formula above), `_ga_minus` inverts ϕ by bisection in the log domain, and `ga_llr_means_recursive` follows exactly the same minus-first/plus-second recursion as the encoder, so index i here is index i there. `ga_construct` returns a Boolean mask of the K most reliable positions.

11.5 Programs 6 and 7 — BPSK modulation and the AWGN channel

```python
# ---------------------------------------------------------------------------
# 3. Channel: BPSK + AWGN
# ---------------------------------------------------------------------------
def bpsk(x):
    return 1.0 - 2.0 * x                        # 0 -> +1, 1 -> -1


def awgn(s, ebn0_db, rate, rng):
    """Add white Gaussian noise. Es = 1, Es/N0 = R * Eb/N0."""
    esn0 = 10 ** (ebn0_db / 10) * rate
    sigma = np.sqrt(1 / (2 * esn0))
    return s + sigma * rng.standard_normal(s.shape), sigma


def channel_llr(y, sigma):
    return 2 * y / sigma ** 2
```

How it works. With $E_s = 1$ and code rate R, $E_s/N_0 = R \cdot E_b/N_0$ and $\sigma = \sqrt{1/(2E_s/N_0)}$ (Chapter 2). `channel_llr` returns $2y/\sigma^2$ (Chapter 6). The random generator `rng` is passed in, so results are reproducible for a fixed seed.

11.6 The f and g functions

```
# ---------------------------------------------------------------------------
# 4. f and g functions
# ---------------------------------------------------------------------------
def f_minsum(a, b):
    """f(a,b) ~= sign(a) sign(b) min(|a|,|b|)   (min-sum approximation)."""
    return np.sign(a) * np.sign(b) * np.minimum(np.abs(a), np.abs(b))


def f_exact(a, b):
    """f(a,b) = 2 atanh( tanh(a/2) tanh(b/2) )   (exact box-plus)."""
    t = np.tanh(a / 2) * np.tanh(b / 2)
    t = np.clip(t, -1 + 1e-15, 1 - 1e-15)
    return 2 * np.arctanh(t)


def g_func(a, b, c):
    """g(a,b,c) = b + (1-2c) a ,  c = partial-sum bit already decided."""
    return b + (1 - 2 * c.astype(float)) * a
```

11.7 Programs 3, 4 and 5 — SC, SCL and CA-SCL decoders

```
# ---------------------------------------------------------------------------
# 5. SC / SCL / CA-SCL decoder (one recursive routine, vectorised)
# ---------------------------------------------------------------------------
class ListDecoder:
    """
    LLR-based successive-cancellation LIST decoder.
      * L = 1  gives the plain SC decoder.
      * crc  (optional) gives the CRC-aided SCL decoder.
    The decoder processes B received blocks in parallel (numpy arrays
    of shape (B, L, ...)) so that simulations run fast in pure Python.

    Path metric (min-sum form, Balatsoukas-Stimming et al. 2015):
        PM <- PM + |L|   if the bit decision disagrees with sign(L)
        PM <- PM         otherwise.     Smaller PM = more likely path.
    """

    def __init__(self, info_mask, L=1, crc=None, exact_f=False):
        self.info = np.asarray(info_mask, dtype=bool)
        self.N = self.info.size
        self.L = L
        self.crc = crc
        self.f = f_exact if exact_f else f_minsum

    # -- leaf: one bit decision for every path of every block --------------
    def _leaf(self, llr, idx):
        B, L = llr.shape
```

```
        pen0 = np.where(llr < 0, -llr, 0.0)       # cost of deciding 0
        pen1 = np.where(llr > 0, llr, 0.0)        # cost of deciding 1
        if not self.info[idx]:                    # frozen bit: always 0
            self.PM = self.PM + pen0
            u = np.zeros((B, L, 1), dtype=np.uint8)
            return u, u.copy(), np.tile(np.arange(L), (B, 1))
        # information bit: every path forks into two candidates
        cand = np.concatenate([self.PM + pen0, self.PM + pen1], axis=1)  # (B,2L)
        order = np.argsort(cand, axis=1, kind="stable")[:, :L]           # keep L best
        self.PM = np.take_along_axis(cand, order, axis=1)
        parent = order % L
        bit = (order // L).astype(np.uint8)[:, :, None]
        return bit, bit.copy(), parent

    # -- recursive node ------------------------------------------------------
    def _node(self, llr, start):
        """llr: (B, L, n).  Returns (codeword c, info vector u, parent map)."""
        n = llr.shape[2]
        if n == 1:
            return self._leaf(llr[:, :, 0], start)
        h = n // 2
        a, b = llr[:, :, :h], llr[:, :, h:]
        # left child: LLRs from f
        c1, u1, p1 = self._node(self.f(a, b), start)
        # right child: LLRs from g, using the surviving parents' LLRs
        a1 = np.take_along_axis(a, p1[:, :, None], axis=1)
        b1 = np.take_along_axis(b, p1[:, :, None], axis=1)
        c2, u2, p2 = self._node(g_func(a1, b1, c1), start + h)
        # re-order left results according to the right child's survivors
        c1 = np.take_along_axis(c1, p2[:, :, None], axis=1)
        u1 = np.take_along_axis(u1, p2[:, :, None], axis=1)
        parent = np.take_along_axis(p1, p2, axis=1)
        return np.concatenate([c1 ^ c2, c2], axis=2), \
               np.concatenate([u1, u2], axis=2), parent

    def decode(self, llr):
        """
        llr : (B, N) channel LLRs.
        Returns u_hat (B, N) chosen by: best PM (SC/SCL) or
        best PM among CRC-passing paths (CA-SCL).
        """
        llr = np.atleast_2d(llr)
        B = llr.shape[0]
        self.PM = np.full((B, self.L), np.inf)
        self.PM[:, 0] = 0.0                       # only one active path at start
        lam = np.repeat(llr[:, None, :], self.L, axis=1)
        _, u, _ = self._node(lam, 0)
        self.last_u_list, self.last_pm = u, self.PM
        # paths are already sorted by PM (best first) after the last split
        if self.crc is None:
            return u[:, 0, :]
        payload = u[:, :, self.info]              # (B, L, K)
        ok = self.crc.check(payload) & np.isfinite(self.PM)
        pick = np.where(ok.any(axis=1), ok.argmax(axis=1), 0)
```

```
264         return u[np.arange(B), pick, :]
```

11.7.1 How the decoder works

- **Shapes.** LLR arrays have shape (B, L, n): B received blocks, L paths, n = node size. All paths of all blocks are processed together with NumPy operations.
- **Recursion.** `_node` is the tree of Chapter 6: f to the left child, g to the right child, partial sums $[\beta_L \oplus \beta_R \,|\, \beta_R]$ returned up.
- **Paths that move.** When a leaf prunes and duplicates paths, path l after the leaf may come from path `parent[l]` before it. Every node therefore returns a `parent` map. The parent node uses it to (i) pick the right LLRs for the g step and (ii) re-order the left child's partial sums after the right child has finished. This replaces explicit memory copying and keeps the code short.
- **Leaves.** Frozen leaf: add the penalty for deciding 0. Information leaf: build $2L$ candidates, sort with `argsort`, keep the L best (Chapter 7). Initially only path 0 is active; the others have metric $+\infty$ and are never selected while finite candidates exist.
- **CRC selection.** After the last leaf the paths are sorted by metric; with a CRC the first path whose payload passes the check is returned (Chapter 8).
- **SC** is simply `ListDecoder(info, L=1)`.

11.8 CRC

```
267 # ---------------------------------------------------------------------------
268 # 6. CRC (as a linear map over GF(2))
269 # ---------------------------------------------------------------------------
270 CRC_POLYS = {
271     # degree-list form (exponents with coefficient 1), from 3GPP TS 38.212 5.1
272     "CRC6":   [6, 5, 0],
273     "CRC11":  [11, 10, 9, 5, 0],
274     "CRC24C": [24, 23, 21, 20, 17, 15, 13, 12, 8, 4, 2, 1, 0],
275 }
276
277
278 def crc_bits(msg, exps):
279     """Plain long-division CRC (register starts at zero). msg: 1-D 0/1."""
280     r = exps[0]
281     g = np.zeros(r + 1, dtype=np.uint8)
282     for e in exps:
283         g[r - e] = 1                        # g[0] is the x^r coefficient
284     reg = np.concatenate([np.array(msg, dtype=np.uint8), np.zeros(r, np.uint8)])
285     for i in range(len(msg)):
286         if reg[i]:
287             reg[i:i + r + 1] ^= g
288     return reg[-r:]
289
290
```

```python
class CRC:
    """CRC that appends r parity bits: [message | crc]."""

    def __init__(self, name, k_msg):
        self.exps = CRC_POLYS[name]
        self.r = self.exps[0]
        self.k = k_msg
        # Because the CRC is linear, crc(m) = m @ P  (mod 2). Build P once.
        P = np.zeros((k_msg, self.r), dtype=np.uint8)
        for i in range(k_msg):
            e = np.zeros(k_msg, dtype=np.uint8)
            e[i] = 1
            P[i] = crc_bits(e, self.exps)
        self.P = P

    def attach(self, m):
        m = np.atleast_2d(m).astype(np.uint8)
        return np.concatenate([m, (m.astype(int) @ self.P) % 2], axis=1).astype(np.uint8)

    def check(self, word):
        """word (..., k+r) -> True where the CRC is satisfied."""
        m, c = word[..., :self.k].astype(int), word[..., self.k:]
        return np.all((m @ self.P) % 2 == c, axis=-1)
```

How it works. `crc_bits` is the long division of Chapter 8. Because the CRC is linear, the class precomputes the matrix $\mathbf{P}$ whose row i is the CRC of the i-th unit vector; afterwards $\mathbf{p} = \mathbf{mP} \bmod 2$ for any batch of messages, which is very fast.

11.9 Program 8 — BER and BLER calculation

```python
# ---------------------------------------------------------------------------
# 7. Monte-Carlo simulation helper
# ---------------------------------------------------------------------------
def simulate(N, K, ebn0_list, L=1, crc_name=None, design_snr_db=None,
             max_frames=20000, min_errors=200, batch=500, seed=1):
    """
    Returns dict with BER and BLER (on the MESSAGE bits) for each Eb/N0.
    K counts all bits placed on information positions (message + CRC).
    The rate used for Eb/N0 is the message rate (K - r)/N.
    """
    rng = np.random.default_rng(seed)
    r = 0 if crc_name is None else CRC_POLYS[crc_name][0]
    k_msg = K - r
    rate = k_msg / N
    dsnr = design_snr_db if design_snr_db is not None else 2.0
    info = ga_construct(N, K, dsnr, rate=K / N)
    crc = CRC(crc_name, k_msg) if crc_name else None
    dec = ListDecoder(info, L=L, crc=crc)
    ber, bler = [], []
    for eb in ebn0_list:
        bit_err = blk_err = frames = 0
```

```
        while frames < max_frames and blk_err < min_errors:
            m = rng.integers(0, 2, size=(batch, k_msg), dtype=np.uint8)
            payload = crc.attach(m) if crc else m
            u = np.zeros((batch, N), dtype=np.uint8)
            u[:, info] = payload
            x = polar_encode(u)
            y, sigma = awgn(bpsk(x), eb, rate, rng)
            u_hat = dec.decode(channel_llr(y, sigma))
            m_hat = u_hat[:, info][:, :k_msg]
            e = (m_hat != m)
            bit_err += e.sum()
            blk_err += e.any(axis=1).sum()
            frames += batch
        ber.append(bit_err / (frames * k_msg))
        bler.append(blk_err / frames)
    return {"ebn0": list(ebn0_list), "ber": ber, "bler": bler}
```

How it works. For each E_b/N_0 the function generates random messages, attaches the CRC, places the payload on $\mathcal{A}$, encodes, modulates, adds noise, decodes and counts errors on the *message* bits only. It stops when `min_errors` block errors are seen or `max_frames` blocks are simulated. Note the rate used for the noise: the message rate $(K - r)/N$.

Fig. 11.2 shows the simulation loop as a flow chart. Each pass processes a batch of B blocks at once.

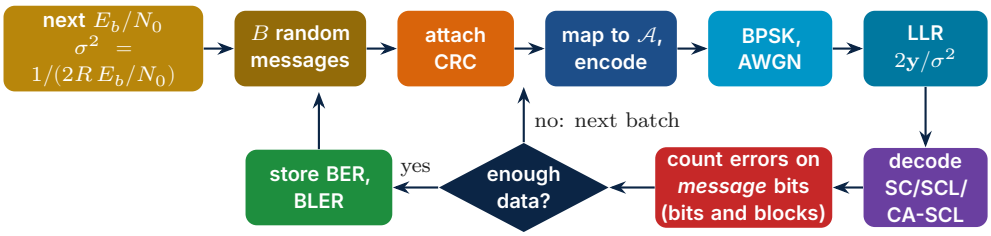


Figure 11.2. Block diagram. The Monte-Carlo loop of `simulate` (Python) and `polar_simulate` (MATLAB). "Enough data" means at least $e_{\min}$ block errors or $F_{\max}$ simulated blocks. R is the message rate, CRC bits excluded.

! Statistical reliability

A BLER estimated from e error events has a relative standard deviation of roughly $1/\sqrt{e}$. With 150 errors this is about 8%; with 10 errors about 30%. In our figures points with fewer than 20 block errors are not plotted.

11.10 Demonstration with outputs

```
"""Demonstrations used in Chapter 11. Run:  python demo_ch11.py"""
import numpy as np
from polar import *

print("1) Encoder ----------------------------------------------")
info = np.array([0, 0, 0, 1, 0, 1, 1, 1], dtype=bool)        # A = {3,5,6,7}
u = np.zeros(8, dtype=np.uint8); u[info] = [1, 0, 1, 1]
```

```
x = polar_encode(u)
print("u =", u, " x =", x)
print("matrix check:", np.array_equal(x, (u.astype(int) @ generator_matrix(8)) % 2))

print("2) Frozen-bit selection ---------------------------------")
print("BEC(0.5) Z, N=8 :", np.round(bec_bhattacharyya(8, 0.5), 4))
print("BEC info set K=4:", np.where(bec_construct(8, 4, 0.5))[0])
print("GA  info set N=16, K=8 @2dB:", np.where(ga_construct(16, 8, 2.0))[0])

print("3) SC decoder -------------------------------------------")
sc = ListDecoder(info, L=1)
llr1 = np.array([-1.5, 2.0, -0.8, 1.2, -0.6, -2.5, 1.8, -3.0])
print("SC  u_hat =", sc.decode(llr1)[0])

print("4) SCL decoder ------------------------------------------")
llr2 = np.array([-2.0, 0.3, -0.4, -2.0, 1.0, -5.8, 3.2, -4.8])
print("SC  u_hat =", sc.decode(llr2)[0], "(wrong)")
scl = ListDecoder(info, L=2)
print("SCL u_hat =", scl.decode(llr2)[0], " path metrics:", scl.last_pm[0])

print("5) CRC-aided SCL ----------------------------------------")
crc = CRC("CRC6", 10)
m = np.array([1, 0, 1, 1, 0, 0, 1, 0, 1, 1], dtype=np.uint8)
print("message + CRC6 :", crc.attach(m)[0])

print("6-7) BPSK + AWGN ----------------------------------------")
rng = np.random.default_rng(0)
y, sigma = awgn(bpsk(x), ebn0_db=2.0, rate=0.5, rng=rng)
print("sigma = %.4f" % sigma)
print("y     =", np.round(y, 2))
print("LLR   =", np.round(channel_llr(y, sigma), 2))

print("8) BER / BLER -------------------------------------------")
r = simulate(N=128, K=64, ebn0_list=[1.0, 2.0, 3.0], L=1, max_frames=20000,
             min_errors=100, batch=1000, seed=7)
for e, b, f in zip(r["ebn0"], r["ber"], r["bler"]):
    print(f"Eb/N0 = {e:.1f} dB   BER = {b:.2e}   BLER = {f:.2e}")
```

```
OUTPUT
1) Encoder ----------------------------------------------
u = [0 0 0 1 0 0 1 1]  x = [1 0 1 0 0 1 0 1]
matrix check: True
2) Frozen-bit selection ---------------------------------
BEC(0.5) Z, N=8 : [0.9961 0.8789 0.8086 0.3164 0.6836 0.1914 0.1211 0.0039]
BEC info set K=4: [3 5 6 7]
GA  info set N=16, K=8 @2dB: [ 7  9 10 11 12 13 14 15]
3) SC decoder -------------------------------------------
SC  u_hat = [0 0 0 1 0 0 1 1]
4) SCL decoder ------------------------------------------
SC  u_hat = [0 0 0 0 0 0 1 1] (wrong)
SCL u_hat = [0 0 0 1 0 0 1 1]  path metrics: [2.  2.7]
5) CRC-aided SCL ----------------------------------------
message + CRC6 : [1 0 1 1 0 0 1 0 1 1 0 1 1 0 1 1]
6-7) BPSK + AWGN ----------------------------------------
```

```
sigma = 0.7943
y     = [-0.9   0.9  -0.49  1.08  0.57 -0.71  2.04 -0.25]
LLR   = [-2.85  2.84 -1.56  3.43  1.82 -2.26  6.45 -0.79]
8) BER / BLER --------------------------------------------
Eb/N0 = 1.0 dB   BER = 1.20e-01   BLER = 3.95e-01
Eb/N0 = 2.0 dB   BER = 4.07e-02   BLER = 1.52e-01
Eb/N0 = 3.0 dB   BER = 6.99e-03   BLER = 3.10e-02
```

All numbers agree with the hand calculations of Chapters 4–8: the same codeword, the same SC error, the same SCL path metrics 2.0 and 2.7, and the same CRC6 bits 011011. In part 6–7 the noise flipped no sign (all LLR signs match $\mathbf{x} = 10100101$), but $y_7 = -0.25$ is close to the threshold.

11.11 Program 9 — BER vs E_b/N_0 graphs

```python
import json, numpy as np, matplotlib.pyplot as plt
from scipy.special import erfc

res = json.load(open("sim_A.json"))                  # written by run_sims.py
Q = lambda x: 0.5 * erfc(x / np.sqrt(2))
fig, ax = plt.subplots(figsize=(7, 4.2))
for name, label in [("SC", "SC"), ("SCL8", "SCL, L=8"), ("CASCL8", "CA-SCL, L=8")]:
    r = res[name]
    ax.semilogy(r["ebn0"], r["ber"], marker="o", label=label)
eb = np.linspace(0, 4, 50)
ax.semilogy(eb, Q(np.sqrt(2 * 10 ** (eb / 10))), ":", label="uncoded BPSK")
ax.set_xlabel("Eb/N0 (dB)"); ax.set_ylabel("BER"); ax.grid(True, which="both")
ax.legend(); plt.savefig("ber.pdf")
```

11.12 Program 10 — Comparing block lengths and code rates

The script below produced all three simulation data sets of the book: decoders (Fig. 7.3), block lengths (Fig. 9.6) and code rates (Fig. 11.3). On a laptop the three groups take a few minutes each.

```python
"""Monte-Carlo simulations used for every BER/BLER figure in the book."""
import json, sys, time, numpy as np
from polar import simulate

def run(tag, **kw):
    t = time.time()
    r = simulate(**kw)
    r["ber"] = [float(v) for v in r["ber"]]; r["bler"] = [float(v) for v in r["bler"]]
    r.update({k: v for k, v in kw.items() if k != "ebn0_list"})
    print(tag, f"{time.time()-t:.0f}s", r["bler"], flush=True)
    return r

group = sys.argv[1]
E = lambda a, b: list(np.round(np.arange(a, b + 1e-9, 0.5), 2))
```

```
out = {}
common = dict(max_frames=100000, min_errors=150, batch=2000, seed=2026)
if group == "A":       # decoders, N = 256, R = 1/2
    out["SC"]      = run("SC",  N=256, K=128, ebn0_list=E(0.5, 4.0), L=1, **common)
    out["SCL2"]    = run("L2",  N=256, K=128, ebn0_list=E(0.5, 3.5), L=2, **common)
    out["SCL4"]    = run("L4",  N=256, K=128, ebn0_list=E(0.5, 3.5), L=4, **common)
    out["SCL8"]    = run("L8",  N=256, K=128, ebn0_list=E(0.5, 3.5), L=8, **common)
    out["CASCL8"]  = run("CA8", N=256, K=139, ebn0_list=E(0.5, 3.0), L=8, crc_name="CRC11",
     **common)
    out["CASCL16"] = run("CA16",N=256, K=139, ebn0_list=E(0.5, 2.5), L=16, crc_name="CRC11"
    , **common)
elif group == "B":     # block lengths, SC, R = 1/2
    for N in [64, 128, 256, 512, 1024]:
        out[f"N{N}"] = run(f"N{N}", N=N, K=N // 2, ebn0_list=E(0.0, 5.0 if N < 256 else
    4.0), L=1, **common)
elif group == "C":     # code rates, SC, N = 256
    for K in [64, 128, 192]:
        out[f"K{K}"] = run(f"K{K}", N=256, K=K, ebn0_list=E(0.0, 5.5), L=1, **common)
json.dump(out, open(f"../data/sim_{group}.json", "w"), indent=1)
print("done", group)
```

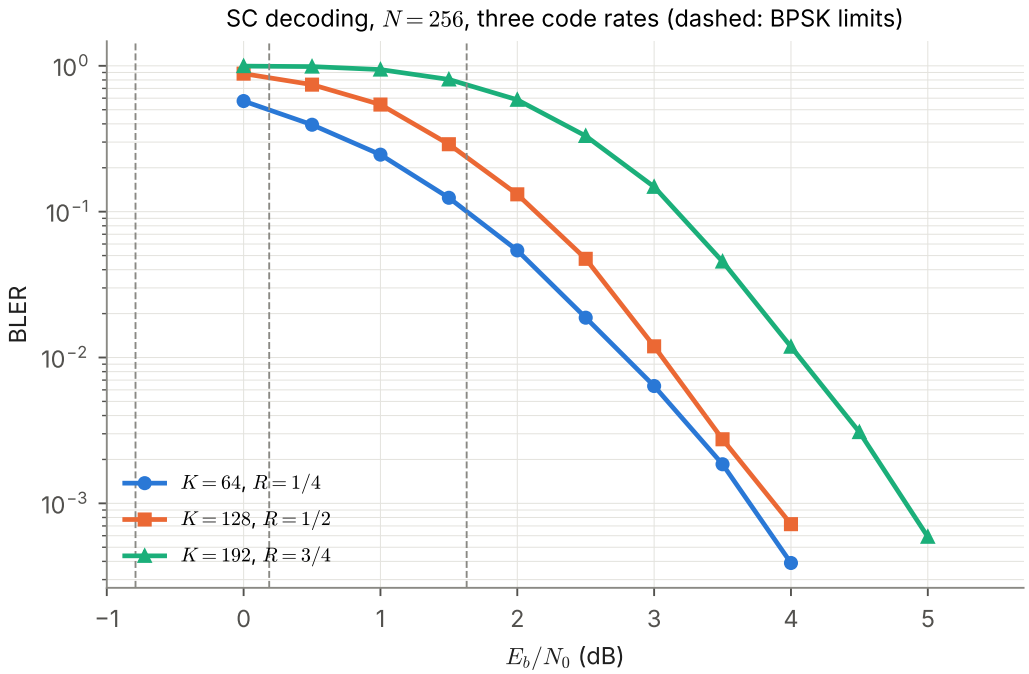


Figure 11.3. SC decoding of $N = 256$ codes at three rates (GA at 2 dB). Dashed lines: BPSK-input Shannon limits.

11.13 How the code was tested

1. **Encoder.** `polar_encode(u)` equals $\mathbf{u}\mathbf{G}_N$ (matrix product) for $N = 2, \ldots, 256$ and random $\mathbf{u}$.
2. **Noiseless decoding.** With large LLRs, SC and SCL return the transmitted $\mathbf{u}$.
3. **SC equals the bit-channel definition.** For $N = 8$ and 200 random LLR vectors, the SC

decoder with exact f made the same decisions as a brute-force evaluation of $\ln \frac{W_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}|0)}{W_N^{(i)}(\mathbf{y}, \hat{u}_0^{i-1}|1)}$ that sums over all future bits.

4. **SCL equals ML for a large list.** For the $(8, 4)$ code and $L = 16 \geq 2^K$, SCL returned the minimum-discrepancy codeword in 300 of 300 random trials.
5. **CRC.** Attached words pass the check; any single flipped bit fails; the classroom example $1011 \to 000$ is reproduced.

Speed

Pure Python recursion over single bits is slow. Processing thousands of blocks and all list paths at once with NumPy gave about 1–2 ms per $(256, 128)$ block for SCL with $L = 8$ on one CPU core — fast enough for all curves in this book. For research-grade simulations at BLER 10^{-5} and below, port `_node` to C++, Numba or a GPU.

11.14 Chapter summary

Key concepts

- One library covers encoding, construction, channel, three decoders, CRC and simulation.
- Vectorising over blocks and list paths makes Python fast enough for teaching-level simulations.
- The parent-map recursion is a compact way to implement list decoding without explicit memory copies.
- Always validate a decoder against brute force on small codes before trusting simulation curves.

Visual recap • Chapter 11



`simulate()` loops this chain and counts BER/BLER

Common student mistakes

! Common mistake

- Using the payload rate K/N instead of the message rate for σ in CA-SCL simulations.
- Stopping a simulation after a fixed number of blocks, regardless of how many errors were counted.
- Re-using the same random seed for different SNR points *and* comparing tiny differences between them.
- Testing only the BER of the codeword bits instead of the message bits.

Practice problems

Problem 11.1. Modify `simulate` so that it also returns the number of block errors per point.

Solution Append `blk_err` to a list `nerr` inside the loop and add `"errors": nerr` to the returned dictionary.

Problem 11.2. Use the library to compare SC with exact f and with min-sum f at $N = 256$, $R = 1/2$, 2.5 dB.

Solution Create `ListDecoder(info, L=1, exact_f=True)` and the default decoder, decode the same LLR batches and count errors. Expect a small difference (typically below 0.1 dB) in favour of the exact function.

Problem 11.3. How would you simulate puncturing of the first p coded bits?

Solution After `channel_llr`, set `llr[:, :p] = 0`; make sure $u_0, \dots, u_{p-1}$ are frozen; use $E = N - p$ in the rate for σ.

Self-assessment quiz

1. Which function would you call to obtain SC decoding?
2. Why does every recursive call return a `parent` array?
3. What does `np.isfinite(PM)` protect against in CA-SCL selection?

Answers: 1) `ListDecoder(info, L=1).decode(llr)`. 2) Because pruning may re-order and duplicate paths; the parents need to know where their data went. 3) Never-activated paths with infinite metric.

CHAPTER

12

Performance Analysis and Engineering Design



IN THIS CHAPTER YOU WILL LEARN

- how to read BER and BLER curves;
- E_b/N_0, E_s/N_0 and SNR, and how to convert;
- coding gain, measured on our own simulations;
- latency, complexity and memory of SC, SCL and CA-SCL;
- polar versus LDPC: where each one wins;
- engineering trade-offs and a design walk-through;
- current research directions.

12.1 BER and BLER curves

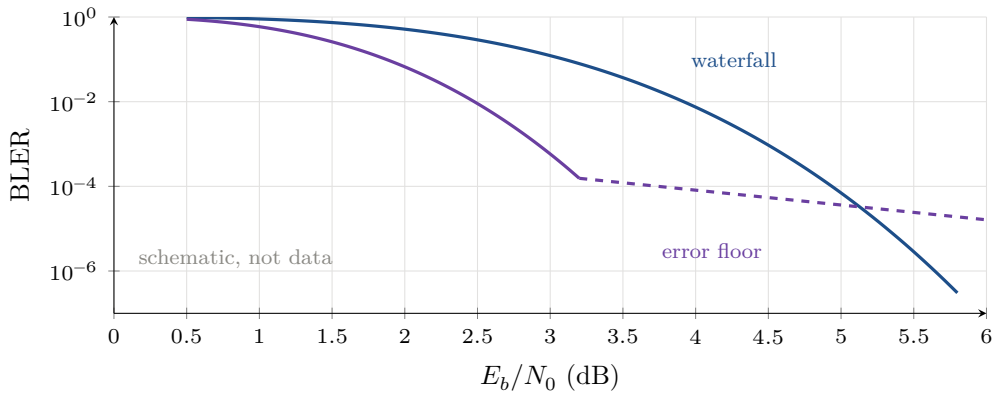


Figure 12.1. Schematic shapes of error-rate curves: a steep waterfall and, for some codes and decoders, a flatter error floor at low error rates. Steeper is better.

- **Waterfall region:** the error rate drops quickly with SNR. Longer codes and stronger decoders give a steeper waterfall (Fig. 9.6).
- **Error floor:** at high SNR the slope can become small, dominated by a few low-weight codewords or decoder failure patterns. With CA-SCL, polar codes have no pronounced floor

in the ranges of practical interest, but a small minimum distance limits the slope (plain SCL in Fig. 7.3).

- **BER vs BLER:** BER is always lower than BLER, because a block error usually contains only some wrong bits. Systems are specified by BLER; BER is useful when comparing with uncoded BPSK.

12.2 E_b/N_0 and SNR

$$\frac{E_s}{N_0} = R \cdot m \cdot \frac{E_b}{N_0}, \qquad \left(\frac{E_s}{N_0}\right)_{\text{dB}} = \left(\frac{E_b}{N_0}\right)_{\text{dB}} + 10\log_{10}(R\,m),$$

where m is the number of coded bits per modulation symbol ($m = 1$ BPSK, 2 QPSK, 4 16-QAM). The "SNR" reported by a receiver is a signal-to-noise power ratio per symbol; with ideal Nyquist pulses and one complex sample per symbol it equals E_s/N_0.

Fig. 12.2 shows how a simulator turns the E_b/N_0 on the horizontal axis of a figure into the noise it adds. Every block must use the same definitions, otherwise curves from different codes cannot be compared.

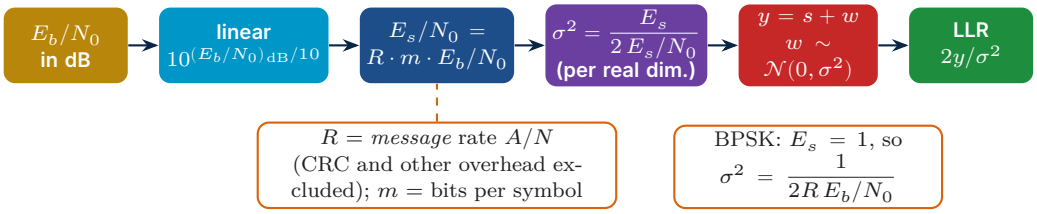


Figure 12.2. Block diagram. From E_b/N_0 to noise variance and LLRs: the normalisation used in every simulation of this book.

Worked Example 12.1 • Conversions

Rate 1/2 with QPSK at $E_b/N_0 = 2\,\text{dB}$: $10\log_{10}(0.5 \cdot 2) = 0$, so $E_s/N_0 = 2\,\text{dB}$. Rate 1/3 with BPSK at $E_b/N_0 = 2\,\text{dB}$: $E_s/N_0 = 2 - 4.77 = -2.77\,\text{dB}$, i.e. every coded symbol is weaker than the noise.

12.3 Coding gain

Definition 12.1 • Coding gain

The **coding gain** at a target error rate is the reduction in required E_b/N_0 compared with an uncoded system (or with another code) at the same *information* error rate.

Worked Example 12.2 • Coding gain from our simulations ($N = 256$, $R = 1/2$)

Uncoded BPSK needs 6.79 dB for BER 10^{-3} and 8.40 dB for BER 10^{-4}. From Fig. 8.4 (log-linear interpolation):

	SC	SCL $L = 8$	CA-SCL $L = 8$	CA-SCL $L = 16$
E_b/N_0 for BER 10^{-3}	3.23 dB	2.62 dB	2.60 dB	2.45 dB
coding gain at BER 10^{-3}	3.6 dB	4.2 dB	4.2 dB	4.3 dB
E_b/N_0 for BER 10^{-4}	4.00 dB	3.49 dB	≈ 3.1 dB (extrap.)	—
coding gain at BER 10^{-4}	4.4 dB	4.9 dB	≈ 5.3 dB	—

A gain of 4.9 dB means the coded link needs about 3 times less transmit power than the uncoded one for the same information BER — with a code of only 256 bits. Longer codes give more.

12.4 Decoder latency

SC (tree, one level per cycle)	$2N - 2$ cycles; $N = 256$: 510 cycles
Semi-parallel SC	$2N + \frac{N}{P}\log_2 \frac{N}{4P}$ cycles with P processing elements (Leroux et al.)
Fast-SSC	visits only special nodes: for our $(256, 128)$ GA code the tree reduces from 511 nodes to 51 visited nodes (26 special nodes)
SCL	like SC plus a sorting step for each information bit; sorter delay grows with L
CA-SCL	SCL plus one CRC check per surviving path (cheap); early termination can stop failed candidates

12.5 Computational complexity and memory

	SC	SCL (L)	CA-SCL (L, CRC-r)
f/g operations	$N \log_2 N$	$L\,N \log_2 N$	$L\,N \log_2 N$
example $N = 256$	2 048	16 384 ($L = 8$)	16 384 ($L = 8$)
sorting	—	K selections of L from $2L$	same
LLR memory	$\approx 2N$	$\approx N + L \cdot N$	same
extra	—	path metrics, pointers	CRC check of L paths

12.6 SC vs SCL vs CRC-SCL

	SC	SCL	CA-SCL
BLER 10^{-2}, $(256, 128)$	3.07 dB	2.56 dB ($L = 8$)	2.37 dB ($L = 8$), 2.19 dB ($L = 16$)
Slope at low BLER	moderate	limited by $d_{\min}$	steep
Cost	lowest	$\approx L\times$ SC	$\approx L\times$ SC
Error detection	needs separate CRC	needs separate CRC	built in (partly used for selection)
Typical use	theory, very high throughput	research baseline	5G NR and most practical systems

12.7 Comparison with LDPC codes

- **Short blocks (up to a few hundred bits):** CA-SCL polar codes are among the best known practical codes and were preferred for 5G control. LDPC codes at these lengths need careful design to avoid early error floors.
- **Long blocks, high throughput:** LDPC codes with parallel belief-propagation decoders reach very high throughput and approach capacity well; SCL's sequential nature makes very high throughput harder.
- **Rate flexibility:** both are flexible; NR LDPC uses two base graphs with lifting and circular-buffer rate matching, NR polar uses one nested sequence with puncturing/shortening/repetition.
- **HARQ with incremental redundancy:** natural for NR LDPC (data channels); control channels in NR do not use it.

Important note

This comparison is qualitative. A quantitative comparison must use the same N, K, channel, decoder complexity and target BLER; published results vary with all of these.

12.8 Practical design trade-offs

Choice	Larger / stronger gives …	but costs …
Block length N	steeper curves, closer to capacity	latency, memory, decoder size
List size L	better BLER (with CRC)	area, power, sorting delay
CRC length r	better selection and detection	rate loss (critical for small A)
Design SNR (GA)	construction matched to operating point	mismatch if the channel differs
Quantization bits	less implementation loss	memory and logic
Rate-matching mode	best mode per K/E	complexity of pre-freezing rules

Worked Example 12.3 • Design walk-through: a 40-bit control message

Requirement. $A = 40$ bits, BLER $\leq 10^{-2}$, error detection better than 10^{-5}, decoder must be small.
1. CRC. Detection $\leq 10^{-5}$ after list selection: with $L = 8$ about 3 CRC bits are spent; need $2^{-(r-3)} \leq 10^{-5} \Rightarrow r \geq 20$. Choose CRC24C (as NR DCI): $K = 64$.
2. Length. With $E = 216$ channel bits available (two CCEs), NR rules give $N = 256$ with puncturing (Chapter 10).
3. Construction. NR sequence after removing pre-frozen positions; or GA at the expected operating SNR.
4. Decoder. CA-SCL with $L = 8$ (good trade-off, Chapter 8); Fast-SSC-style special nodes to cut latency.
5. Verification. Simulate BLER and *false-alarm* rate (random noise input passing the CRC) with enough frames: to verify 10^{-5} you need well over 10^6 noise-only trials.

12.9 Current research directions

- **PAC codes** (polarization-adjusted convolutional codes, Arıkan 2019): a convolutional pre-transform before the polar transform; with sequential or list decoding they approach finite-length bounds at short lengths.
- **Improved decoders:** SC-flip (re-decode flipping the least reliable decisions), automorphism ensemble decoding, and list decoders with lower sorting cost.
- **Large and non-binary kernels:** kernels bigger than 2×2 (and q-ary kernels, e.g. ternary) give faster polarization (better scaling exponent) at higher decoding cost.
- **Construction and learning:** reliability sequences for fading and MIMO channels; machine-learning-aided construction and decoding.
- **Beyond 5G:** ultra-reliable low-latency communication, short packets for IoT, joint source–channel and security applications, and quantum polar codes.

12.10 Chapter summary

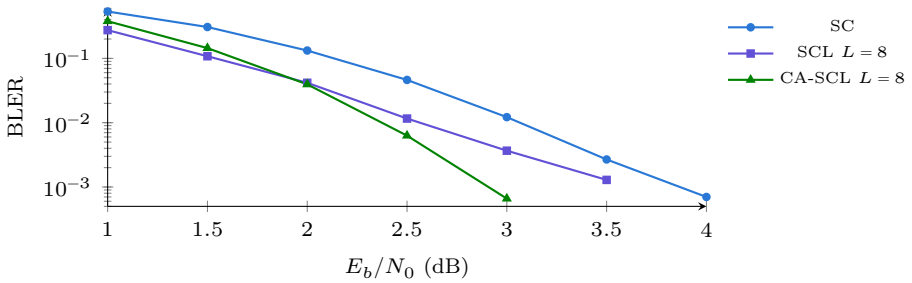
Key concepts

- Read curves by their waterfall position and slope; specify systems in BLER.
- $E_s/N_0 = R\,m\,E_b/N_0$; coding gain is measured at the same information error rate.
- On a $(256, 128)$ code, CA-SCL gains ≈ 0.7 dB over SC at BLER 10^{-2} and ≈ 5 dB over uncoded BPSK at BER 10^{-4}.
- Latency and memory scale with N and L; Fast-SSC removes most of the SC tree work.

Important formulas

SNR conversion	$(E_s/N_0)_{\text{dB}} = (E_b/N_0)_{\text{dB}} + 10\log_{10}(Rm)$
SC latency	$2N - 2$ cycles
SCL operations	$L\,N\log_2 N$
CRC detection after list	$\approx L\,2^{-r}$

Visual recap • Chapter 12



Common student mistakes

! Common mistake

- Reading coding gain at a BLER where one of the curves has only a handful of errors.
- Comparing a BER curve of one system with a BLER curve of another.
- Forgetting m (bits per symbol) when converting between E_b/N_0 and E_s/N_0.
- Claiming "polar is better than LDPC" without stating length, rate, decoder and complexity.

Practice problems

Problem 12.1. A rate-3/4 code uses 16-QAM at $E_b/N_0 = 6\,\text{dB}$. Find E_s/N_0.

Solution $10\log_{10}(0.75 \cdot 4) = 4.77\,\text{dB}$, so $E_s/N_0 = 10.77\,\text{dB}$.

Problem 12.2. Uncoded BPSK needs 9.59 dB for BER 10^{-5}. A coded system needs 4.1 dB. What is the coding gain, and the power ratio?

Solution 5.49 dB, i.e. $10^{0.549} = 3.5$ times less power.

Problem 12.3. How many f/g operations does CA-SCL with $L = 8$ need for $N = 1024$?

Solution $8 \times 1024 \times 10 = 81\,920$.

Self-assessment quiz

1. What limits the slope of plain SCL curves at low BLER?

2. Why do control channels prefer polar codes in NR?
3. What extra simulation is needed to verify CRC error detection?

Answers: 1) The minimum distance of the polar code (SCL is near ML). 2) Short blocks, low BLER, flexible rate matching, early termination in blind decoding. 3) False-alarm tests with noise-only or wrong-RNTI inputs.

PART V

Expert Practice

Fully solved problems, interview and examination questions, and an end-to-end final project.

CHAPTER

13



Fully Solved Problems

IN THIS CHAPTER YOU WILL LEARN

Twenty problems, from basic to advanced. Try each one with paper and pencil before reading the solution. Every numerical answer was checked with the code of Chapter 11. Unless stated otherwise: $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ in natural order, BPSK $0 \mapsto +1$, min-sum f, approximate path metric.

13.1 Basic level

Problem 13.1. (BPSK error rate) Uncoded BPSK runs at $E_b/N_0 = 5\,\text{dB}$. Find the BER.

Solution $10^{0.5} = 3.162$; $\sqrt{2 \times 3.162} = 2.515$; $P_b = Q(2.515) = 5.95 \times 10^{-3}$.

Problem 13.2. (Rate and noise) A polar code has $N = 128$, $K = 80$. BPSK with $E_s = 1$, $E_b/N_0 = 3\,\text{dB}$. Find R, E_s/N_0, N_0 and σ.

Solution $R = 80/128 = 0.625$. $E_s/N_0 = 0.625 \times 10^{0.3} = 0.625 \times 1.995 = 1.247$. $N_0 = 1/1.247 = 0.802$. $\sigma = \sqrt{N_0/2} = 0.633$.

Problem 13.3. (Kronecker product) Compute $\mathbf{G}_4 = \mathbf{F} \otimes \mathbf{F}$, give the row weights, and verify $\mathbf{G}_4\mathbf{G}_4 = \mathbf{I}$ over GF(2) for the first row of the product.

Solution $\mathbf{G}_4 = \begin{bmatrix} 1&0&0&0 \\ 1&1&0&0 \\ 1&0&1&0 \\ 1&1&1&1 \end{bmatrix}$, row weights $1, 2, 2, 4$ ($= 2^{w(i)}$ for $i = 0, 1, 2, 3$). Row 0 of $\mathbf{G}_4\mathbf{G}_4$ is row 0 of $\mathbf{G}_4$ times $\mathbf{G}_4 = 1 \cdot (1000) = 1000$. Row 3: $(1111)\mathbf{G}_4 = 1000 \oplus 1100 \oplus 1010 \oplus 1111 = 0001$.

Problem 13.4. (Channel LLRs) With $\sigma = 0.9$ the receiver observes $\mathbf{y} = (0.7, -1.3, 0.1, 1.1)$. Compute the LLRs and the hard decisions.

Solution $2/\sigma^2 = 2.469$. $\mathbf{L} = (1.73, -3.21, 0.25, 2.72)$; hard decisions $(0, 1, 0, 0)$. The third decision is very unreliable.

Problem 13.5. (Erasure recursion) For BEC(0.2) and $N = 4$, compute all $Z(W_4^{(i)})$, choose $\mathcal{A}$ for $K = 2$, and bound the SC block error probability.

Solution $\epsilon^- = 2(0.2) - 0.04 = 0.36$, $\epsilon^+ = 0.04$.
$Z_0 = 2(0.36) - 0.36^2 = 0.5904$, $Z_1 = 0.36^2 = 0.1296$, $Z_2 = 2(0.04) - 0.04^2 = 0.0784$, $Z_3 = 0.0016$.
$\mathcal{A} = \{2, 3\}$, bound $P_e \leq 0.0784 + 0.0016 = 0.080$. Check: capacities add to $4 \times 0.8 = 3.2$.

13.2 Intermediate level

Problem 13.6. (Encoding with all stages) $(8, 4)$ code, $\mathcal{A} = \{3, 5, 6, 7\}$. Encode $\mathbf{m} = (0, 1, 1, 0)$.

Solution $\mathbf{u} = (0, 0, 0, 0, 0, 1, 1, 0)$.
Stage 1 (span 4): $v = (u_0 \oplus u_4,\ u_1 \oplus u_5,\ u_2 \oplus u_6,\ u_3 \oplus u_7,\ u_4, u_5, u_6, u_7) = (0, 1, 1, 0, 0, 1, 1, 0)$.
Stage 2 (span 2): $w = (v_0 \oplus v_2,\ v_1 \oplus v_3,\ v_2, v_3,\ v_4 \oplus v_6,\ v_5 \oplus v_7,\ v_6, v_7) = (1, 1, 1, 0, 1, 1, 1, 0)$.
Stage 3 (span 1): $x = (w_0 \oplus w_1, w_1, w_2 \oplus w_3, w_3, w_4 \oplus w_5, w_5, w_6 \oplus w_7, w_7) = (0, 1, 1, 0, 0, 1, 1, 0)$.
Matrix check: $\text{row}_5 \oplus \text{row}_6 = 11001100 \oplus 10101010 = 01100110$.

Problem 13.7. (Information set and distance) Using the BEC(0.5) table for $N = 16$ (Chapter 4), find $\mathcal{A}$ for $K = 6$, the SC bound and $d_{\min}$.

Solution The six smallest Z: indices $15, 14, 13, 11, 7, 12$, so $\mathcal{A} = \{7, 11, 12, 13, 14, 15\}$.
Bound: $0.00002 + 0.00780 + 0.01466 + 0.03664 + 0.10011 + 0.22752 = 0.387$.
Binary weights: $7 \to 3$, $11 \to 3$, $12 \to 2$, $13 \to 3$, $14 \to 3$, $15 \to 4$; smallest row weight $2^2 = 4$: $d_{\min} = 4$.

Problem 13.8. (f and g) Compute exact f, min-sum f and both g values for $a = -2.4$, $b = 0.9$.

Solution $\tanh(-1.2) = -0.8337$, $\tanh(0.45) = 0.4219$, product -0.3517, exact $f = 2\tanh^{-1}(-0.3517) = -0.735$. Min-sum: $(-)(+)\min(2.4, 0.9) = -0.9$.
$g(a, b, 0) = 0.9 - 2.4 = -1.5$; $g(a, b, 1) = 0.9 + 2.4 = 3.3$.

Problem 13.9. (SC decoding, $N = 4$) $\mathcal{A} = \{2, 3\}$. Received LLRs $(-0.8,\ 1.5,\ 0.4,\ -1.1)$. Decode with SC.

Solution Root f: $f(-0.8, 0.4) = -0.4$, $f(1.5, -1.1) = -1.1$.
u_0: $f(-0.4, -1.1) = +0.4$, frozen $\Rightarrow 0$. u_1: $g(-0.4, -1.1, 0) = -1.5$, frozen $\Rightarrow 0$. Partial sums $(0, 0)$.
Root g with $(0, 0)$: $(0.4 - 0.8,\ -1.1 + 1.5) = (-0.4,\ 0.4)$.
u_2: $f(-0.4, 0.4) = -0.4 \Rightarrow \hat{u}_2 = 1$. u_3: $g(-0.4, 0.4, 1) = 0.4 + 0.4 = 0.8 \Rightarrow \hat{u}_3 = 0$.
$\hat{\mathbf{u}} = (0, 0, 1, 0)$, $\hat{\mathbf{x}} = (1, 0, 1, 0)$, message $(1, 0)$.

Problem 13.10. (SC decoding, $N = 8$) $(8, 4)$ code, $\mathcal{A} = \{3, 5, 6, 7\}$. Decode the LLRs

$$\alpha = (1.2,\ -0.7,\ -1.4,\ 0.5,\ -0.3,\ 1.6,\ -2.2,\ -0.9).$$

Solution

Root f	$(-0.3, -0.7, 1.4, -0.5)$
Node $u_0..u_3$, f	$(-0.3, 0.5) \Rightarrow u_0$: $L = -0.3$ (frozen, 0); u_1: $L = 0.2$ (frozen, 0)
Node $u_0..u_3$, g	$(1.1, -1.2) \Rightarrow u_2$: $L = -1.1$ (frozen, 0); u_3: $L = -0.1 \Rightarrow 1$
Left partial sums	$(1, 1, 1, 1)$
Root g $(= \alpha_b - \alpha_a)$	$(-1.5, 2.3, -0.8, -1.4)$
Node $u_4..u_7$, f	$(0.8, -1.4) \Rightarrow u_4$: $L = -0.8$ (frozen, 0); u_5: $L = -0.6 \Rightarrow 1$
Node $u_4..u_7$, g with $(1, 1)$	$(0.7, -3.7) \Rightarrow u_6$: $L = -0.7 \Rightarrow 1$; u_7: $L = -4.4 \Rightarrow 1$

$\hat{\mathbf{u}} = (0, 0, 0, 1, 0, 1, 1, 1)$, message $(1, 1, 1, 1)$, codeword $\hat{\mathbf{x}} = (0, 1, 1, 0, 1, 0, 0, 1)$. Note u_3 was decided with $|L| = 0.1$ — a risky decision that a list decoder would double-check.

Problem 13.11. (CRC) Compute the CRC of $\mathbf{m} = 11010110$ with $g(D) = D^3 + D + 1$.

Solution Divide $11010110\,000$ by 1011 (XOR at each leading one): $11010110000 \to 01100110000 \to 00111110000 \to 00010010000 \to 00000100000 \to 00000001100 \to 00000000111$.
The last three bits are the remainder: $\mathbf{p} = 111$. Transmitted word $11010110\,111$ (verified with `crc_bits`).

Problem 13.12. (Polarization weight) Compute the PW ranking for $N = 8$ and compare it with the NR sequence.

Solution $\beta = 2^{1/4} = 1.189$, $\beta^2 = 1.414$. $W_0 = 0$, $W_1 = 1$, $W_2 = 1.189$, $W_3 = 2.189$, $W_4 = 1.414$, $W_5 = 2.414$, $W_6 = 2.603$, $W_7 = 3.603$. Order: $0, 1, 2, 4, 3, 5, 6, 7$ — identical to the NR sub-sequence for $N = 8$.

13.3 Advanced level

Problem 13.13. (SCL, $N = 4$) $\mathcal{A} = \{2, 3\}$, $L = 2$, LLRs $(0.6, -0.4, -0.9, 0.3)$. Give all candidates with metrics.

Solution u_0 (F): $L = f(f(0.6, -0.9), f(-0.4, 0.3)) = f(-0.6, -0.3) = +0.3$; path "0", PM 0.
u_1 (F): $L = g(-0.6, -0.3, 0) = -0.9$; deciding 0 costs 0.9: "00", PM 0.9.
u_2 (I): $L = +0.1$: "000" (0.9), "001" (1.0) — both kept.
u_3 (I): on "000" $L = -0.4$: "0001" (0.9), "0000" (1.3); on "001" $L = +0.2$: "0010" (1.0), "0011" (1.2).
Keep "0001" (0.9) and "0010" (1.0). Output $\hat{\mathbf{u}} = (0, 0, 0, 1)$, $\hat{\mathbf{x}} = (1, 1, 1, 1)$.
Check by discrepancy: hard decisions $(0, 1, 1, 0)$; codeword 1111 disagrees at positions 0 and 3: $0.6 + 0.3 = 0.9$.

Problem 13.14. (Bhattacharyya construction for AWGN) BI-AWGN with $E_s/N_0 = 0.5$. Using the BEC-style recursion, compute bounds on Z for $N = 4$ and choose $\mathcal{A}$ for $K = 2$.

Solution $Z = e^{-0.5} = 0.607$. Level 1: $Z^- \le 0.845$, $Z^+ = 0.368$. Level 2: $Z_0 \le 0.976$, $Z_1 \le 0.714$, $Z_2 \le 0.600$, $Z_3 = 0.135$. $\mathcal{A} = \{2, 3\}$, SC bound 0.736 (loose at such low SNR).

Problem 13.15. (Gaussian approximation by hand) Channel LLR mean $m = 2$. Using Chung's closed form for ϕ (valid for $0 < x < 10$), find m^-, m^+ and the error estimates.

Solution $\phi(2) = e^{-0.4527 \cdot 2^{0.86} + 0.0218} = 0.4494$. Target $1 - (1 - 0.4494)^2 = 0.6968$. Inverse of the closed form: $x = ((0.0218 - \ln t)/0.4527)^{1/0.86}$ gives $m^- = 0.823$ (numerical integration gives 0.822). $m^+ = 4$. Errors $Q(\sqrt{m/2})$: original $Q(1) = 0.159$; minus $Q(0.642) = 0.261$; plus $Q(1.414) = 0.079$.

Problem 13.16. (Fast-SSC) Decode Problem 13.10 with Rep and SPC nodes.

Solution Rep node $u_0..u_3$: $\sum \alpha = -0.3 - 0.7 + 1.4 - 0.5 = -0.1 < 0 \Rightarrow \beta_L = (1, 1, 1, 1)$.
g: $(-1.5, 2.3, -0.8, -1.4)$. SPC node: hard $(1, 0, 1, 1)$, parity $1 \Rightarrow$ flip the least reliable ($|-0.8|$, position 2): $\beta_R = (1, 0, 0, 1)$.
$\hat{\mathbf{x}} = [(0, 1, 1, 0) \,|\, (1, 0, 0, 1)]$, the same as SC.

Problem 13.17. (Puncturing) $N = 4$, $\mathcal{A} = \{2, 3\}$. Bit x_0 is punctured. The other LLRs are $(\cdot, 1.2, -0.9, 1.4)$. Decode with SC.

Solution Set $L_0 = 0$. Root f: $f(0, -0.9) = 0$, $f(1.2, 1.4) = 1.2$. u_0: $f(0, 1.2) = 0$, frozen. u_1: $g = 1.2$, frozen.
Root g: $(-0.9 + 0,\ 1.4 + 1.2) = (-0.9, 2.6)$. u_2: $f = -0.9 \Rightarrow 1$. u_3: $g(-0.9, 2.6, 1) = 3.5 \Rightarrow 0$.
$\hat{\mathbf{u}} = (0, 0, 1, 0)$, $\hat{\mathbf{x}} = (1, 0, 1, 0)$: the punctured bit is recovered as $\hat{x}_0 = 1$. The frozen bit u_0 "absorbed" the lost information — this is why the first indices must be frozen when the first bits are punctured.

Problem 13.18. (Systematic encoding) $N = 4$, $\mathcal{A} = \{2, 3\}$, $\mathbf{m} = (1, 0)$. Encode systematically.

Solution $\mathbf{u} = (0, 0, 1, 0) \Rightarrow \mathbf{v} = \mathbf{u}\mathbf{G}_4 = (1, 0, 1, 0)$. Zero v_0, v_1: $(0, 0, 1, 0)$. Encode again: $\mathbf{x} = (1, 0, 1, 0)$. Then $x_2, x_3 = (1, 0) = \mathbf{m}$. (Here the systematic and non-systematic codewords happen to coincide.)

Problem 13.19. (5G length selection) UCI with $A = 100$ bits, CRC11, $E = 300$, $n_{\max} = 10$. Find K, N, the mode and the number of pre-frozen leading indices.

Solution $K = 111$. $\lceil \log_2 300 \rceil = 9$; $300 \le \frac{9}{8} \cdot 256 = 288$? No $\Rightarrow n_1 = 9$. $n_2 = \lceil \log_2 888 \rceil = 10$. $n = 9$, $N = 512$.
$E < N$, $K/E = 0.37 \le 0.4375$: puncturing of 212 bits. $E < \frac{3}{4}N = 384 \Rightarrow$ first $\lceil \frac{9N}{16} - \frac{E}{4} \rceil = \lceil 288 - 75 \rceil = 213$ indices are pre-frozen (in addition to the $J(n)$, $n < 212$).

Problem 13.20. (CRC and list size) CA-SCL with $L = 8$ uses CRC11. Estimate the undetected-error probability when the transmitted message is not in the list. Which CRC length gives $\leq 10^{-6}$?

Solution $\approx L\,2^{-r} = 8/2048 = 3.9 \times 10^{-3}$. Need $8 \cdot 2^{-r} \leq 10^{-6} \Rightarrow 2^r \geq 8 \times 10^6 \Rightarrow r \geq 23$. CRC24 is the natural choice — one reason NR DCI uses CRC24C.

13.4 Summary of methods

Encoding	butterflies, span $N/2, \ldots, 1$; check with rows of $\mathbf{G}_N$
Construction	BEC recursion, binary index rule, GA, PW, NR sequence
SC	tree: f left, g right, partial sums up; frozen $\to 0$
SCL	split at information bits, penalty $\lvert L\rvert$, keep L best
Checks	capacity conservation, matrix re-encoding, correlation discrepancy, CRC

CHAPTER

14



Interview and Examination Questions

IN THIS CHAPTER YOU WILL LEARN

Thirty-two questions with model answers, in four levels: beginner, intermediate, advanced and engineering interview. The answers are written the way an examiner would like to read them: short, precise, with the key formula or number.

14.1 Beginner level

Problem 14.1. What is a polar code, in one sentence?

Solution A linear block code of length $N = 2^n$ with generator $\mathbf{G}_N = \mathbf{F}^{\otimes n}$, where the K most reliable input positions carry data and the others are frozen to known values.

Problem 14.2. What does "polarization" mean?

Solution By combining and splitting N copies of a channel, the N synthetic bit-channels become either almost perfect or almost useless; the fraction of almost perfect ones tends to the capacity $I(W)$.

Problem 14.3. What is a frozen bit?

Solution An input bit placed on an unreliable bit-channel whose value is fixed in advance (normally 0) and known to the decoder.

Problem 14.4. What is the code rate of a $(1024, 512)$ polar code? How many frozen bits does it have?

Solution $R = 1/2$; 512 frozen bits.

Problem 14.5. Why must N be a power of two, and how do practical systems get other lengths?

Solution Because $\mathbf{G}_N$ is a Kronecker power of a 2×2 kernel. Other lengths are obtained by puncturing, shortening or repetition.

Problem 14.6. What is the difference between BER and BLER?

Solution BER is the fraction of wrong information bits; BLER is the fraction of blocks with at least one wrong bit. Systems are specified by BLER.

Problem 14.7. Write the LLR of BPSK over AWGN.

Solution $L = 2y/\sigma^2$ with $0 \mapsto +1$; positive values favour bit 0.

Problem 14.8. Who invented polar codes and when?

Solution Erdal Arıkan; the main paper was published in IEEE Trans. Information Theory in July 2009.

14.2 Intermediate level

Problem 14.9. Derive the erasure probabilities of W^- and W^+ for a BEC(ϵ).

Solution $u_0 = x_0 \oplus x_1$ needs both outputs: $\epsilon^- = 1 - (1-\epsilon)^2 = 2\epsilon - \epsilon^2$. u_1 is lost only if both outputs are erased: $\epsilon^+ = \epsilon^2$.

Problem 14.10. Prove that one polarization step conserves capacity.

Solution $(u_0, u_1) \mapsto (x_0, x_1)$ is invertible, so $I(U_0U_1; Y_0Y_1) = 2I(W)$. By the chain rule this equals $I(U_0; Y_0Y_1) + I(U_1; Y_0Y_1U_0) = I(W^-) + I(W^+)$.

Problem 14.11. Give the f and g functions and explain each in words.

Solution $f(a,b) = 2\tanh^{-1}(\tanh\frac{a}{2}\tanh\frac{b}{2}) \approx \operatorname{sign} a \operatorname{sign} b \min(|a|,|b|)$: LLR of an XOR of two bits. $g(a,b,c) = b + (1-2c)a$: two independent views of the same bit after removing the known bit c.

Problem 14.12. What is the complexity of encoding and SC decoding?

Solution Both $O(N \log N)$: $\frac{N}{2}\log_2 N$ XORs for encoding and $N \log_2 N$ f/g operations for SC.

Problem 14.13. Why does SC decoding suffer from error propagation?

Solution Each decision is final and is used in the partial sums of all later g computations; a wrong early decision corrupts later LLRs.

Problem 14.14. How is the information set chosen?

Solution By ranking bit-channel reliabilities (Bhattacharyya/BEC recursion, density evolution, GA, PW or a standard sequence) and taking the K best indices.

Problem 14.15. What is the minimum distance of a polar code?

Solution $d_{\min} = \min_{i \in \mathcal{A}} 2^{w(i)}$, where $w(i)$ is the Hamming weight of the binary expansion of i.

Problem 14.16. What happens if a frozen bit's LLR is negative during SC decoding?

Solution Nothing special: the decoder still sets it to the frozen value 0. Using the sign would be a bug.

14.3 Advanced level

Problem 14.17. Derive the LLR-based path metric of SCL.

Solution $\mathrm{PM} = -\ln P(\hat{u}_0^i|\mathbf{y}) = \sum_j \ln(1+e^{-(1-2\hat{u}_j)L_j})$, using $P(\hat{u}_j|\mathbf{y}, \hat{u}_0^{j-1}) = 1/(1+e^{-(1-2\hat{u}_j)L_j})$. With $\ln(1+e^x) \approx \max(0,x)$: add $|L_j|$ when the decision disagrees with the sign of L_j.

Problem 14.18. Why does SCL without CRC saturate as L grows?

Solution For large L, SCL approaches ML decoding of the polar code; the remaining errors are due to codewords that really are closer to the received signal, i.e. to the small minimum distance of the code.

Problem 14.19. Why does a CRC improve list decoding so much?

Solution It selects the correct candidate when it is in the list but not at the top; the CRC–polar concatenation has a larger minimum distance, and CA-SCL is a near-ML decoder for it.

Problem 14.20. Explain the Gaussian approximation.

Solution Assume every bit-channel LLR is consistent Gaussian $\mathcal{N}(m, 2m)$; track the mean: $m^+ = 2m$, $m^- = \phi^{-1}(1-(1-\phi(m))^2)$; estimate $P_e \approx Q(\sqrt{m/2})$.

Problem 14.21. Puncturing or shortening: which LLR does the receiver use, and which $\mathbf{u}$ positions are frozen?

Solution Puncturing: LLR 0, freeze the corresponding low indices (bit-channels become useless). Shortening: LLR $+\infty$, freeze the corresponding high indices so that the shortened coded bits are known zeros.

Problem 14.22. Name four special nodes of Fast-SSC and how they are decoded.

Solution Rate-0: all zeros. Rate-1: hard decisions. Repetition: sign of the LLR sum. SPC: hard decisions, flip the least reliable bit if the parity is odd.

Problem 14.23. What is the speed of polarization?

Solution For any $\beta < 1/2$ the Bhattacharyya parameters of good channels decay like 2^{-N^β} (Arıkan–Telatar), so SC block error probability can be made $O(2^{-N^\beta})$ at rates below capacity. The finite-length gap shrinks like $N^{-1/\mu}$ with $\mu \approx 3.6$ (BEC, SC).

Problem 14.24. Describe systematic polar encoding.

Solution Same codebook, but the message is placed in $x_{\mathcal{A}}$. Encode $\mathbf{u}$ with the message on $\mathcal{A}$, zero the frozen positions of the result, encode again. Better BER, same BLER.

14.4 Engineering interview level

Problem 14.25. Why did 3GPP choose polar codes for control and LDPC for data?

Solution Control blocks are short and need very low BLER, flexible sizes and efficient blind decoding: CA-SCL polar codes are excellent there. Data blocks are long and need very high throughput and HARQ: LDPC with parallel decoding fits better.

Problem 14.26. How does a UE find its DCI?

Solution It blindly decodes PDCCH candidates in its search spaces; the DCI CRC is scrambled with the UE's RNTI, so only the intended UE's CRC check passes. Distributed CRC allows early termination of wrong candidates.

Problem 14.27. How is N chosen in NR?

Solution $n = \max(\min(n_1, n_2, n_{\max}), 5)$ with $n_1 \approx \lceil \log_2 E \rceil$ (minus one when E is just above a power of two and the rate is low), $n_2 = \lceil \log_2(8K) \rceil$, $n_{\max} = 9$ (DL) or 10 (UL).

Problem 14.28. Your SCL decoder works with $L = 1$ but loses performance with $L = 8$. What do you check?

Solution Path-metric sign convention and sorting direction; frozen-bit penalties; copying of partial sums with LLRs; parent re-ordering of partial sums; LLR scaling (exact metric or fixed point); CRC bit order.

Problem 14.29. How do you estimate the BLER of a decoder at 10^{-4} reliably?

Solution Count errors, not frames: run until about 100 or more block errors (relative accuracy about 10%), which needs on the order of 10^6 frames; use fixed seeds and report confidence intervals.

Problem 14.30. What dominates the area of an SCL decoder chip?

Solution Memory: L copies of LLR and partial-sum memories; then the metric sorter. Logic for f/g is comparatively small.

Problem 14.31. How many quantization bits do LLRs need?

Solution Typically 5–6 bits for channel LLRs and a few more internally with saturation; losses around 0.1 dB are commonly reported.

Problem 14.32. A colleague says "polar codes are always better than LDPC". How do you answer?

Solution It depends on block length, rate, decoder complexity, throughput and latency. Polar CA-SCL is very strong at short lengths; LDPC is usually preferred for long, high-throughput

blocks. A fair comparison fixes all these parameters.

14.5 How to prepare

- Be able to draw the $N = 4$ and $N = 8$ butterfly and the SC tree from memory.
- Practice one complete SC and one SCL example by hand (Chapters 6, 7 and 13).
- Know the three key numbers of NR polar coding: sequence length 1024, $n_{\max} = 9/10$, CRC24C/11/6.
- Run the code of Chapter 11 and change one parameter at a time; explain every change in the curves.

CHAPTER

15



Final Professional Project

IN THIS CHAPTER YOU WILL LEARN

- specify a realistic polar-coded control link;
- build every block: data, CRC, encoder, modulator, channel, decoder;
- follow one frame through the whole chain with real numbers;
- measure BER, BLER and undetected errors;
- plot and interpret the results like an engineer;
- extend the project in research directions.

15.1 Project specification

Message	$A = 40$ random bits (the size of a typical 5G DCI)
Error detection	CRC24C (as in NR DCI), $r = 24 \Rightarrow$ payload $K = 64$
Polar code	$N = 128$, information set by GA at design $E_b/N_0 = 2\,\text{dB}$ for the payload rate $K/N = 0.5$
Modulation, channel	BPSK, real AWGN; E_b/N_0 defined per *message* bit: $R = A/N = 0.3125$
Receivers	(1) SC, CRC used only to detect errors; (2) CA-SCL with $L = 8$
Metrics	BER and BLER of the 40 message bits; undetected errors (wrong message that passes the CRC)
Statistics	stop after 150 block errors or 10^5 blocks; points with fewer than 20 errors are not plotted

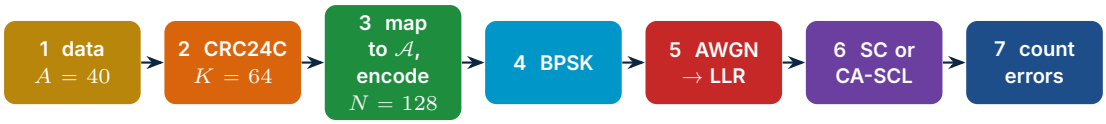


Figure 15.1. The seven steps of the project, numbered as in the code comments.

15.2 The complete program

```python
"""
final_project.py -- End-to-end polar-coded link (Chapter 15).

A 5G-DCI-like control message: A = 40 information bits + CRC24C (K = 64),
mother code N = 128, BPSK over AWGN. Two receivers are compared:
  * SC decoder, CRC used only for error detection
  * CRC-aided SCL decoder with L = 8
For each Eb/N0 we measure BER, BLER and the undetected-error rate
(wrong message that nevertheless passes the CRC).
"""
import json
import numpy as np
from polar import (ga_construct, CRC, polar_encode, bpsk, awgn,
                   channel_llr, ListDecoder)

# ------------------------------------------------------------------ set-up
N, A, CRC_NAME = 128, 40, "CRC24C"
crc = CRC(CRC_NAME, A)
K = A + crc.r                               # 64 payload bits
R = A / N                                   # message rate 0.3125
info = ga_construct(N, K, design_snr_db=2.0, rate=K / N)
receivers = {"SC": ListDecoder(info, L=1, crc=None),
             "CA-SCL L=8": ListDecoder(info, L=8, crc=crc)}


def transmit(m, ebn0_db, rng):
    """Steps 2-5: CRC, mapping, polar encoding, BPSK, AWGN -> LLRs."""
    payload = crc.attach(m)                 # (B, K)  message | CRC
    u = np.zeros((m.shape[0], N), dtype=np.uint8)
    u[:, info] = payload                    # frozen positions stay 0
    x = polar_encode(u)
    y, sigma = awgn(bpsk(x), ebn0_db, R, rng)
    return payload, u, x, y, channel_llr(y, sigma)


# ------------------------------------------------- one frame, printed
rng = np.random.default_rng(2026)
m = rng.integers(0, 2, size=(1, A), dtype=np.uint8)        # step 1
payload, u, x, y, llr = transmit(m, 4.5, rng)
print("message (40 bits)      :", "".join(map(str, m[0])))
print("CRC24C (24 bits)       :", "".join(map(str, payload[0, A:])))
print("info positions (first 8):", np.where(info)[0][:8], "...")
print("codeword x[0:16]       :", "".join(map(str, x[0, :16])))
print("received y[0:6]        :", np.round(y[0, :6], 2))
```

```
print("LLR[0:6]               :", np.round(llr[0, :6], 2))
print("hard-decision errors   :", int(np.sum((llr[0] < 0) != (x[0] == 1))), "of", N)
for name, dec in receivers.items():                          # step 6
    u_hat = dec.decode(llr)
    m_hat = u_hat[:, info][:, :A]
    ok = crc.check(u_hat[:, info])[0]
    print(f"{name:12s}: bit errors = {int(np.sum(m_hat != m))}, CRC pass = {bool(ok)}")

# ------------------------------------------------ Monte-Carlo sweep
def run(dec, ebn0_list, max_frames=100_000, min_errors=150, batch=2000, seed=1):
    rng = np.random.default_rng(seed)
    out = {"ebn0": [], "ber": [], "bler": [], "undetected": [], "frames": []}
    for eb in ebn0_list:
        be = bl = und = fr = 0
        while fr < max_frames and bl < min_errors:
            m = rng.integers(0, 2, size=(batch, A), dtype=np.uint8)
            _, _, _, _, llr = transmit(m, eb, rng)
            u_hat = dec.decode(llr)
            pay_hat = u_hat[:, info]
            wrong = np.any(pay_hat[:, :A] != m, axis=1)
            be += int(np.sum(pay_hat[:, :A] != m))
            bl += int(wrong.sum())
            und += int(np.sum(wrong & crc.check(pay_hat)))      # step 7
            fr += batch
        for k, v in zip(out, [eb, be / (fr * A), bl / fr, und / fr, fr]):
            out[k].append(v)
        print(f"  {eb:4.1f} dB  BER {be/(fr*A):.2e}  BLER {bl/fr:.2e}  undetected {und}/{fr
    }")
    return out

if __name__ == "__main__":
    E = [2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0]
    results = {}
    for name, dec in receivers.items():
        print(name)
        results[name] = run(dec, E)
    json.dump(results, open("../data/final_project.json", "w"), indent=1)
```

MATLAB users

The same project in MATLAB is `final_project.m`. It is printed in Appendix D together with the output it produced.

15.3 Step-by-step numerical example (one frame)

Running the first part of the program (seed 2026, $E_b/N_0 = 4.5\,\text{dB}$) prints:

```
OUTPUT
message (40 bits)      : 1101011010100011111011100100001000111110
CRC24C (24 bits)       : 010110010001100101000000
info positions (first 8): [30 31 45 46 47 51 53 54] ...
codeword x[0:16]       : 0100010011000110
```

```
received y[0:6]        : [ 0.78 -1.24  1.23  0.8   0.83 -0.46]
LLR[0:6]               : [ 2.75 -4.35  4.33  2.81  2.92 -1.61]
hard-decision errors   : 16 of 128
SC          : bit errors = 11, CRC pass = False
CA-SCL L=8  : bit errors = 0, CRC pass = True
```

Worked Example 15.1 • Reading the frame

1. **Data and CRC.** The 40 random bits get 24 CRC bits; the 64-bit payload is placed on $\mathcal{A}$. The least reliable information position is index 30: indices 0–29 are all frozen.
2. **Channel.** At $E_b/N_0 = 4.5\,\text{dB}$ with $R = 0.3125$: $E_s/N_0 = 0.3125 \times 10^{0.45} = 0.881$, $\sigma = \sqrt{1/(2 \cdot 0.881)} = 0.753$ and $L = 2y/\sigma^2 = 3.53\,y$. Check: $y_0 = 0.78 \Rightarrow L_0 = 2.75$. Bit $x_0 = 0$ was sent as $+1$ and received correctly.
3. **Noise.** 16 of the 128 received signs are wrong (raw error rate 12.5%) — a hard frame.
4. **SC** makes an early wrong decision and propagates it: 11 of the 40 message bits are wrong, and the CRC correctly flags the block as bad.
5. **CA-SCL** keeps 8 candidates; the one that passes the CRC is the transmitted message: 0 bit errors.

15.4 Monte-Carlo results

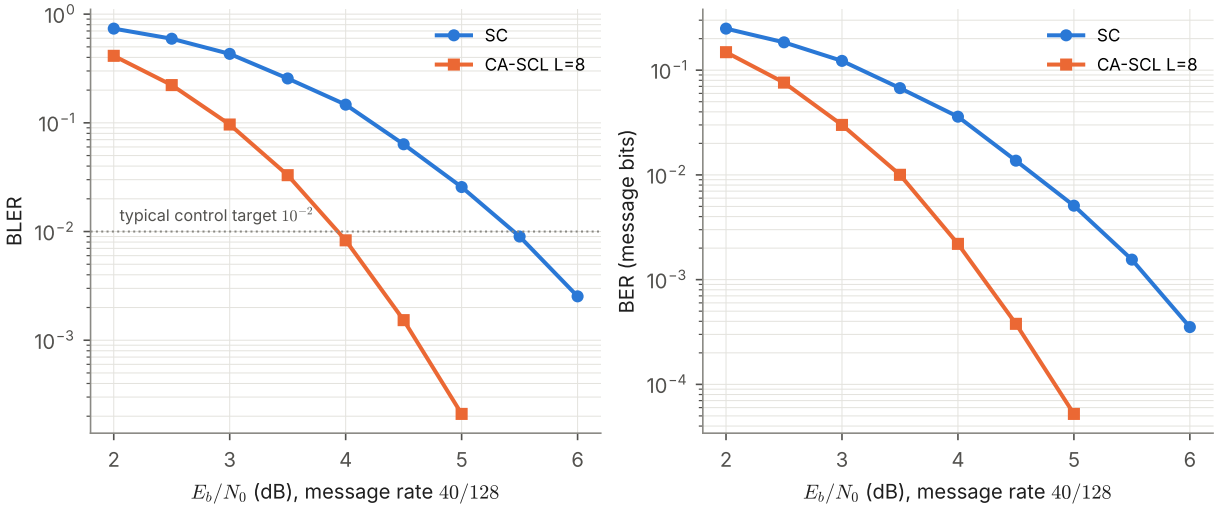


Figure 15.2. Project results: $A = 40$, CRC24C, $N = 128$ over BPSK-AWGN.

E_b/N_0 (dB)	3.0	3.5	4.0	4.5	5.0	5.5
BLER, SC	4.3×10^{-1}	2.6×10^{-1}	1.5×10^{-1}	6.4×10^{-2}	2.6×10^{-2}	9.0×10^{-3}
BLER, CA-SCL $L = 8$	9.7×10^{-2}	3.3×10^{-2}	8.3×10^{-3}	1.5×10^{-3}	2.1×10^{-4}	—
BER, CA-SCL $L = 8$	3.0×10^{-2}	1.0×10^{-2}	2.2×10^{-3}	3.8×10^{-4}	5.2×10^{-5}	—
undetected (CA-SCL)	0	0	0	0	0	0

At 5.5 dB CA-SCL produced only 2 block errors in 10^5 frames (BLER $\approx 2 \times 10^{-5}$) and at 6.0 dB none, too

few for a reliable estimate; SC reached 2.5×10^{-3} at 6.0 dB. No undetected error was observed in any run (4.3×10^5 CA-SCL frames in total).

15.5 Performance interpretation

1. **Gain of CA-SCL.** At the control target BLER 10^{-2}, SC needs about 5.4 dB and CA-SCL about 3.9 dB: a gain of ≈ 1.5 dB, larger than for the $(256, 128)$ code of Chapter 8 because the 24 CRC bits give the list decoder a very strong judge.
2. **Cost of the CRC.** The 24 CRC bits raise the inner code rate from $40/128 = 0.31$ to $64/128 = 0.5$. In E_b/N_0 per message bit, this overhead is $10 \log_{10}(64/40) = 2.0$ dB of energy that carries no data. This is why the curves sit to the right of the $(256, 128)$ results: small control messages are expensive to protect. It is also why NR uses only CRC6/CRC11 for small UCI.
3. **Shape.** The CA-SCL curve is much steeper than SC: one decade of BLER per roughly 0.5–0.7 dB around 10^{-3}.
4. **Error detection.** With $L = 8$ about 3 CRC bits are used for list selection, leaving roughly $2^{-21} \approx 5 \times 10^{-7}$ false-pass probability per failed list. With a few thousand failed lists, observing zero undetected errors is exactly what theory predicts. Verifying the detection target itself needs 10^7 or more noise-only trials.
5. **BER vs BLER.** BER is about 3–4 times smaller than BLER: a failed block typically contains many wrong bits (error propagation in SC, or a wrong but CRC-valid-looking path in CA-SCL).

What a professional report would add

Confidence intervals for each point; curves for $L = 2, 4, 16, 32$; the NR reliability sequence instead of GA; the real NR rate matching for aggregation levels 1–16 ($E = 108, \ldots, 1728$); fading channels; fixed-point LLRs; and latency/complexity figures from an implementation. Each of these is a natural extension below.

15.6 Extensions for students and researchers

1. **List size study.** Repeat for $L = 1, 2, 4, 8, 16, 32$ and plot the required E_b/N_0 at BLER 10^{-2} versus $\log_2 L$.
2. **CRC length study.** Replace CRC24C by CRC11 and CRC6 (change `CRC_NAME`) and plot the trade-off between BLER and undetected-error rate.
3. **5G rate matching.** Implement the sub-block interleaver $J(n)$ and bit selection for $E = 216$ (aggregation level 2) with $N = 256$, including the pre-frozen set of Chapter 10.
4. **Construction mismatch.** Design with GA at 0, 2 and 4 dB and test at 4 dB.
5. **Fixed-point decoder.** Quantize channel LLRs to 6 bits with saturation and measure the loss.
6. **Fading.** Replace AWGN by Rayleigh block fading and compare with the AWGN curves.
7. **Early termination.** Interleave CRC bits (distributed CRC) and count how often wrong candidates can be stopped early.

15.7 Project checklist

Encoder verified against $\mathbf{u}\mathbf{G}_N$; decoders verified on small codes by brute force
E_b/N_0 defined per message bit; σ computed with the message rate
Errors counted on message bits only; CRC bits excluded
Enough error events per point; low-count points marked or removed
Undetected errors counted separately from detected failures
Random seeds fixed and reported; code and data saved with the figures

15.8 Final words

You started with bits, noise and the question "why do we need coding?". You can now explain why channels polarize, encode with butterflies, decode with SC, SCL and CA-SCL by hand and in code, choose frozen sets, read the 5G polar chain, and evaluate a design with honest simulations. The next step is yours: change one block of the project, predict what will happen, and check it.

Appendix A — Mathematical Prerequisites

A.1 Logarithms and decibels

- $\log_b x = y \iff b^y = x$. Natural log $\ln = \log_e$; information is measured with $\log_2$ (bits).
- Rules: $\log(xy) = \log x + \log y$, $\log(x/y) = \log x - \log y$, $\log x^a = a \log x$, $\log_2 x = \ln x / \ln 2$.
- Decibels: $x_{\text{dB}} = 10 \log_{10} x$. Doubling $= +3.01$ dB; $\times 10 = +10$ dB; $\times \frac{1}{2} = -3.01$ dB.
- Useful: $\ln(1 + e^x) \approx \max(0, x)$ for $|x|$ large; $\tanh(x/2) = \frac{e^x - 1}{e^x + 1}$.

Worked Example 15.2 • dB arithmetic

$E_b/N_0 = 2$ dB and $R = 1/2$: $E_s/N_0 = 2 - 3.01 = -1.01$ dB $= 0.792$ (linear).

A.2 Probability

- Conditional probability and Bayes: $P(A|B) = P(B|A)P(A)/P(B)$. With equal priors, $\frac{P(c=0|y)}{P(c=1|y)} = \frac{p(y|0)}{p(y|1)}$, which is why the likelihood ratio is also a posterior ratio.
- Independence: $P(A \cap B) = P(A)P(B)$; independent LLRs of the same bit add.
- Gaussian density $\mathcal{N}(\mu, \sigma^2)$: $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$. Tail: $P(x > \mu + a) = Q(a/\sigma)$.
- $Q(x) = \frac{1}{2} \operatorname{erfc}(x/\sqrt{2})$. Values: $Q(1) = 0.159$, $Q(2) = 0.0228$, $Q(3) = 1.35 \times 10^{-3}$, $Q(4) = 3.17 \times 10^{-5}$.
- Expectation is linear; a martingale is a process whose expected next value equals its current value.

A.3 Binary arithmetic and GF(2)

$\oplus$	0	1
0	0	1
1	1	0

$\cdot$	0	1
0	0	0
1	0	1

- Addition is XOR, multiplication is AND. $a \oplus a = 0$, so subtraction equals addition.
- Binary expansion: $i = \sum_j b_j 2^j$; e.g. $13 = 1101_2$; weight $w(13) = 3$.
- Polynomials over GF(2) are bit strings: $D^3 + D + 1 \leftrightarrow 1011$; division is repeated XOR (CRC).

A.4 Matrices over GF(2)

- Row vector times matrix: $\mathbf{x} = \mathbf{uG}$ means $x_j = \bigoplus_i u_i G_{ij}$ — XOR of the rows of $\mathbf{G}$ selected by the ones of $\mathbf{u}$.
- A square matrix is invertible over GF(2) if its rows are linearly independent. Lower-triangular with ones on the diagonal (like $\mathbf{G}_N$) $\Rightarrow$ invertible.
- $\mathbf{FF} = \left[\begin{smallmatrix} 1 & 0 \\ 1 \oplus 1 & 1 \end{smallmatrix}\right] = \mathbf{I}$, hence $\mathbf{G}_N^{-1} = \mathbf{G}_N$.

A.5 Kronecker products

- Definition: $\mathbf{A} \otimes \mathbf{B} = [a_{ij}\mathbf{B}]$; size $(m_A m_B) \times (n_A n_B)$.
- Mixed-product rule: $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$. With it, $\mathbf{G}_N \mathbf{G}_N = (\mathbf{FF})^{\otimes n} = \mathbf{I}$.
- Not commutative: $\mathbf{F} \otimes \mathbf{I}_2 \neq \mathbf{I}_2 \otimes \mathbf{F}$.
- Entry rule for polar codes: $(\mathbf{F}^{\otimes n})_{ij} = 1$ iff the binary digits of j are $\leq$ those of i position by position ("j is a sub-mask of i"). Hence row i has weight $2^{w(i)}$.

Worked Example 15.3 • Using the sub-mask rule

Row $6 = 110_2$ of $\mathbf{G}_8$ has ones in columns $j \in \{000, 010, 100, 110\} = \{0, 2, 4, 6\}$: 10101010 (Chapter 5).

Appendix B — Glossary

AWGN	Additive white Gaussian noise channel, $y = s + w$, $w \sim \mathcal{N}(0, N_0/2)$.
Bhattacharyya parameter Z	$\sum_y \sqrt{W(y\|0)W(y\|1)}$; small means reliable; upper-bounds the bit error probability.
Bit-channel $W_N^{(i)}$	Synthetic channel seen by u_i when $\mathbf{y}$ and u_0^{i-1} are known.
BLER	Block error rate; fraction of blocks with at least one error.
BP decoding	Iterative message passing on the butterfly graph.
Butterfly	One XOR/copy element of the encoder graph.
CA-SCL	CRC-aided successive cancellation list decoding.
Channel combining	Building W_N from N copies of W via $\mathbf{G}_N$.
Channel splitting	Defining the N bit-channels from W_N.
Code construction	Choice of the information set.
CRC	Cyclic redundancy check; polynomial remainder used for error detection.
DCI, UCI	Downlink / uplink control information in 5G NR.
Density evolution	Exact tracking of LLR densities through f and g.
Distributed CRC	CRC bits interleaved with information bits for early termination.
Early termination	Stopping decoding when an embedded CRC check fails.
Fast-SSC	Decoder that processes special nodes (Rate-0, Rate-1, Rep, SPC) in one step.
Frozen bit / set $\mathcal{F}$	Input bits with a fixed known value, placed on unreliable bit-channels.
GA	Gaussian approximation construction; tracks LLR means.
Generator matrix $\mathbf{G}_N$	$\mathbf{F}^{\otimes n}$; $\mathbf{x} = \mathbf{u}\mathbf{G}_N$.
Information set $\mathcal{A}$	Indices carrying data (and CRC/PC bits).
Kernel $\mathbf{F}$	$\left[\begin{smallmatrix}1 & 0\\ 1 & 1\end{smallmatrix}\right]$, the basic polarization transform.
LLR	$\ln \frac{P(0\|y)}{P(1\|y)}$; sign = decision, magnitude = confidence.
List size L	Number of paths kept by an SCL decoder.
Mother code	The $N = 2^n$ polar code before rate matching.
Partial sums β	Re-encoded decisions passed up the SC tree.
Path metric	Cost of a decoding path; $-\ln$ of its probability.
PBCH	5G broadcast channel carrying the master information block.
PC bits	Parity-check bits used in NR for small UCI.
Polarization	Bit-channels becoming almost perfect or almost useless.
Polarization weight	SNR-independent reliability formula $\sum_j b_j 2^{j/4}$.
Puncturing	Not transmitting some coded bits (receiver LLR 0).
Rate matching	Adapting N coded bits to E transmitted bits.

Reliability sequence	Ordered list of indices from least to most reliable (NR: length 1024).
Repetition (rate matching)	Sending some coded bits more than once; LLRs add.
RNTI	Radio network temporary identifier; masks the DCI CRC.
SC	Successive cancellation decoding.
SCAN	Soft cancellation decoding with soft outputs.
SCL	Successive cancellation list decoding.
Shortening	Forcing coded bits to known zeros and not sending them (LLR $+\infty$).
Sub-block interleaver	NR permutation of 32 sub-blocks before bit selection.
Systematic code	Codeword contains the message bits unchanged.

Appendix C — Bibliography

Foundational papers

[1] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.

[2] E. Arıkan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, Jul. 2009.

[3] E. Arıkan and E. Telatar, "On the rate of channel polarization," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2009.

[4] S. B. Korada, E. Şaşoğlu and R. Urbanke, "Polar codes: Characterization of exponent, bounds, and constructions," *IEEE Trans. Inf. Theory*, vol. 56, no. 12, pp. 6253–6264, Dec. 2010.

[5] S. H. Hassani, K. Alishahi and R. Urbanke, "Finite-length scaling for polar codes," *IEEE Trans. Inf. Theory*, vol. 60, no. 10, pp. 5875–5898, Oct. 2014.

Decoding

[6] I. Tal and A. Vardy, "List decoding of polar codes," *IEEE Trans. Inf. Theory*, vol. 61, no. 5, pp. 2213–2226, May 2015.

[7] K. Niu and K. Chen, "CRC-aided decoding of polar codes," *IEEE Commun. Lett.*, vol. 16, no. 10, pp. 1668–1671, Oct. 2012.

[8] B. Li, H. Shen and D. Tse, "An adaptive successive cancellation list decoder for polar codes with cyclic redundancy check," *IEEE Commun. Lett.*, vol. 16, no. 12, pp. 2044–2047, Dec. 2012.

[9] A. Balatsoukas-Stimming, M. Bastani Parizi and A. Burg, "LLR-based successive cancellation list decoding of polar codes," *IEEE Trans. Signal Process.*, vol. 63, no. 19, pp. 5165–5179, Oct. 2015.

[10] A. Alamdar-Yazdi and F. R. Kschischang, "A simplified successive-cancellation decoder for polar codes," *IEEE Commun. Lett.*, vol. 15, no. 12, pp. 1378–1380, Dec. 2011.

[11] G. Sarkis, P. Giard, A. Vardy, C. Thibeault and W. J. Gross, "Fast polar decoders: Algorithm and implementation," *IEEE J. Sel. Areas Commun.*, vol. 32, no. 5, pp. 946–957, May 2014.

[12] U. U. Fayyaz and J. R. Barry, "Low-complexity soft-output decoding of polar codes," *IEEE J. Sel. Areas Commun.*, vol. 32, no. 5, pp. 958–966, May 2014.

[13] S. A. Hashemi, C. Condo and W. J. Gross, "Fast and flexible successive-cancellation list decoders for polar codes," *IEEE Trans. Signal Process.*, vol. 65, no. 21, pp. 5756–5769, Nov. 2017.

[14] C. Leroux, A. J. Raymond, G. Sarkis and W. J. Gross, "A semi-parallel successive-cancellation decoder for polar codes," *IEEE Trans. Signal Process.*, vol. 61, no. 2, pp. 289–299, Jan. 2013.

Encoding and construction

[15] E. Arıkan, "Systematic polar coding," *IEEE Commun. Lett.*, vol. 15, no. 8, pp. 860–862, Aug. 2011.

[16] R. Mori and T. Tanaka, "Performance of polar codes with the construction using density evolution," *IEEE Commun. Lett.*, vol. 13, no. 7, pp. 519–521, Jul. 2009.

[17] I. Tal and A. Vardy, "How to construct polar codes," *IEEE Trans. Inf. Theory*, vol. 59, no. 10, pp. 6562–6582, Oct. 2013.

[18] P. Trifonov, "Efficient design and decoding of polar codes," *IEEE Trans. Commun.*, vol. 60, no. 11, pp. 3221–3227, Nov. 2012.

[19] S.-Y. Chung, T. J. Richardson and R. L. Urbanke, "Analysis of sum-product decoding of low-density parity-check codes using a Gaussian approximation," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 657–670, Feb. 2001.

[20] G. He *et al.*, "β-expansion: A theoretical framework for fast and recursive construction of polar codes," in *Proc. IEEE GLOBECOM*, 2017.

[21] E. Arıkan, "From sequential decoding to channel polarization and back again," arXiv:1908.09594, 2019.

5G NR

[22] 3GPP TS 38.212, "NR; Multiplexing and channel coding," Release 15 and later.

[23] 3GPP TS 38.211, "NR; Physical channels and modulation," Release 15 and later.

[24] 3GPP TS 38.213, "NR; Physical layer procedures for control," Release 15 and later.

[25] V. Bioglio, C. Condo and I. Land, "Design of polar codes in 5G New Radio," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 29–40, 2021.

Textbooks

[26] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[27] T. Richardson and R. Urbanke, *Modern Coding Theory*. Cambridge University Press, 2008.

[28] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed. Pearson, 2004.

Appendix D — MATLAB Implementation

IN THIS CHAPTER YOU WILL LEARN

- a complete MATLAB polar-code library (11 short files);
- how each Python program of Chapter 11 maps to MATLAB;
- the 1-based indexing rule for bit-channel numbers;
- how the vectorised SC/SCL/CA-SCL decoder works;
- a demonstration script with its printed output;
- BER/BLER scripts and the final project in MATLAB;
- how the code was verified against the Python library.

D.1 Overview and requirements

This appendix gives a MATLAB version of every program of Chapters 11 and 15. The book's mathematics, conventions and worked examples are unchanged; only the language changes. Copy all `.m` files into one folder, make it the current folder (or add it to the path), and run `demo_polar`.

MATLAB file	Purpose	Python equivalent
`polar_encode.m`	butterfly encoder $\mathbf{x} = \mathbf{u}\mathbf{G}_N$ (vector or batch)	`polar_encode`
`generator_matrix.m`	$\mathbf{G}_N = \mathbf{F}^{\otimes n}$ (for testing)	`generator_matrix`
`bec_bhattacharyya.m`	exact BEC reliabilities	`bec_bhattacharyya`
`ga_construct.m`	Gaussian-approximation construction	`ga_construct`
`polar_list_decode.m`	SC ($L = 1$), SCL and CA-SCL decoder	`ListDecoder`
`take_paths.m`	helper: re-order list paths	`take_along_axis`
`crc_bits.m`	CRC by long division	`crc_bits`
`crc_make.m, crc_attach.m, crc_check.m`	CRC6/11/24C as a linear map	`class CRC`
`polar_simulate.m`	Monte-Carlo BER/BLER	`simulate`
`demo_polar.m`	demonstrations with printed output	`demo_ch11.py`
`plot_ber_bler.m, compare_lengths_rates.m`	figures of Chapters 7, 8, 9, 11	`run_sims.py`
`final_project.m`	end-to-end project of Chapter 15	`final_project.py`
`test_polar_matlab.m`	self-tests and comparison with Python	`test_polar.py`

Requirements

MATLAB R2016b or newer (the decoder uses implicit expansion). No toolbox is needed: the code uses only core functions (`sort`, `reshape`, `xor`, `interp1`, `trapz`, `erfc`, `rng`). Everything was also tested with the free GNU Octave 8.4, which produced all outputs printed in this appendix.

D.2 Conventions and the 1-based index rule

All conventions of the book are kept: $\mathbf{F} = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$, $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ without bit reversal, $\mathbf{x} = \mathbf{u}\mathbf{G}_N$, BPSK $0 \mapsto +1$, $L = \ln \frac{P(0)}{P(1)}$, min-sum f and the approximate path metric.

! Index i in the book is element $i+1$ in MATLAB

The book numbers bits $u_0, \ldots, u_{N-1}$. MATLAB arrays start at 1, so u_i is stored in `u(i+1)`. The information set $\mathcal{A} = \{3, 5, 6, 7\}$ of the $(8, 4)$ code is therefore the logical mask `info = logical([0 0 0 1 0 1 1 1])`. When you print an information set as book indices, subtract one: `find(info) - 1`.

Python	MATLAB
`u[:, info] = payload`	`u(:, info) = payload;`
`x ^= y`	`x = xor(x, y);`
`rng = np.random.default_rng(7)`	`rng(7);`
`rng.standard_normal((B, N))`	`randn(B, N)`
`np.take_along_axis(a, p[:, :, None], 1)`	`take_paths(a, p)`
arrays `(B, L, n)`, 0-based	arrays `B×L×n`, 1-based

D.3 Program 1 — Polar encoder

polar_encode.m

```
function x = polar_encode(u)
%POLAR_ENCODE  Butterfly polar encoder, x = u*G_N (mod 2), G_N = F^(kron n).
%   u : 1-by-N vector or B-by-N matrix (one block per row) of 0/1 values.
%   x : same size as u.
%   Convention (as in 3GPP NR): F = [1 0; 1 1], natural order, no bit reversal.
%   Each stage XORs the lower half of every block into the upper half.
x = logical(u);
[B, N] = size(x);
h = N/2;
while h >= 1
    x = reshape(x.', 2*h, []);            % each column = one block of 2h bits
    x(1:h, :) = xor(x(1:h, :), x(h+1:2*h, :));
    x = reshape(x, N, B).';
    h = h/2;
end
x = double(x);
```

```
end
```

How it works. Each column of the reshaped array is one block of $2h$ bits; the line `x(1:h,:) = xor(...)` is the butterfly stage "upper half ← upper half ⊕ lower half" of Chapter 5, applied to all blocks of all B rows at once. The spans are $N/2, N/4, \ldots, 1$, giving $\frac{N}{2}\log_2 N$ XORs per codeword.

generator_matrix.m

```
function G = generator_matrix(N)
%GENERATOR_MATRIX  G_N = F^(kron n) with F = [1 0; 1 1]  (for testing).
F = [1 0; 1 1];
G = 1;
while size(G,1) < N
    G = mod(kron(F, G), 2);
end
end
```

D.4 Program 2 — Frozen-bit selection

bec_bhattacharyya.m

```
function z = bec_bhattacharyya(N, eps)
%BEC_BHATTACHARYYA  Exact Bhattacharyya parameters (erasure probabilities)
%   of the N bit-channels of a BEC(eps), natural index order (index i -> z(i+1)).
%       minus (worse)  channel:  Z -> 2Z - Z^2
%       plus  (better) channel:  Z -> Z^2
if N == 1
    z = eps;
    return
end
z = [bec_bhattacharyya(N/2, 2*eps - eps^2), bec_bhattacharyya(N/2, eps^2)];
end
```

The first half of the output is the minus branch and the second half the plus branch, so `z(i+1)` is $Z(W_N^{(i)})$ with the binary-index rule of Chapter 4.

ga_construct.m

```
function [info, m] = ga_construct(N, K, design_ebn0_db, rate)
%GA_CONSTRUCT  Gaussian-approximation construction for BPSK over AWGN.
%   info : 1-by-N logical, true at the K most reliable positions.
%   m    : mean LLR of every bit-channel (larger = more reliable).
%   rate : rate used to convert Eb/N0 to Es/N0 (default K/N).
%       plus  child:  m+ = 2m
%       minus child:  m- = phi^-1( 1 - (1 - phi(m))^2 )
if nargin < 3, design_ebn0_db = 2.0; end
if nargin < 4, rate = K/N; end
esn0 = 10^(design_ebn0_db/10) * rate;
m = ga_rec(N, 4*esn0);                  % channel LLR mean = 2/sigma^2
[~, order] = sort(m);
info = false(1, N);
```

```
info(order(end-K+1:end)) = true;
end

function m = ga_rec(N, m0)
% Level by level: each mean m becomes the pair (m-, m+). Interleaving the
% pairs gives natural index order (first split = most significant bit).
m = m0;
while numel(m) < N
    m = reshape([ga_minus(m); 2*m], 1, []);
end
end

function mm = ga_minus(m)
% Solve log(phi(mm)) = log(2p - p^2), p = phi(m), by bisection in the log
% domain (vectorised over all means of one level).
lp = log_phi(m);
p = exp(lp);
target = lp + log(2 - p);
lo = zeros(size(m)); hi = m;
for it = 1:80
    mid = (lo + hi)/2;
    up = log_phi(mid) > target;
    lo(up) = mid(up);
    hi(~up) = mid(~up);
end
mm = (lo + hi)/2;
end

function lp = log_phi(x)
% log of phi(x) = 1 - E[tanh(U/2)], U ~ N(x, 2x)   (vectorised).
%   x < 10 : exact value from a table built once by numerical integration
%   x >= 10: asymptotic form  sqrt(pi/x) exp(-x/4) (1 - 10/(7x))
persistent lxg lt
if isempty(lxg)
    xg = logspace(-8, 1, 4000);
    t = linspace(-12, 12, 4001);          % standard normal grid
    w = exp(-t.^2/2) / sqrt(2*pi);
    lt = zeros(size(xg));
    for k = 1:numel(xg)
        u = xg(k) + sqrt(2*xg(k)) * t;
        lt(k) = log(max(1 - trapz(t, w .* tanh(u/2)), 1e-300));
    end
    lxg = log(xg);
end
lp = zeros(size(x));
s = x <= 1e-8;            lp(s) = log1p(-x(s)/2);
s = x > 1e-8 & x < 10;    lp(s) = interp1(lxg, lt, log(x(s)));
s = x >= 10;              lp(s) = 0.5*log(pi./x(s)) - x(s)/4 + log(1 - 10./(7*x(s)));
end
```

How it works. `ga_rec` builds the tree level by level: each mean m becomes the pair (m^-, m^+) and the pairs are interleaved, which gives the natural index order (the first split is the most significant bit of the index). All m^- of one level are found together by a vectorised bisection on $\ln \phi$. The function ϕ is evaluated exactly (numerical integration, stored once in a `persistent` table) for

$x < 10$ and by the asymptotic formula above; working with logarithms avoids the two numerical traps described in Chapter 9.

D.5 Programs 3, 4 and 5 — SC, SCL and CA-SCL decoder

polar_list_decode.m

```
function [u_hat, u_list, PM] = polar_list_decode(llr, info, L, crc)
%POLAR_LIST_DECODE  LLR-based SC / SCL / CRC-aided SCL polar decoder.
%   llr  : B-by-N channel LLRs (one block per row), L = ln P(0)/P(1).
%   info : 1-by-N logical information set (frozen bits are 0).
%   L    : list size. L = 1 gives the plain SC decoder.
%   crc  : (optional) struct from crc_make(); enables CA-SCL selection.
%   u_hat  : B-by-N decided u vectors.
%   u_list : B-by-L-by-N final list, sorted by path metric (best first).
%   PM     : B-by-L path metrics (min-sum form, smaller = better).
%
%   Path metric update (Balatsoukas-Stimming et al., 2015):
%       PM <- PM + |LLR|  if the decision disagrees with sign(LLR).
if nargin < 3, L = 1; end
if nargin < 4, crc = []; end
[B, N] = size(llr);
st.info = info;
st.PM = inf(B, L);
st.PM(:, 1) = 0;                                % one active path at start
lam = repmat(reshape(llr, B, 1, N), 1, L, 1);     % B-by-L-by-N
[~, u_list, ~, st] = node(lam, 1, st);
PM = st.PM;
if isempty(crc)
    u_hat = reshape(u_list(:, 1, :), B, N);       % best path metric
else
    payload = u_list(:, :, info);                 % B-by-L-by-K
    ok = reshape(crc_check(crc, reshape(payload, B*L, [])), B, L) & isfinite(PM);
    pick = ones(B, 1);
    for b = 1:B
        f = find(ok(b, :), 1);
        if ~isempty(f), pick(b) = f; end
    end
    idx = sub2ind([B, L], (1:B).', pick);
    U = reshape(u_list, B*L, N);
    u_hat = U(idx, :);
end
end

% ------------------------------------------------------------------ tree node
function [c, u, parent, st] = node(lam, start, st)
% lam: B-by-L-by-n LLRs of this node; start: index of its first leaf (1-based)
n = size(lam, 3);
if n == 1
    [c, u, parent, st] = leaf(lam, start, st);
    return
end
h = n/2;
```

```
a = lam(:, :, 1:h);  b = lam(:, :, h+1:n);
% left child: f (min-sum)
[c1, u1, p1, st] = node(sign(a).*sign(b).*min(abs(a), abs(b)), start, st);
% right child: g, with the surviving parents' LLRs
a1 = take_paths(a, p1);  b1 = take_paths(b, p1);
[c2, u2, p2, st] = node(b1 + (1 - 2*c1).*a1, start + h, st);
% re-order left results by the right child's survivors
c1 = take_paths(c1, p2);
u1 = take_paths(u1, p2);
parent = p1(sub2ind(size(p1), repmat((1:size(p1,1)).', 1, size(p1,2)), p2));
c = cat(3, double(xor(c1, c2)), c2);      % partial sums [c1 xor c2 | c2]
u = cat(3, u1, u2);
end

% ------------------------------------------------------------- leaf
function [c, u, parent, st] = leaf(lam, i, st)
llr = lam(:, :, 1);                       % B-by-L
[B, L] = size(llr);
pen0 = (llr < 0) .* (-llr);               % cost of deciding 0
pen1 = (llr > 0) .* llr;                  % cost of deciding 1
if ~st.info(i)                            % frozen bit: always 0
    st.PM = st.PM + pen0;
    c = zeros(B, L);
    parent = repmat(1:L, B, 1);
else                                      % information bit: split and prune
    cand = [st.PM + pen0, st.PM + pen1];  % B-by-2L
    [srt, ord] = sort(cand, 2);           % stable sort
    ord = ord(:, 1:L);
    st.PM = srt(:, 1:L);
    parent = mod(ord - 1, L) + 1;
    c = double(ord > L);
end
u = c;
end
```

take_paths.m

```
function A = take_paths(A, p)
%TAKE_PATHS  Re-order the paths (2nd dimension) of a B-by-L-by-n array:
%   A(b, l, :) <- A(b, p(b,l), :)   for every block b and path l.
[B, L, n] = size(A);
base = repmat((1:B).', 1, L) + (p - 1) * B;          % B-by-L linear index
idx = base + reshape((0:n-1) * B * L, 1, 1, n);      % implicit expansion
A = A(idx);
end
```

D.5.1 How the decoder works

- **Shapes.** LLRs are stored in a $B \times L \times n$ array: B received blocks (rows), L list paths, $n =$ size of the current tree node. One call decodes a whole batch of blocks.
- **Tree.** `node` is the recursion of Chapter 6: f (min-sum) to the left child, g to the right child, partial sums $[\beta_L \oplus \beta_R \,|\, \beta_R]$ returned up.

- **Leaves.** A frozen leaf adds the penalty of deciding 0. An information leaf builds $2L$ candidates, sorts them (`sort` is stable) and keeps the L smallest metrics (Chapter 7).
- **Moving paths.** After pruning, path l may come from path `parent(b,l)`. Every node returns this map; the parent node uses `take_paths` to fetch the right LLRs for g and to re-order the left child's results. No explicit memory copying is needed.
- **CA-SCL.** After the last leaf the list is sorted by metric; the first path whose payload passes `crc_check` is returned (Chapter 8). With `L = 1` and no CRC the function is the plain SC decoder.

D.6 CRC functions

crc_bits.m

```
function p = crc_bits(msg, exps)
%CRC_BITS  CRC remainder of msg(D)*D^r divided by g(D), by long division.
%   exps: exponents of g(D) with coefficient 1, highest first, e.g. [3 1 0].
r = exps(1);
g = zeros(1, r+1);
g(r - exps + 1) = 1;                        % g(1) is the D^r coefficient
reg = [msg(:).' zeros(1, r)];
for i = 1:numel(msg)
    if reg(i)
        reg(i:i+r) = xor(reg(i:i+r), g);
    end
end
p = reg(end-r+1:end);
end
```

crc_make.m

```
function crc = crc_make(name, k)
%CRC_MAKE  Build a CRC as a linear map: crc bits = mod(m * P, 2).
%   name : 'CRC6', 'CRC11' or 'CRC24C' (3GPP TS 38.212, clause 5.1)
%   k    : message length in bits.
switch upper(name)
    case 'CRC6',   e = [6 5 0];
    case 'CRC11',  e = [11 10 9 5 0];
    case 'CRC24C', e = [24 23 21 20 17 15 13 12 8 4 2 1 0];
    otherwise, error('unknown CRC');
end
crc.r = e(1);
crc.k = k;
crc.exps = e;
crc.P = zeros(k, crc.r);
for i = 1:k
    unit = zeros(1, k);  unit(i) = 1;
    crc.P(i, :) = crc_bits(unit, e);
end
end
```

crc_attach.m

```
function w = crc_attach(crc, m)
%CRC_ATTACH  Append CRC bits to every row
      of m:  w = [m | mod(m*P,2)].
w = [m, mod(m * crc.P, 2)];
end
```

crc_check.m

```
function ok = crc_check(crc, w)
%CRC_CHECK  True for every row of w = [m
     | p] whose CRC is correct.
m = w(:, 1:crc.k);
p = w(:, crc.k+1:end);
ok = all(mod(m * crc.P, 2) == p, 2);
end
```

Because the CRC is linear, `crc_make` stores the matrix $\mathbf{P}$ whose row i is the CRC of the i-th unit vector. Then the CRC of any batch of messages is one product, `mod(m*P, 2)`.

D.7 Programs 6, 7 and 8 — BPSK, AWGN and BER/BLER

polar_simulate.m

```
function res = polar_simulate(N, K, ebn0_list, L, crc_name, varargin)
%POLAR_SIMULATE  Monte-Carlo BER/BLER of a polar code, BPSK over AWGN.
%   K counts all bits on information positions (message + CRC).
%   Eb/N0 is per MESSAGE bit: rate = (K - r)/N.
%   Options (name/value): 'design', 2.0 | 'max_frames', 2e4 |
%                         'min_errors', 200 | 'batch', 500 | 'seed', 1
if nargin < 4 || isempty(L), L = 1; end
if nargin < 5, crc_name = ''; end
opt = struct('design', 2.0, 'max_frames', 2e4, 'min_errors', 200, ...
             'batch', 500, 'seed', 1);
for i = 1:2:numel(varargin), opt.(varargin{i}) = varargin{i+1}; end
rng(opt.seed);
crc = [];  r = 0;
if ~isempty(crc_name)
    r = crc_make(crc_name, 1).r;          % CRC length
    crc = crc_make(crc_name, K - r);
end
k = K - r;                                % message bits
rate = k / N;
info = ga_construct(N, K, opt.design, K/N);
res.ebn0 = ebn0_list;  res.ber = zeros(size(ebn0_list));
res.bler = res.ber;    res.errors = res.ber;
for s = 1:numel(ebn0_list)
    esn0 = 10^(ebn0_list(s)/10) * rate;
    sigma = sqrt(1/(2*esn0));
    be = 0; bl = 0; fr = 0;
    while fr < opt.max_frames && bl < opt.min_errors
        B = opt.batch;
        m = double(rand(B, k) < 0.5);
        if isempty(crc), pay = m; else, pay = crc_attach(crc, m); end
        u = zeros(B, N);  u(:, info) = pay;
        x = polar_encode(u);
        y = (1 - 2*x) + sigma * randn(B, N);          % BPSK + AWGN
        llr = 2*y / sigma^2;
```

```
        u_hat = polar_list_decode(llr, info, L, crc);
        m_hat = u_hat(:, info);  m_hat = m_hat(:, 1:k);
        e = (m_hat ~= m);
        be = be + sum(e(:));
        bl = bl + sum(any(e, 2));
        fr = fr + B;
    end
    res.ber(s) = be / (fr * k);
    res.bler(s) = bl / fr;
    res.errors(s) = bl;
    fprintf('  Eb/N0 = %4.1f dB   BER = %.2e   BLER = %.2e   (%d errors / %d frames)\n', ...
            ebn0_list(s), res.ber(s), res.bler(s), bl, fr);
end
end
```

BPSK and AWGN are the two lines `y = (1 - 2*x) + sigma*randn(B, N)` and `llr = 2*y/sigma^2`, with $\sigma = \sqrt{1/(2E_s/N_0)}$ and $E_s/N_0 = R\,E_b/N_0$ computed with the *message* rate (Chapter 2). Errors are counted on message bits only.

D.8 Demonstration with outputs

demo_polar.m

```
%DEMO_POLAR  Small demonstrations with printed outputs (MATLAB version of
%            demo_ch11.py). Run from the folder that holds the library.
disp('1) Encoder ------------------------------------------');
info = logical([0 0 0 1 0 1 1 1]);            % A = {3,5,6,7} (0-based)
u = zeros(1, 8);  u(info) = [1 0 1 1];
x = polar_encode(u);
fprintf('u = %s   x = %s\n', mat2str(u), mat2str(x));
fprintf('matrix check: %d\n', isequal(x, mod(u * generator_matrix(8), 2)));

disp('2) Frozen-bit selection ----------------------------');
fprintf('BEC(0.5) Z, N=8 : %s\n', mat2str(round(1e4*bec_bhattacharyya(8, 0.5))/1e4));
[~, o] = sort(bec_bhattacharyya(8, 0.5));
fprintf('BEC info set K=4 (0-based): %s\n', mat2str(sort(o(1:4)) - 1));
fprintf('GA info set N=16, K=8 @2dB (0-based): %s\n', mat2str(find(ga_construct(16, 8, 2.0)
    ) - 1));

disp('3) SC decoder --------------------------------------');
llr1 = [-1.5 2.0 -0.8 1.2 -0.6 -2.5 1.8 -3.0];
fprintf('SC  u_hat = %s\n', mat2str(polar_list_decode(llr1, info, 1)));

disp('4) SCL decoder -------------------------------------');
llr2 = [-2.0 0.3 -0.4 -2.0 1.0 -5.8 3.2 -4.8];
fprintf('SC  u_hat = %s (wrong)\n', mat2str(polar_list_decode(llr2, info, 1)));
[uh, ~, PM] = polar_list_decode(llr2, info, 2);
fprintf('SCL u_hat = %s   path metrics: %s\n', mat2str(uh), mat2str(PM));

disp('5) CRC ---------------------------------------------');
crc = crc_make('CRC6', 10);
```

```
fprintf('message + CRC6 : %s\n', mat2str(crc_attach(crc, [1 0 1 1 0 0 1 0 1 1])));

disp('6-7) BPSK + AWGN ------------------------------------');
rng(0);
esn0 = 10^(2.0/10) * 0.5;  sigma = sqrt(1/(2*esn0));
y = (1 - 2*x) + sigma * randn(1, 8);
fprintf('sigma = %.4f\n', sigma);
fprintf('y     = %s\n', mat2str(round(100*y)/100));
fprintf('LLR   = %s\n', mat2str(round(200*y/sigma^2)/100));

disp('8) BER / BLER ---------------------------------------');
res = polar_simulate(128, 64, [1 2 3], 1, '', 'max_frames', 2e4, ...
                     'min_errors', 100, 'batch', 1000, 'seed', 7);
```

```
OUTPUT
1) Encoder ------------------------------------------
u = [0 0 0 1 0 0 1 1]   x = [1 0 1 0 0 1 0 1]
matrix check: 1
2) Frozen-bit selection -----------------------------
BEC(0.5) Z, N=8 : [0.9961 0.8789 0.8086 0.3164 0.6836 0.1914 0.1211 0.0039]
BEC info set K=4 (0-based): [3 5 6 7]
GA info set N=16, K=8 @2dB (0-based): [7 9 10 11 12 13 14 15]
3) SC decoder ---------------------------------------
SC  u_hat = [0 0 0 1 0 0 1 1]
4) SCL decoder --------------------------------------
SC  u_hat = [0 0 0 0 0 0 1 1] (wrong)
SCL u_hat = [0 0 0 1 0 0 1 1]   path metrics: [2 2.7]
5) CRC ----------------------------------------------
message + CRC6 : [1 0 1 1 0 0 1 0 1 1 0 1 1 0 1 1]
6-7) BPSK + AWGN ------------------------------------
sigma = 0.7943
y     = [-1.97 1.61 -1.33 1.4 1.46 -0.57 0.52 -1.03]
LLR   = [-6.25 5.09 -4.22 4.44 4.62 -1.81 1.65 -3.28]
8) BER / BLER ---------------------------------------
Eb/N0 =  1.0 dB   BER = 1.36e-01   BLER = 4.49e-01   (449 errors / 1000 frames)
Eb/N0 =  2.0 dB   BER = 3.33e-02   BLER = 1.31e-01   (131 errors / 1000 frames)
Eb/N0 =  3.0 dB   BER = 5.65e-03   BLER = 2.60e-02   (104 errors / 4000 frames)
```

Parts 1–5 are exactly the numbers of the book (Worked Examples 5.3, 6.3, 7.1, 7.2 and 8.2). Parts 6–8 use random numbers: MATLAB and Octave generate different streams for the same seed, so your values will differ slightly, but the BLER must be close to the Python result of Chapter 11 (1.5×10^{-1} at 2 dB, 3.1×10^{-2} at 3 dB).

D.9 Programs 9 and 10 — BER/BLER figures, block lengths and rates

plot_ber_bler.m

```
%PLOT_BER_BLER  BER/BLER versus Eb/N0 for SC, SCL and CA-SCL (N=256, R=1/2).
%   Takes a few minutes in MATLAB; reduce 'max_frames' for a quick look.
N = 256;  E = 0.5:0.5:3.5;
opts = {'max_frames', 5e4, 'min_errors', 150, 'batch', 1000, 'seed', 2026};
disp('SC');            sc  = polar_simulate(N, 128, E, 1, '', opts{:});
```

```
disp('SCL L=8');       scl = polar_simulate(N, 128, E, 8, '', opts{:});
disp('CA-SCL L=8');    ca  = polar_simulate(N, 139, E, 8, 'CRC11', opts{:});
uncoded = 0.5 * erfc(sqrt(10.^(E/10)));                % Q(sqrt(2 Eb/N0))

figure('Position', [100 100 950 380]);
subplot(1, 2, 1);
semilogy(E, sc.bler, 'o-', E, scl.bler, 's-', E, ca.bler, '^-', 'LineWidth', 1.6);
grid on; xlabel('E_b/N_0 (dB)'); ylabel('BLER');
legend('SC', 'SCL, L=8', 'CA-SCL, L=8, CRC-11', 'Location', 'southwest');
title('N = 256, R = 1/2');
subplot(1, 2, 2);
semilogy(E, sc.ber, 'o-', E, scl.ber, 's-', E, ca.ber, '^-', E, uncoded, 'k:', 'LineWidth',
    1.6);
grid on; xlabel('E_b/N_0 (dB)'); ylabel('BER');
legend('SC', 'SCL, L=8', 'CA-SCL, L=8', 'uncoded BPSK', 'Location', 'southwest');
title('N = 256, R = 1/2');
```

Expected result. The curves of Fig. D.1; the BLER values agree with the table in Chapter 7 within Monte-Carlo accuracy.

compare_lengths_rates.m

```
%COMPARE_LENGTHS_RATES  SC decoding: effect of block length and code rate.
opts = {'max_frames', 5e4, 'min_errors', 150, 'batch', 1000, 'seed', 2026};
figure('Position', [100 100 950 380]);
subplot(1, 2, 1); hold on;
E = 0:0.5:4.5;
for N = [64 128 256 512 1024]
    fprintf('N = %d\n', N);
    r = polar_simulate(N, N/2, E, 1, '', opts{:});
    ok = r.errors >= 20;                       % plot reliable points only
    semilogy(E(ok), r.bler(ok), 'o-', 'LineWidth', 1.5, 'DisplayName', sprintf('N = %d', N)
    );
end
set(gca, 'YScale', 'log'); grid on; legend('Location', 'southwest');
xlabel('E_b/N_0 (dB)'); ylabel('BLER'); title('SC, R = 1/2');
subplot(1, 2, 2); hold on;
E = 0:0.5:5.5;
for K = [64 128 192]
    fprintf('K = %d\n', K);
    r = polar_simulate(256, K, E, 1, '', opts{:});
    ok = r.errors >= 20;
    semilogy(E(ok), r.bler(ok), 'o-', 'LineWidth', 1.5, 'DisplayName', sprintf('R = %d/256'
    , K));
end
set(gca, 'YScale', 'log'); grid on; legend('Location', 'southwest');
xlabel('E_b/N_0 (dB)'); ylabel('BLER'); title('SC, N = 256');
```

D.10 Final project in MATLAB

final_project.m

```
%FINAL_PROJECT  End-to-end polar-coded control link (MATLAB version of
%   final_project.py): A = 40 bits + CRC24C (K = 64), N = 128, BPSK/AWGN.
%   Receivers: SC (CRC used only for detection) and CA-SCL with L = 8.
%   Measures BER, BLER and undetected errors (wrong message passing the CRC).
N = 128;  A = 40;
crc  = crc_make('CRC24C', A);
K    = A + crc.r;                                % 64 payload bits
R    = A / N;                                    % message rate 0.3125
info = ga_construct(N, K, 2.0, K/N);
names = {'SC', 'CA-SCL L=8'};  Ls = [1 8];  crcs = {[], crc};

% ------------------------------------------------------------ one frame
rng(2026);
m   = double(rand(1, A) < 0.5);                  % step 1: data
pay = crc_attach(crc, m);                        % step 2: CRC
u   = zeros(1, N);  u(info) = pay;               % step 3: map + encode
x   = polar_encode(u);
eb  = 4.5;  sigma = sqrt(1/(2*R*10^(eb/10)));
y   = (1 - 2*x) + sigma*randn(1, N);             % steps 4-5: BPSK + AWGN
llr = 2*y/sigma^2;
fprintf('hard-decision errors: %d of %d\n', sum((llr < 0) ~= (x == 1)), N);
for d = 1:2                                      % step 6: decode
    uh = polar_list_decode(llr, info, Ls(d), crcs{d});
    fprintf('%-11s: bit errors = %d, CRC pass = %d\n', names{d}, ...
            sum(uh(find(info, A)) ~= m), crc_check(crc, uh(info)));
end

% ------------------------------------------------------------ Monte-Carlo
E = 2:0.5:5.5;  B = 2000;
BLER = zeros(2, numel(E));  BER = BLER;  UND = BLER;
for d = 1:2
    fprintf('%s\n', names{d});
    rng(1);
    for s = 1:numel(E)
        sigma = sqrt(1/(2*R*10^(E(s)/10)));
        be = 0; bl = 0; und = 0; fr = 0;
        while fr < 1e5 && bl < 150
            m = double(rand(B, A) < 0.5);
            u = zeros(B, N);  u(:, info) = crc_attach(crc, m);
            llr = 2*((1 - 2*polar_encode(u)) + sigma*randn(B, N))/sigma^2;
            ph  = polar_list_decode(llr, info, Ls(d), crcs{d});
            ph  = ph(:, info);
            wrong = any(ph(:, 1:A) ~= m, 2);
            be  = be + sum(sum(ph(:, 1:A) ~= m));
            bl  = bl + sum(wrong);
            und = und + sum(wrong & crc_check(crc, ph));   % step 7
            fr  = fr + B;
        end
        BER(d, s) = be/(fr*A);  BLER(d, s) = bl/fr;  UND(d, s) = und;
        fprintf('  %4.1f dB  BER %.2e  BLER %.2e  undetected %d/%d\n', ...
```

```
                E(s), BER(d, s), BLER(d, s), und, fr);
    end
end
figure;
semilogy(E, BLER(1, :), 'o-', E, BLER(2, :), 's-', 'LineWidth', 1.6); grid on;
hold on; semilogy(E([1 end]), [1e-2 1e-2], 'k:');      % target BLER 10^-2
xlabel('E_b/N_0 (dB), message rate 40/128'); ylabel('BLER');
legend([names, {'target 10^{-2}'}], 'Location', 'southwest');
```

Running `final_project` in GNU Octave 8.4 printed:

```
OUTPUT
hard-decision errors: 14 of 128
SC         : bit errors = 0, CRC pass = 1
CA-SCL L=8 : bit errors = 0, CRC pass = 1
SC
3.0 dB  BER 1.21e-01  BLER 4.12e-01  undetected 0/2000
4.0 dB  BER 3.55e-02  BLER 1.42e-01  undetected 0/2000
5.0 dB  BER 5.22e-03  BLER 2.60e-02  undetected 0/6000
5.5 dB  BER 1.56e-03  BLER 9.62e-03  undetected 0/16000
CA-SCL L=8
3.0 dB  BER 3.05e-02  BLER 9.80e-02  undetected 0/2000
4.0 dB  BER 2.56e-03  BLER 9.22e-03  undetected 0/18000
4.5 dB  BER 3.72e-04  BLER 1.58e-03  undetected 0/96000
5.0 dB  BER 3.40e-05  BLER 1.40e-04  undetected 0/100000
```

(Only some of the E_b/N_0 points are shown.) The one printed frame is different from Chapter 15 because the random stream is different, but the statistics agree with the Python results of Chapter 15 within Monte-Carlo accuracy:

BLER at $E_b/N_0 =$	3.0 dB	4.0 dB	4.5 dB	5.0 dB
SC, Python (Ch. 15)	4.3×10^{-1}	1.5×10^{-1}	6.4×10^{-2}	2.6×10^{-2}
SC, MATLAB	4.1×10^{-1}	1.4×10^{-1}	6.2×10^{-2}	2.6×10^{-2}
CA-SCL $L = 8$, Python	9.7×10^{-2}	8.3×10^{-3}	1.5×10^{-3}	2.1×10^{-4}
CA-SCL $L = 8$, MATLAB	9.8×10^{-2}	9.2×10^{-3}	1.6×10^{-3}	1.4×10^{-4}

At 5.0 dB the CA-SCL values rest on only 21 (Python) and 14 (MATLAB) block errors, so their difference is within the statistical uncertainty. No undetected error occurred in either implementation.

D.11 Self-test script

test_polar_matlab.m

```
%TEST_POLAR_MATLAB  Self-tests of the MATLAB polar library.
% 1) encoder equals u*G_N
for N = [2 4 8 16 64 256]
    u = double(rand(50, N) < 0.5);
    assert(isequal(polar_encode(u), mod(u * generator_matrix(N), 2)));
end
```

```
disp('encoder ok');
% 2) worked examples of the book
info = logical([0 0 0 1 0 1 1 1]);
assert(isequal(polar_encode([0 0 0 1 0 0 1 1]), [1 0 1 0 0 1 0 1]));
assert(isequal(polar_list_decode([-1.5 2.0 -0.8 1.2 -0.6 -2.5 1.8 -3.0], info, 1), [0 0 0 1
    0 0 1 1]));
llr2 = [-2.0 0.3 -0.4 -2.0 1.0 -5.8 3.2 -4.8];
assert(isequal(polar_list_decode(llr2, info, 1), [0 0 0 0 0 0 1 1]));
[uh, ~, PM] = polar_list_decode(llr2, info, 2);
assert(isequal(uh, [0 0 0 1 0 0 1 1]) && max(abs(PM - [2.0 2.7])) < 1e-12);
disp('book examples ok');
% 3) CRC
assert(isequal(crc_bits([1 1 0 1], [3 1 0]), [0 0 1]));
c = crc_make('CRC6', 10);
w = crc_attach(c, [1 0 1 1 0 0 1 0 1 1]);
assert(isequal(w(11:16), [0 1 1 0 1 1]) && crc_check(c, w));
w(3) = 1 - w(3);  assert(~crc_check(c, w));
disp('crc ok');
% 4) identical decisions to the Python library on random noisy blocks
%    (pyref.mat holds LLRs and decisions produced by the Python code)
R = load('pyref.mat');
tags = {'N64L1', 'N64L4', 'N128L8c', 'N256L8'};
Ls = [1 4 8 8];
for t = 1:numel(tags)
    llr = R.([tags{t} '_llr']);  info = logical(R.([tags{t} '_info']));
    if tags{t}(end) == 'c'
        crc = crc_make('CRC11', sum(info) - 11);
    else
        crc = [];
    end
    uh = polar_list_decode(llr, info, Ls(t), crc);
    fprintf('%-8s same decisions as Python: %d of %d blocks\n', tags{t}, ...
            sum(all(uh == R.([tags{t} '_uh']), 2)), size(llr, 1));
end
% 5) GA construction equals Python
[info, m] = ga_construct(1024, 512, 2.0, 0.5);
p = R.ga1024;  big = p > 1e-3;          % ignore near-zero means (useless channels)
[~, o] = sort(p);  info_py = false(1, 1024);  info_py(o(end-511:end)) = true;
fprintf('GA: max relative difference %.1e, same information set: %d\n', ...
        max(abs(m(big) - p(big)) ./ p(big)), isequal(info, info_py));
```

```
OUTPUT
encoder ok
book examples ok
crc ok
N64L1    same decisions as Python:  300 of 300 blocks
N64L4    same decisions as Python:  300 of 300 blocks
N128L8c  same decisions as Python:  300 of 300 blocks
N256L8   same decisions as Python:  300 of 300 blocks
GA: max relative difference 7.3e-07, same information set:  1
```

The file `pyref.mat` (supplied with the code) contains the LLRs and decisions produced by the Python library.

D.12 Verification against the Python library

1. **Encoder.** `polar_encode(u)` equals `mod(u*generator_matrix(N),2)` for $N = 2, \dots, 256$.
2. **Book examples.** The SC example (Worked Example 6.3), the SC failure and its SCL correction with path metrics 2.0 and 2.7 (Worked Examples 7.1–7.2), and the CRC examples are reproduced exactly.
3. **Bit-exact decoder.** 300 noisy blocks for each of four configurations were decoded by both libraries from the *same* LLRs:

Configuration	identical decisions	block errors in the set
$N = 64$, $K = 32$, SC	300 / 300	85
$N = 64$, $K = 32$, SCL $L = 4$	300 / 300	43
$N = 128$, $K = 64$, CA-SCL $L = 8$, CRC11	300 / 300	40
$N = 256$, $K = 128$, SCL $L = 8$	300 / 300	34

4. **Construction.** For $N = 1024$ the GA mean LLRs agree with Python to a relative error below 10^{-6}, and the information sets for $K = 128, 256, 512, 768$ are identical.
5. **Statistics.** Independent Monte-Carlo runs (different random streams) give the same curves within statistical accuracy (Fig. D.1).

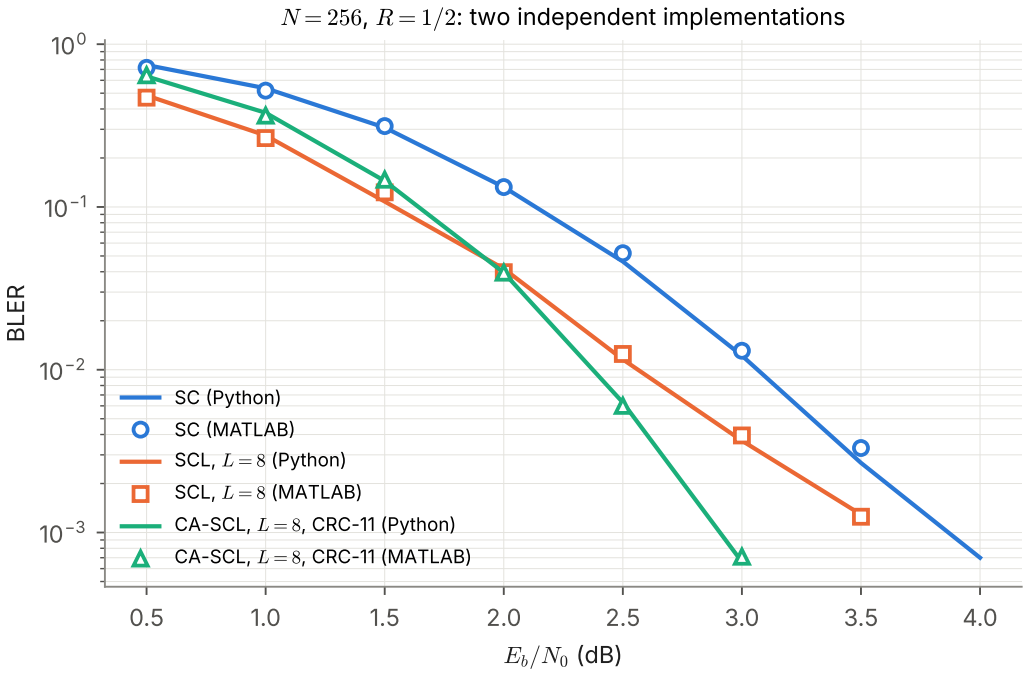


Figure D.1. BLER of the $(256, 128)$ code: MATLAB code (run in GNU Octave, markers) versus the Python library (lines). Points with fewer than 20 block errors are not shown.

Speed

The decoder is vectorised over blocks and list paths, so MATLAB processes about 1000 blocks of the $(256, 128)$ code with $L = 8$ in roughly 2–5 seconds on one core. Use `'batch'` of 1000–2000 blocks. For BLER below 10^{-5}, consider `parfor` over E_b/N_0 points or a MEX

implementation of `node`.

D.13 Common MATLAB mistakes

! Common mistake

- Using book index i directly as a MATLAB index (off by one). Keep information sets as logical masks.
- Writing `x = mod(u*G, 2)` with `u` as a column vector: the book uses *row* vectors, $\mathbf{x} = \mathbf{u}\mathbf{G}_N$.
- Forgetting that `xor` returns `logical`; convert with `double()` before arithmetic such as `1 - 2*c` (the library already does this).
- Calling `sort` with `'descend'` for path metrics: small metrics are good.
- Re-seeding `rng` inside the Monte-Carlo loop, which repeats the same noise.

Summary

The MATLAB library mirrors the Python library line for line in function: one encoder, two construction methods, one decoder for SC/SCL/CA-SCL, a linear CRC and a simulation function. It produces bit-identical decisions to the Python code on the same LLRs, so every worked example and every conclusion of the book holds unchanged for MATLAB users.